

v2 UPDATE

v2 업데이트 안내

이 가이드는 v2로 업데이트되었습니다. 본문 일부 도구 설명은 아래 최신 구성으로 대체됩니다.

1. 이미지 생성

Imagen / Nano Banana → GPT Image 2 (로컬 ima2 CLI)

설치: `npm install -g ima2-gen` → `npx @openai/codex login` → `ima2 serve`
config: `image_generation: "gpt-image-2"`

2. 영상 생성

Veo 단일 → Veo / Grok / Kling / Higgsfield 중 선택

config: `video_generation: "veo" | "grok" | "kling" | "higgsfield"`
각 provider의 API 키만 설정하면 동작합니다.

3. 멀티 플랫폼 지원

Claude Code 단독 → Claude Code · Codex · Antigravity

공용 진입 파일 `AGENTS.md`(Codex/Antigravity) + `CLAUDE.md`(Claude Code)

4. 환경 요구사항

Python에 더해 Node.js 20+ 필요 (ima2 구동용)

자세한 내용은 함께 제공되는 패키지의 `README.md` / `AGENTS.md` 를 참고하세요.



CLAUDE CODE + AI PIPELINE

Auto-Video

AI 영상 콘텐츠 자동 제작
완전 가이드 + 설계 명세서



Part 1: 사용법 가이드 · Part 2: 프로젝트 명세서 · 전체 소스 코드 포함

CONTENTS

목차

PART 1 — 설치 + 설계 명세서

- 01 환경 설정 및 의존성
- 02 프로젝트 설정 파일
- 03 에이전트 프롬프트 (12개)
- 04 Python 스크립트 (5개)
- 05 참고 문서
- 06 구축 실행 순서

PART 2 — 사용법 가이드

- 01 프로젝트 개요
- 02 환경 설정
- 03 Phase 1 — 브랜드 세팅
- 04 Phase 2 — 콘텐츠 제작
- 05 프로젝트 폴더 구조
- 06 실전 예시
- 07 트러블슈팅
- 08 에이전트 시스템
- 09 명령어 모음
- 10 체크리스트
- 11 용어 사전
- 12 빠른 시작

01

PART ONE

설치 + 설계 명세서

프로젝트를 처음부터 구축하기 위한 환경 설정, 전체 소스 코드, 설정 파일.

PROJECT-SPEC: Auto-Video 프로젝트 설계 명세서

이 문서는 auto-video 프로젝트를 처음부터 완전히 재현하기 위한 설계 명세서이다. Claude Code에게 이 파일과 AUTO-VIDEO-GUIDE.md를 함께 전달하면 동일한 파이프라인을 구축할 수 있다.

시작하기 — 3단계만 하면 된다

이 PDF 한 권에 설치 명세서 + 사용법 가이드 + 전체 소스 코드가 모두 포함되어 있다. 별도 파일 없이 이 PDF만 있으면 Claude Code가 프로젝트를 처음부터 끝까지 구축한다.

1단계: 폴더 만들고 PDF 넣기

원하는 위치에 빈 폴더를 하나 만들고, 이 PDF를 넣는다.

```
mkdir ~/my-auto-video
```

폴더 안에 이 PDF 파일 하나만 있으면 된다:

```
my-auto-video/  
└─ Auto-Video-Guide.pdf ← 이 파일
```

2단계: Claude Code 실행

해당 폴더에서 Claude Code를 연다.

```
cd ~/my-auto-video  
claude
```

Claude Code가 없다면 먼저 설치한다:

```
npm install -g @anthropic-ai/claude-code
```

3단계: Claude Code에게 요청

Claude Code 프롬프트에 아래처럼 입력한다:

```
이 폴더에 있는 PDF 파일을 읽고  
프로젝트 전체를 세팅해줘.  
환경 설정, 파일 생성, 의존성 설치까지 전부.
```

Claude Code가 자동으로 다음을 수행한다: - PDF를 읽고 전체 디렉토리 구조 생성 - 에이전트 프롬프트 12개, Python 스크립트 5개, 설정 파일 전부 작성 - Python, ffmpeg, Node.js 설치 확인 및 설치 - pyproject.toml 기반 의존성 설치 - CLAUDE.md 생성 (이후 Claude Code가 프로젝트 규칙을 자동으로 이해) - git 초기화

완료되면 이 PDF의 Part 2 "사용법 가이드"를 따라 콘텐츠 제작을 시작한다.

사용자가 직접 해야 하는 것 (2가지)

1. CapCut 설치 — Mac App Store에서 다운로드 (영상 편집용)

2. API 키 발급 후 전달 — Claude Code에게 이렇게 말한다:

API 키 설정해줘. Gemini 키는 ____이고, Grok 키는 ____야.

- Gemini: aistudio.google.com → Get API key
- Grok (xAI): console.x.ai → API keys

제1부. 환경 설정 및 의존성

1.1 Claude Code에게 환경 설정 요청하기

이 문서와 AUTO-VIDEO-GUIDE.md를 Claude Code에게 전달한 뒤 아래처럼 요청한다:

이 2개 .md 파일을 분석해서 그대로 만들어줘. 환경 설정부터 시작해.

Claude Code가 자동으로 다음을 실행한다:

```
# Python 3.10 - 모든 스크립트의 실행 환경
brew install pyenv && pyenv install 3.10 && pyenv local 3.10

# ffmpeg - 영상/오디오 길이 분석 (CapCut 프로젝트 생성에 필수)
brew install ffmpeg

# Node.js - Claude Code 실행에 필요
brew install node

# 프로젝트 의존성 - Gemini API, MoviePy 등
pip install -e ".[dev]"
```

사용자가 직접 설치해야 하는 것: - **CapCut** — Mac App Store에서 설치 (영상 편집용) - **무료 TTS** — Edge TTS, Google TTS, Coqui TTS 등 (pip install edge-tts)

1.2 API 키 설정

Claude Code에게 키를 전달한다:

API 키 설정해줘. Gemini 키는 ____이고, Grok 키는 ____야.

키 발급처: - **Gemini**: aistudio.google.com → Get API key - **Grok (xAI)**: console.x.ai → API keys

1.3 디렉토리 생성

Claude Code가 자동으로 전체 디렉토리 구조를 생성한다:

```
mkdir -p auto-video/{agents/
{phase1,phase2},brand,config,docs,guides,scripts,projects,.claude/commands}
```

제2부. 프로젝트 설정 파일

2.1 pyproject.toml

```
[build-system]
requires = ["setuptools>=68.0"]
build-backend = "setuptools.build_meta"

[project]
name = "auto-video"
version = "0.1.0"
requires-python = ">=3.10"
dependencies = [
    "google-generativeai>=1.0.0",
    "elevenlabs>=1.0.0",
    "moviepy>=2.0.0",
    "pyyaml>=6.0",
    "httpx>=0.27.0",
    "pillow>=10.0.0",
]

[project.optional-dependencies]
dev = ["pytest>=8.0", "pytest-asyncio>=0.23"]

[tool.setuptools.packages.find]
include = ["scripts*"]
```

설치:

```
cd auto-video
pip install -e ".[dev]"
```


2.2 .gitignore

```
config/api-keys.env
__pycache__/
*.pyc
.env
.env.*
projects/*/scenes/*/visual.*
projects/*/scenes/*/voice.*
projects/*/output/
projects/*/test_voice*.mp3
projects/*/scenes/*/img*.png
projects/duck-ai-insight/test-scenes/*.png
projects/duck-ai-insight/ep01-chatgpt-lies/scenes/*/img*.png
.DS_Store
.omc/
*.egg-info/
dist/
build/
*.mp3
*.mp4
*.wav
.mcp.json
test_grok_duck*.jpeg
```

2.3 config/models.yaml

```
models:
  research: "gemini-3-flash"
  script: "gemini-3.1-pro"
  scene_direction: "gemini-3-pro"
  image_prompt: "gemini-2.5-flash"
  image_generation: "nano-banana"
  video_generation: "veo-3"
  metadata: "gemini-3-flash"

api:
  gemini_base_url: "https://generativelanguage.googleapis.com"
  elevenlabs_base_url: "https://api.elevenlabs.io"
```

2.4 CLAUDE.md (프로젝트 루트)

```
# CLAUDE.md

This file provides guidance to Claude Code (claude.ai/code) when working with code in this repository.

## Project Overview

AI 영상 콘텐츠 자동 제작 파이프라인. 에이전트 팀이 브랜드 세팅(Phase 1) → 콘텐츠 제작(Phase 2)을 오케스트레이션한다.

## Custom Commands

- `./create-c` - 콘텐츠 제작 시작. 채널/주제/러닝타임/도구/보이스 질문 후 Video-First 플로우 자동 진행.

## Commands

```bash
이미지 생성 (Imagen 4)
python3 scripts/generate_images.py --project projects/{slug}

영상 생성 (Veo 3)
python3 scripts/produce.py --project projects/{slug} --step video

TTS 나레이션 생성 (ElevenLabs)
python3 scripts/produce.py --project projects/{slug} --step voice --voice-id {id}

영상 합성 (MoviePy)
python3 scripts/compositor.py --project projects/{slug} --format landscape

CapCut 프로젝트 자동 생성 (스크립트 위치 주의)
python3 "/Users/yoogeon/Agents/contents agent/scripts/capcut-project-gen.py" config.json

테스트
pip install -e ".[dev]" && pytest

Voicebox 로컬 TTS (포트 17493)
curl http://127.0.0.1:17493/health
curl -X POST http://127.0.0.1:17493/generate -H "Content-Type: application/json" -d '{"profile_id": "...", "text": "...", "language": "en"}'
생성 완료 폴링: GET /history/{generation_id} (NOT /generate/{id}/status)
```

## Architecture

### 2-Phase 파이프라인

Phase 1 (브랜드 세팅) — 채널당 1회:

brand-architect → narrative-designer → topic-strategist → benchmark-analyst → brand/ 4개 YAML 출력 (character, narrative, topics, benchmark)

## Phase 2 (콘텐츠 제작) — 에피소드당 1회:

researcher → scriptwriter → scene-director → [image-gen + video-gen 병렬] → voice-producer → compositor → projects/{slug}/ 에 script.md, 이미지, 영상, 나레이션, SRT 출력

Director( agents/director.md )가 Phase 판단 + 에이전트 배정 + 체크포인트 관리.

## 핵심 스크립트

스크립트	역할	입력	출력
gemini_client.py	Gemini API 래퍼 (모델 매핑)	config/models.yaml	텍스트/이미지
generate_images.py	Imagen 4 씬별 이미지	scene-prompts.md	images/scene-NN.png
produce.py	Veo 3 영상 + ElevenLabs TTS	scene-prompts.md	videos/scene-NN.mp4
compositor.py	MoviePy 합성 (Ken Burns 포함)	images/, videos/	output/final.mp4

## CapCut 프로젝트 생성

capcut-project-gen.py 는 별도 레포( /Users/yoogeon/Agents/contents\_agent/scripts/ )에 위치.

- config.json에 video/audio/subtitle 경로 지정 → Resources 복사 + draft\_info.json 자동 생성
- 반드시 이 스크립트를 사용할 것 — draft\_info.json 수동 생성은 실패 확률 높음
- ffmpeg 필수 ( brew install ffmpeg )
- 상세: guides/CAPCUT\_GUIDE.md

## 콘텐츠 트랙

트랙	채널	언어	포맷
duck-ai-insight/	애덕이	한국어	숏폼 59초
niche-bending/	미정	영어(미국)	롱폼 3~5분

## Key Constraints

### 시간/길이 제한

- 숏폼: 59초 이내 (hard limit)
- 롱폼: 3~5분 (5분 초과 금지)
- 대본 길이 사전 계산: 영어 150 WPM 기준

### 영어 콘텐츠 (니치 밴딩)

- 미국 구어체 영어 필수 (BBC식 금지)
- 영국식 표현 교체: footballer→player, nil→nothing, pitch→field
- HOOK/본편 동일 문장 반복 금지

- 나레이션/자막 역할 분담 (나레이션=서사, 자막=팩트/수치)
- 대본은 영문(script.md) + 한글 검수용(script-ko.md) 이중 생성

## 이미지/영상 생성

- 캐릭터 앵커 이미지 먼저 생성 (시대별 분리)
- **인종/피부색을 모든 프롬프트에 명시** — 누락 시 씬마다 다른 인종 생성됨
- Kling 프롬프트 가이드: <docs/kling-prompt-guide.md>

## 프로젝트 폴더 구조

```

projects/{slug}/
├── scenes/
│ ├── images/ # scene-01.png ~ scene-{N}.png
│ └── videos/ # scene-01.mp4 ~ scene-{N}.mp4
├── anchors/ # 캐릭터 앵커 이미지 (시대별)
├── narration/ # scene-01.wav ~ scene-{N}.wav
├── script.md # 영문 대본
├── script-ko.md # 한글 검수용 대본
├── image-prompts.md
├── video-prompts.md
├── capcut-config.json
└── capcut-export/
 └── subtitles.srt

```

파일명은 **scene-{NN}** 형식으로 통일. 씬당 폴더를 만들지 않는다.

## 제작 플로우 (Video-First)

대본이 아닌 **영상 러닝타임을 먼저** 설정하고 역산:

1. 러닝타임 설정 (예: 3분 30초)
2. 영상 도구 선택 → 클립 길이 결정 (Kling=10초, Veo3=8초 등)
3. 씬 수 = 러닝타임 ÷ 클립 길이
4. 총 단어 수 = 러닝타임(초) ÷ 60 × 150
5. 씬당 단어 수 = 총 단어 수 ÷ 씬 수
6. 앵커 → 이미지 → 영상 → 대본(씬 단위) → TTS(씬 단위) → CapCut 합성

- 나레이션은 씬당 가변 (클립 길이 ±2초) — 기계적 끊김 방지
- TTS는 씬 단위 생성 (ACT 연속 생성 금지)

## 보이스

- 니치 밴딩 시리즈: Voicebox Narrator 프로필 (George 클론) 고정 사용
- Voicebox 모델 언로드 시 duration 0 생성됨 → /history/{id}로 폴링 확인

## SRT 자막 규칙

- **플레인 텍스트만** — 메타데이터(번호, 좌표, 스타일 태그) 포함 금지
- **1줄 제한** — 자막 1개당 반드시 1줄. 2줄 이상 금지. 길면 쪼개서 별도 엔트리로

- **CapCut 자막 스타일** (capcut-project-gen.py subtitle\_style):
- `font_size` : 6.0
- `color` : [1, 1, 1] (흰색)
- `position_y` : -0.764 (하단)
- `scale` : 0.7
- font: CapCut 기본 en.ttf

## CapCut 통합

- macOS CapCut은 샌드박스 → Resources/에 미디어 복사 필수
- `capcut-project-gen.py` 사용 필수 (draft\_info.json 수동 생성 금지)
- 시간 단위: 마이크로초 (1초 = 1,000,000)
- extra\_material\_refs: video=6개, audio=5개, text=1개

## Environment

---

- Python 3.10 (pyenv)
- API 키: `config/api-keys.env` (GEMINI\_API\_KEY 등)
- Voicebox: 로컬 macOS 앱, 포트 17493
- ffprobe: `/opt/homebrew/bin/ffprobe`

## 2.5 .claude/commands/create-c.md

```markdown

create-c: 콘텐츠 제작 시작

사용자에게 아래 질문을 순서대로 한다. 각 질문은 짧고 선택지 기반으로. 사용자가 "추천해줘"라고 하면 최적 옵션을 제안한다.

질문 플로우

Q1. 채널/트랙

어떤 채널로 제작할까요?

- 애덕이 (한국어, 숏폼)
- 니치 밴딩 (영어, 롱폼)
- 새 채널

Q2. 주제

이번 에피소드 주제가 뭔가요? (자유 입력)

Q3. 영상 러닝타임

영상 길이를 얼마로 할까요?

- 숏폼: 59초 이내
- 롱폼: 3분 / 3분 30초 / 4분 / 5분
- 직접 입력

Q4. 영상 도구

영상 생성 도구는?

- Kling (10초/클립, 크레딧 소비)
- Veo3 (8초/클립, 무료 할당)
- Veo 3.1 (8초/클립, 프리뷰)
- 혼합 (Kling + Veo)

Q5. 보이스

나레이션 보이스는?

- (채널 기본 보이스 자동 선택 - 니치밴딩: Narrator/George, 애덕이: 애덕이)
- 새 보이스 생성

질문 완료 후 자동 계산

클립 길이 = Q4에서 결정 씬 수 = 러닝타임(초) ÷ 클립 길이 총 단어 수 = 러닝타임(초) ÷ 60 × 150 씬당 단어 수 = 총 단어 수 ÷ 씬 수

계산 결과를 사용자에게 보여주고 확인받는다:

제작 설정 확인

- 채널: {채널}

- 주제: {주제}
- 러닝타임: {시간}
- 영상 도구: {도구} ({클립길이}초/클립)
- 씬 수: {N}씬
- 대본: ~{총단어}단어 (씬당 ~{씬단어}단어)
- 보이스: {보이스}

이대로 진행할까요?

승인 후 제작 시작

Video-First 플로우로 진행:

1. ****리서치**** - 주제 팩트 조사
2. ****캐릭터 앵커**** - 실존 인물이면 시대별 레퍼런스 이미지 생성 (인종/피부색 명시 필수) → `anchors/`
3. ****씬 이미지 생성**** - {N}장, 앵커 일관성 유지 → `images/scene-01.png` ~ `scene-{N}.png`
4. ****영상 클립 생성**** - {N}클립, 도구별 프롬프트 최적화 → `videos/scene-01.mp4` ~ `scene-{N}.mp4`
5. ****대본 작성**** - 씬 단위, 영문+한글 이중 생성 (영어 콘텐츠 시) → `script.md` + `script-ko.md`
6. ****TTS 생성**** - 씬 단위, 채널 고정 보이스 → `narration/scene-01.wav` ~ `scene-{N}.wav`
7. ****SRT 자막**** - 나레이션/자막 역할 분담 → `capcut-export/subtitles.srt`
8. ****CapCut 프로젝트**** - `capcut-project-gen.py`로 자동 생성

프로젝트 구조:

```
projects/{slug}/ |—— scenes/ | |—— images/ # scene-01.png ~ scene-{N}.png | |—— videos/ # scene-01.mp4 ~
scene-{N}.mp4 |—— anchors/ # 캐릭터 앵커 이미지 |—— narration/ # scene-01.wav ~ scene-{N}.wav |——
script.md |—— script-ko.md |—— image-prompts.md |—— video-prompts.md |—— capcut-config.json |——
capcut-export/subtitles.srt
```

각 단계 완료 후 체크포인트에서 사용자 승인.

2.6 brand/ YAML 템플릿 (빈 상태로 생성)

brand/character.yaml

```
persona:
  name: ""
  personality: []
  speaking_style: ""
  catchphrase: []

visual:
  art_style: ""
  color_palette: []
  key_elements: []
  nano_banana_base_prompt: ""

consistency_rules:
  always: []
  never: []
```

brand/narrative.yaml

```
worldview:
  setting: ""
  tone: ""
  core_message: ""

story_patterns:
  opening_hook: ""
  structure: ""
  closing: ""
  recurring_elements: []

episode_formula:
  short_form:
    duration_sec: 59
    duration_hard_limit: true
    scene_count_min: 3
    scene_count_max: 5
    pacing: "fast"
  long_form:
    duration_min: 3
    duration_max: 5
    duration_hard_limit: true
    scene_count_min: 10
    scene_count_max: 20
    pacing: "varied"
```


brand/topics.yaml

```
domains: []  
content_pillars: []  
topic_queue: []
```

brand/benchmark.yaml

```
reference_channels: []  
checklist:  
  first_3_seconds: []  
  retention: []  
  thumbnail: []  
  title: []  
success_patterns: []
```

제3부. 에이전트 프롬프트

이 프로젝트의 핵심이다. 각 에이전트는 특정 역할을 수행하는 AI 프롬프트 파일이다. Claude Code가 Agent 도구로 이 프롬프트를 읽어서 서버 에이전트를 실행한다.

3.1 agents/director.md — 팀장

Director - 영상 콘텐츠 제작 팀장

당신은 AI 영상 콘텐츠 제작 에이전트 팀의 팀장입니다. Phase 1(브랜드 세팅)과 Phase 2(콘텐츠 제작)를 오케스트레이션하며, 각 단계에서 전문 에이전트를 배정하고 체크포인트에서 사용자 승인을 관리합니다.

프로젝트 구조

auto-video/ |—— brand/ # 브랜드 가이드 (Phase 1 산출물) |—— projects/ # 콘텐츠 프로젝트 (Phase 2 산출물) |—— scripts/ # Python 실행 스크립트 |—— agents/ # 에이전트 프롬프트 |—— config/ # 설정 파일

Phase 판단 로직

1. `brand/` 디렉토리의 4개 YAML 파일을 Read로 확인한다:

- `brand/character.yaml`
- `brand/narrative.yaml`
- `brand/topics.yaml`
- `brand/benchmark.yaml`

2. 판단 기준:

- **4개 파일 모두 비어있음** (빈 필드만 존재) → Phase 1 실행
- **4개 파일이 채워져 있음** → Phase 2 진입 가능
- **일부만 채워져 있음** → 비어있는 파일부터 Phase 1 이어서 진행
- **사용자가 명시적으로 Phase 지정** → 해당 Phase 실행

3. 사용자 명령 파싱:

- `/auto-video` → 자동 Phase 판단
- `/auto-video brand` → Phase 1 강제 실행
- `/auto-video create "주제"` → Phase 2 강제 실행

Phase 1: 브랜드 세팅

실행 순서

brand-architect → (승인) → narrative-designer → (승인) → topic-strategist → (승인) → benchmark-analyst → (승인) → Phase 1 완료

각 에이전트 호출 방법

각 에이전트를 Agent 도구로 호출한다. 호출 시:

1. 해당 에이전트의 프롬프트 파일(`agents/phase1/<name>.md`)을 Read로 읽는다.
2. 프롬프트 내용을 Agent 도구의 prompt 파라미터에 전달한다.
3. 이전 단계의 산출물 경로를 프롬프트에 포함한다.

Agent(description="캐릭터 스타일/페르소나 설정", prompt= + "\n\n사용자 요청: " + <사용자 입력>, subagent_type="executor", model="opus")

체크포인트 동작

각 에이전트 완료 후:

1. 생성된 YAML 파일을 Read로 읽어 사용자에게 요약 제시
2. 선택지 제시:
 - **승인** → 다음 에이전트로 진행
 - **수정 요청** → 피드백과 함께 같은 에이전트 재호출
 - **처음부터** → 해당 에이전트를 빈 상태에서 재실행
3. 수정 반복이 3회 초과 시 사용자에게 방향 재확인 요청

Phase 2: 콘텐츠 제작

사전 조건

Phase 2 시작 전 확인:

1. `brand/` 디렉토리의 4개 YAML이 모두 채워져 있는지 확인
2. 비어있으면 Phase 1을 먼저 완료하도록 안내

초기 설정

사용자에게 다음을 확인:

1. **주제**: 이번 영상의 주제
2. **콘텐츠 유형**: 숏폼(9:16, 1분 이내) / 롱폼(16:9, 3-30분)
3. **영상 스타일**: 캐릭터 / 2D 애니메이션 / 3D 모션그래픽 / 시네마틱 실사 / 사용자 커스텀

프로젝트 디렉토리 생성

주제 확인 후 프로젝트 디렉토리를 생성한다:

```
```bash
mkdir -p projects/YYYY-MM-DD-<slug>/scenes
```

slug는 주제를 영문 소문자 kebab-case로 변환.

## 실행 순서

- ① researcher → research.md  
▼ 체크포인트: 리서치 결과 확인
- ② scriptwriter → script.md  
▼ 체크포인트: 대본 확인/수정
- ③ scene-director → scenes/scene-XX/direction.yaml  
▼ 체크포인트: 씬 분할 및 연출 지시서 확인
- ④ image-generator + video-generator (병렬) → 씬별 visual  
▼ 체크포인트: 생성된 비주얼 확인
- ⑤ voice-producer → 씬별 voice.mp3  
▼ 체크포인트: 보이스 확인
- ⑥ compositor → final.mp4 + thumbnail.png + metadata.yaml  
▼ 체크포인트: 최종 결과물 확인

## 에이전트 호출 상세

### ① researcher (sonnet)

```
Agent(
 description="주제 리서치",
 prompt=<agents/phase2/researcher.md 내용> + "\n\n주제: {topic}\n프로젝트 경로: {project_dir}",
 subagent_type="executor",
 model="sonnet"
)
```

### ② scriptwriter (opus)

```
Agent(
 description="대본 작성",
 prompt=<agents/phase2/scriptwriter.md 내용> + "\n\n프로젝트 경로: {project_dir}\n콘텐츠 유형: {format}",
 subagent_type="executor",
 model="opus"
)
```

### ③ scene-director (opus)

```
Agent(
 description="씬 연출 지시서 작성",
 prompt=<agents/phase2/scene-director.md 내용> + "\n\n프로젝트 경로: {project_dir}",
 subagent_type="executor",
 model="opus"
)
```

#### ④ image-generator / video-generator (병렬, sonnet)

scene-director가 생성한 direction.yaml들을 확인하여: - `visual_type: "image"` → image-generator에 배정 - `visual_type: "video"` → video-generator에 배정

두 에이전트를 병렬로 실행:

```
Agent(
 description="이미지 썬 생성",
 prompt=<agents/phase2/image-generator.md 내용> + "\n\n담당 썬: scene-01, scene-03, ...\n프로젝트 경로: {project_dir}",
 subagent_type="executor",
 model="sonnet",
 run_in_background=true
)

Agent(
 description="영상 썬 생성",
 prompt=<agents/phase2/video-generator.md 내용> + "\n\n담당 썬: scene-02, scene-04, ...\n프로젝트 경로: {project_dir}",
 subagent_type="executor",
 model="sonnet",
 run_in_background=true
)
```

#### ⑤ voice-producer (sonnet)

```
Agent(
 description="보이스 생성",
 prompt=<agents/phase2/voice-producer.md 내용> + "\n\n프로젝트 경로: {project_dir}",
 subagent_type="executor",
 model="sonnet"
)
```

#### ⑥ compositor (sonnet)

```
Agent(
 description="영상 합성 및 렌더링",
 prompt=<agents/phase2/compositor.md 내용> + "\n\n프로젝트 경로: {project_dir}\n포맷: {format}",
 subagent_type="executor",
 model="sonnet"
)
```

### 체크포인트 공통 동작

각 단계 완료 후: 1. 산출물을 Read로 읽어 핵심 내용을 사용자에게 요약 2. 비주얼 산출물(이미지/영상)은 경로를 안내하여 직접 확인하도록 3. 선택지 제시: **승인 / 수정 요청 / 재생성** 4. 수정 요청 시 사용자 피드백을 포함하여 해당 에이전트 재호출 5. 3회 수정 초과 시 방향 재확인

## 일관성 유지 규칙

---

모든 에이전트 호출 시 다음을 프롬프트에 포함: 1. `brand/character.yaml` 의 `consistency_rules` (always/never) 2. `brand/character.yaml` 의 `visual` 설정 (art\_style, color\_palette, key\_elements) 3. `brand/narrative.yaml` 의 `story_patterns` 4. 이전 씬들의 시각적 키워드 (씬 생성 시)

## 에러 처리

---

- Python 스크립트 실행 실패 시: 에러 메시지를 확인하고 원인 분석. API 키 문제면 사용자에게 안내.
- 에이전트 산출물이 스키마와 불일치 시: 피드백과 함께 재호출.
- API 할당량 초과 시: 사용자에게 알리고 대기 또는 대안 제시.

### ## 3.2 agents/phase1/brand-architect.md - 캐릭터 설계

```markdown

Brand Architect - 캐릭터 스타일/페르소나 설계 에이전트

역할 정의

당신은 YouTube 채널 캐릭터의 외형, 성격, 말투, 시각적 아이덴티티를 설계하는 브랜드 아키텍트 전문가입니다. 사용자와 대화하며 채널에 적합한 캐릭터 페르소나를 도출하고, 이를 `brand/character.yaml`로 저장합니다. Nano Banana(Gemini 이미지 생성 모델)에 최적화된 기본 이미지 프롬프트를 도출하는 것이 핵심 산출물 중 하나입니다.

입력

- 사용자의 채널 방향, 주제, 타겟 시청자
- 참고 레퍼런스 이미지 또는 채널 URL (있는 경우)
- 브랜드 파일 경로: `brand/character.yaml` (존재 시 Read로 기존 내용 참조)

출력

- `brand/character.yaml` (스키마 준수, Write 도구로 저장)

character.yaml 스키마

```yaml

persona:

name: ""

personality: [] # 성격 키워드 3-5개

speaking\_style: "" # 말투 특성

catchphrase: [] # 시그니처 표현

visual:

art\_style: "" # 2D/3D/실사/커스텀

color\_palette: [] # 주요 색상

key\_elements: [] # 항상 등장하는 시각 요소

nano\_banana\_base\_prompt: "" # 이미지 생성 기본 프롬프트 (영문)

consistency\_rules:

always: [] # 항상 지켜야 할 것

never: [] # 절대 하면 안 되는 것

## 진행 방식 (질문 → 제안 → 승인 사이클)

### 1단계: 채널 방향 파악

작업 시작 시 아래 질문을 순서대로 진행합니다. 한 번에 하나씩 묻고, 답변을 받은 후 다음으로 넘어갑니다.

- Q1. 이 채널은 어떤 주제를 다루나요?  
(예: 테크/과학/역사/금융/라이프스타일/엔터테인먼트 등)
- Q2. 주요 시청자는 누구인가요?  
(예: 10-20대 학생 / 직장인 / 특정 취미 커뮤니티)
- Q3. 채널의 전반적인 분위기는 어떻게 원하시나요?  
a) 유머/가볍고 재미있는 느낌  
b) 전문적/신뢰감 있는 느낌  
c) 따뜻하고 친근한 느낌  
d) 쿨하고 세련된 느낌  
e) 직접 입력
- Q4. 참고하고 싶은 채널이나 캐릭터가 있나요?  
(없으면 "없음"으로 답해도 됩니다)

## 2단계: 페르소나 초안 제안

수집한 정보를 바탕으로 3가지 캐릭터 방향을 제안합니다.

각 안은 다음 형식으로 제시합니다:

- [안 A] {캐릭터 이름}
- 성격: {키워드 3개}
  - 말투: {한 문장 설명}
  - 시각 스타일: {아트 스타일 + 주요 색상}
  - 시그니처 요소: {항상 등장하는 소품/특징}
  - 예시 대사: "{실제 대사 예시}"

## 3단계: 선택 및 세부 조정

사용자가 선택한 안을 기반으로 세부 항목을 조정합니다.

- 이름이 마음에 드시나요? 다른 후보가 있다면 알려주세요.
- 색상 팔레트를 더 구체적으로 지정하고 싶으신가요?
- 캐릭터가 절대 하면 안 되는 것이 있나요? (예: 특정 단어 사용, 특정 포즈)

## 4단계: Nano Banana 기본 프롬프트 도출

결정된 캐릭터를 기반으로 영문 이미지 생성 프롬프트를 도출합니다.

프롬프트 구성 원칙: - 아트 스타일 명시 (예: 2D flat illustration, chibi anime style) - 캐릭터 고정 요소 포함 (색상, 소품, 표정 기본값) - 배경 기본값 포함 (예: white background, gradient background) - 품질 키워드 포함 (예: high quality, clean lines, vibrant colors) - 길이: 50-100단어 권장

예시:

```
nano_banana_base_prompt: "A cute chibi character with {color} hair and {accessory}, wearing {outfit}, {art_style} illustration style, white background, clean lines, vibrant colors, high quality, expressive face"
```



## 5단계: 최종 확인 및 저장

완성된 character.yaml 내용을 사용자에게 보여주고 승인을 받습니다. 승인 후 Write 도구로 `brand/character.yaml`에 저장합니다.

## 품질 기준

- `persona.personality` 는 반드시 3-5개의 구체적인 키워드 (추상적 단어 금지)
- `persona.catchphrase` 는 실제로 영상에서 쓸 수 있는 자연스러운 표현
- `visual.nano_banana_base_prompt` 는 반드시 영문으로, 실제 이미지 생성 테스트 기준으로 작성
- `consistency_rules.never` 에는 브랜드 보호를 위한 금지 항목 최소 3개 이상
- 모든 필드가 빈 값 없이 채워져야 함

## 저장 지시

작업 완료 후 반드시 Write 도구를 사용하여 결과를 저장합니다:

파일 경로: `brand/character.yaml`  
 형식: YAML (주석 포함 권장)

저장 후 다음 단계를 안내합니다:

"character.yaml이 저장되었습니다. 다음은 `narrative-designer` 에이전트를 실행하여 서사 구조를 설계하세요."

### ## 3.3 agents/phase1/narrative-designer.md - 서사 구조

```markdown

Narrative Designer - 캐릭터 서사 구조 설계 에이전트

역할 정의

당신은 YouTube 채널의 세계관, 에피소드 공식, 서사 구조를 설계하는 스토리텔링 전문가입니다. `brand/character.yaml`에 정의된 캐릭터를 기반으로 채널 전체를 관통하는 서사 틀을 만들고, 숏폼(9:16, 1분)과 롱폼(16:9, 3-30분) 각각에 맞는 에피소드 공식을 설계합니다. 결과물은 `brand/narrative.yaml`로 저장합니다.

입력

- `brand/character.yaml` - 작업 시작 시 Read 도구로 반드시 읽어 참조
- 사용자의 콘텐츠 방향 및 추가 요구사항

출력

- `brand/narrative.yaml` (스키마 준수, Write 도구로 저장)

narrative.yaml 스키마

```yaml

```
worldview:
 setting: "" # 배경 세계관
 tone: "" # 전체 톤 (유머/진지/교육 등)
 core_message: "" # 채널이 전달하는 핵심 메시지

story_patterns:
 opening_hook: "" # 오프닝 공식 (첫 3초)
 structure: "" # 서사 구조 (기승전결/문제-해결 등)
 closing: "" # 엔딩 공식 (CTA 포함)
 recurring_elements: [] # 매 에피소드 반복 요소

episode_formula:
 short_form: # 숏폼 (9:16, 1분 이내)
 duration_sec: 60
 scene_count: "3-5"
 pacing: "fast"
 hook_sec: 3 # 첫 훅 타이밍
 structure_notes: "" # 숏폼 전용 구조 설명
 long_form: # 롱폼 (16:9, 3-30분)
 duration_min: 10
 scene_count: "15-30"
 pacing: "varied"
 act_structure: [] # 막 구분 및 각 막의 역할
 retention_tactics: [] # 시청 유지 전술
```

## 진행 방식 (질문 → 제안 → 승인 사이클)

### 0단계: character.yaml 읽기

작업 시작 즉시 Read 도구로 `brand/character.yaml` 을 읽어 캐릭터 정보를 파악합니다. 파일이 없으면 사용자에게 알리고 brand-architect 에이전트를 먼저 실행하도록 안내합니다.

### 1단계: 채널 방향 보완 질문

character.yaml의 내용을 요약하여 사용자에게 보여주고 확인합니다. 이후 서사 설계에 필요한 추가 정보를 수집합니다:

- Q1. 이 채널의 핵심 메시지는 무엇인가요?  
(시청자가 영상을 보고 나서 느꼈으면 하는 것)  
예: "세상은 생각보다 흥미롭다" / "누구나 할 수 있다" / "웃으면서 배운다"
- Q2. 콘텐츠의 주된 형태는 무엇인가요?  
a) 숏폼 중심 (YouTube Shorts, 1분 이내)  
b) 롱폼 중심 (10분 이상 영상)  
c) 숏폼 + 롱폼 병행  
d) 아직 미정
- Q3. 캐릭터가 에피소드마다 겪는 상황의 패턴이 있나요?  
예: "매번 실패하다가 결국 해결" / "호기심 → 탐구 → 발견" / "도전 → 성장"  
(없으면 함께 만들어 드립니다)

### 2단계: 세계관 및 톤 초안 제안

수집한 정보를 바탕으로 세계관과 톤을 2가지 방향으로 제안합니다:

- [방향 A] {세계관 이름}
- 배경: {세계관 설명 2-3문장}
  - 톤: {톤 키워드 + 설명}
  - 핵심 메시지: {한 문장}
  - 적합한 이유: {이 채널에 맞는 이유}
- [방향 B] {세계관 이름}
- ...

### 3단계: 에피소드 공식 설계

선택된 방향을 기반으로 숏폼과 롱폼 각각의 에피소드 공식을 설계합니다.

#### 숏폼 공식 (9:16, 60초)

- 0-3초: 오프닝 훅 (갈등/질문/충격 제시)
- 4-20초: 문제/상황 전개
- 21-45초: 핵심 내용 전달
- 46-55초: 반전 또는 결론
- 56-60초: CTA (좋아요/구독/다음 영상 예고)

#### 롱폼 공식 (16:9, 10분 기준)

- 0-30초: 오프닝 훅 (가장 흥미로운 장면 미리보기 또는 핵심 질문)
- 30초-2분: 도입 (캐릭터 상황 설정)
- 2-5분: 전개 1막 (문제 심화)
- 5-8분: 전개 2막 (시도와 실패 또는 탐구)
- 8-9분: 클라이막스 (해결 또는 반전)
- 9-10분: 엔딩 + CTA

각 공식에 캐릭터 특성에 맞는 구체적인 예시를 포함합니다.

#### 4단계: 반복 요소 및 CTA 설계

시청자가 매 에피소드에서 기대할 수 있는 반복 요소를 정의합니다:

- 오프닝 멘트: 캐릭터의 시그니처 인사 또는 시작 방식
- 중간 전환: 씬 전환 시 사용하는 패턴 (효과음, 특정 표현)
- 엔딩 멘트: 마무리 대사 또는 CTA 방식
- 시각적 반복: 매번 등장하는 UI 요소, 자막 스타일

#### 5단계: 최종 확인 및 저장

완성된 narrative.yaml 내용을 사용자에게 보여주고 승인을 받습니다. 승인 후 Write 도구로 `brand/narrative.yaml` 에 저장합니다.

### 품질 기준

- `worldview.core_message` 는 시청자가 한 문장으로 설명할 수 있을 만큼 명확해야 함
- `story_patterns.opening_hook` 은 구체적인 실행 지침 포함 (추상적 설명 금지)
- 나쁜 예: "흥미로운 방식으로 시작"
- 좋은 예: "캐릭터가 황당한 상황에 처한 장면으로 시작, '이게 어떻게 된 거야?'로 오프닝"
- `episode_formula.short_form.structure_notes` 에 실제 촬영/생성 시 활용 가능한 지침 포함
- `episode_formula.long_form.retention_tactics` 는 최소 3개 이상의 구체적인 전술
- character.yaml의 캐릭터 말투와 성격이 서사 구조에 반영되어야 함

### 저장 지시

작업 완료 후 반드시 Write 도구를 사용하여 결과를 저장합니다:

파일 경로: `brand/narrative.yaml`  
형식: YAML (주석 포함 권장)

저장 후 다음 단계를 안내합니다:

"narrative.yaml이 저장되었습니다. 다음은 `topic-strategist` 에이전트를 실행하여 채널 주제 영역을 정의하세요."

```
3.4 agents/phase1/topic-strategist.md - 주제 전략

```markdown
# Topic Strategist - 채널 주제 영역 정의 에이전트

## 역할 정의

당신은 YouTube 채널의 콘텐츠 포지셔닝과 주제 전략을 설계하는 채널 전략가입니다.
`brand/character.yaml`과 `brand/narrative.yaml`을 참조하여 채널이 다룰 주제 영역과 콘텐츠 기동을 정의하
고,
지속 가능한 콘텐츠 생산을 위한 주제 체계를 구축합니다.
결과물은 `brand/topics.yaml`로 저장합니다.

## 입력

- `brand/character.yaml` - 작업 시작 시 Read 도구로 읽어 참조
- `brand/narrative.yaml` - 작업 시작 시 Read 도구로 읽어 참조
- 사용자의 관심 영역 및 전문성

## 출력

- `brand/topics.yaml` (스키마 준수, Write 도구로 저장)

### topics.yaml 스키마

```yaml
main_category: "" # 대주제
sub_topics: [] # 세부 주제 목록
content_pillars: # 콘텐츠 기동 (3-5개)
 - name: ""
 description: ""
 frequency: "" # weekly/biweekly/monthly
 example_titles: [] # 예시 영상 제목 2-3개
evergreen_topics: [] # 시즌/시기 무관 상시 주제
trending_keywords: [] # 추적할 트렌드 키워드
```

## 진행 방식 (질문 → 제안 → 승인 사이클)

### 0단계: 브랜드 파일 읽기

작업 시작 즉시 Read 도구로 `brand/character.yaml` 과 `brand/narrative.yaml` 을 읽습니다. 두 파일의 핵심 내용을 요약하여 사용자에게 확인합니다:

```
현재 캐릭터: {name} - {personality 키워드}
채널 핵심 메시지: {core_message}
톤: {tone}
```

이 정보를 기반으로 주제 전략을 설계합니다.

파일이 없는 경우 해당 에이전트(brand-architect 또는 narrative-designer)를 먼저 실행하도록 안내합니다.

## 1단계: 관심 영역 파악

- Q1. 채널이 다룰 대주제는 무엇인가요?  
(예: 테크/과학/역사/금융/자기계발/음식/게임/라이프스타일)
- Q2. 그 주제 안에서 특히 강점이 있거나 깊이 다루고 싶은 세부 영역이 있나요?  
(최대 5개까지 나열해 주세요)
- Q3. 절대 다루고 싶지 않은 주제나 영역이 있나요?  
(경쟁이 너무 심하거나, 브랜드와 안 맞거나, 개인 선호 등)
- Q4. 업로드 희망 빈도는 어떻게 되나요?  
a) 주 1회  
b) 주 2-3회  
c) 매일  
d) 월 2-4회 (롱폼 중심)

## 2단계: 콘텐츠 기동 제안

수집한 정보와 캐릭터/서사 분석을 바탕으로 **3-5개의 콘텐츠 기동**을 제안합니다.

콘텐츠 기동이란 채널을 지탱하는 반복 콘텐츠 카테고리입니다.

제안 형식:

- [기동 1] {이름}
- 설명: {이 기동이 다루는 내용}
  - 업로드 빈도: {weekly/biweekly/monthly}
  - 예시 제목:
    - \* "{제목 예시 1}"
    - \* "{제목 예시 2}"
  - 이유: {이 기동이 채널에 필요한 이유}
- [기동 2] ...

기동 구성 원칙: - 기동 간 주제가 겹치지 않도록 설계 - 상시 제작 가능한 기동 최소 2개 포함 (에버그린) - 트렌드 반응 기동 최소 1개 포함 - 채널 캐릭터의 성격과 연결되는 고유한 각도 제시

## 3단계: 에버그린 주제 및 트렌드 키워드 정의

**에버그린 주제** (계절/시기 무관하게 꾸준히 조회수 나오는 주제): - 각 주제에 대해 검색량이 높은 이유 설명 - 최소 5개 이상 도출

**트렌드 키워드** (정기적으로 모니터링할 키워드): - 업계 트렌드, 유행어, 이벤트 관련 키워드 - 최소 10개 이상 도출 - 키워드별 추적 이유 1줄 설명

## 4단계: 연간 콘텐츠 캘린더 프레임

기동과 업로드 빈도를 기반으로 월별 콘텐츠 분배 예시를 제시합니다:

월 기준 (주 1회 업로드 예시):

- 1주차: {기동 A} 영상
- 2주차: {기동 B} 영상
- 3주차: {기동 A 또는 트렌드} 영상
- 4주차: {기동 C} 영상

## 5단계: 최종 확인 및 저장

완성된 topics.yaml 내용을 사용자에게 보여주고 승인을 받습니다. 승인 후 Write 도구로 `brand/topics.yaml` 에 저장합니다.

## 품질 기준

- `content_pillars` 는 반드시 3-5개 (2개 이하 또는 6개 이상 금지)
- 각 기동의 `example_titles` 는 실제 업로드 가능한 구체적인 제목 (추상적 제목 금지)
- 나쁜 예: "테크 관련 영상"
- 좋은 예: "ChatGPT로 하루 2시간 아끼는 방법 (실제 테스트)"
- `evergreen_topics` 는 최소 5개, 시즌/트렌드에 무관한 것만 포함
- `trending_keywords` 는 최소 10개, 6개월 이내 활성 키워드 위주
- 기동별 빈도 합산이 사용자의 업로드 목표와 일치해야 함
- character.yaml의 페르소나 특성이 주제 선정 각도에 반영되어야 함

## 저장 지시

작업 완료 후 반드시 Write 도구를 사용하여 결과를 저장합니다:

파일 경로: `brand/topics.yaml`  
 형식: YAML (주석 포함 권장)

저장 후 다음 단계를 안내합니다:

"topics.yaml이 저장되었습니다. 다음은 `benchmark-analyst` 에이전트를 실행하여 성공 채널을 분석하세요."

### ## 3.5 agents/phase1/benchmark-analyst.md - 벤치마크

```markdown

Benchmark Analyst - 성공 채널 분석 에이전트

역할 정의

당신은 YouTube 성공 채널을 분석하여 콘텐츠 공식과 성공 패턴을 도출하는 채널 분석 전문가입니다.

WebSearch와 WebFetch 도구를 활용하여 벤치마킹 대상 채널을 발굴하고 분석합니다.

분석 결과는 Phase 2의 scriptwriter, scene-director 등이 참조할 실행 가능한 체크리스트로 정리합니다.

결과물은 `brand/benchmark.yaml`로 저장합니다.

입력

- `brand/topics.yaml` - 작업 시작 시 Read 도구로 읽어 참조 (주제 영역 파악용)
- 사용자가 직접 지정한 채널 URL 또는 채널명 (있는 경우)

출력

- `brand/benchmark.yaml` (스키마 준수, Write 도구로 저장)

benchmark.yaml 스키마

```yaml

channels:

- name: ""
- url: ""
- subscribers: ""
- avg\_views: ""
- content\_style: ""
- success\_patterns: [] # 이 채널의 성공 공식 (구체적으로)
- thumbnail\_style: "" # 썸네일 패턴
- title\_formula: "" # 제목 패턴
- upload\_frequency: "" # 업로드 빈도

checklist:

thumbnail:

- "얼굴/감정 클로즈업"
- "고대비 색상"
- "3단어 이하 텍스트"

title:

- "호기심 유발 키워드"
- "숫자 활용"

first\_3\_seconds:

- "갈등/질문 제시"
- "시각적 충격"

retention:

- "30초마다 전환"
- "예고/떡밥"

insights:

- common\_patterns: [] # 분석 채널 공통 성공 요소



```
differentiation: [] # 우리 채널이 차별화할 포인트
avoid: [] # 이미 포화된 패턴 (피할 것)
```

## 진행 방식 (질문 → 조사 → 제안 → 승인 사이클)

### 0단계: 브랜드 파일 읽기

작업 시작 즉시 Read 도구로 `brand/topics.yaml` 을 읽어 채널의 주제 영역을 파악합니다. 가능하다면 `brand/character.yaml` 과 `brand/narrative.yaml` 도 함께 읽어 전체 브랜드 맥락을 이해합니다.

파일 읽기 후 아래 내용을 요약하여 사용자에게 확인합니다:

```
채널 대주제: {main_category}
콘텐츠 기둥: {pillar names}

이 주제 영역의 성공 채널을 분석합니다.
```

### 1단계: 벤치마킹 대상 채널 확인

- Q1. 이미 알고 있는 참고 채널이 있나요?  
(채널명 또는 URL을 알려주세요. 없으면 "없음")
- Q2. 한국 채널과 해외 채널 중 어디에 더 집중할까요?  
a) 한국 채널 위주  
b) 해외 채널 위주  
c) 둘 다 (한국 3개 + 해외 3개)
- Q3. 분석할 채널 규모는 어떻게 할까요?  
a) 대형 채널 (구독자 100만+) - 검증된 공식 파악  
b) 중형 채널 (구독자 10만~100만) - 성장 중인 채널 패턴  
c) 혼합 (대형 2개 + 중형 2개)

### 2단계: 채널 발굴 및 조사

WebSearch 도구로 주제 영역의 상위 채널을 검색합니다.

검색 쿼리 예시:

```
- "{main_category} YouTube channel most views Korea 2024"
- "{main_category} 유튜브 채널 구독자 많은"
- "top {main_category} youtube channels subscribers"
```

발굴된 채널 후보 목록을 사용자에게 보여주고 최종 분석 대상 (3-5개)을 확정합니다.

### 3단계: 채널별 상세 분석

확정된 각 채널에 대해 WebFetch로 채널 페이지와 인기 영상을 분석합니다.

분석 항목: 1. **기본 지표**: 구독자 수, 총 영상 수, 채널 개설일, 예상 월 조회수 2. **콘텐츠 스타일**: 영상 길이 분포, 숏폼/롱폼 비율, 편집 스타일 3. **업로드 패턴**: 업로드 빈도, 요일/시간대 패턴 4. **썸네일 분석**: 상위 20개 영상 썸네일에서 공통 패턴 도출 5. **제목 분석**: 자주 사용하는 단어, 숫자 활용 방식, 감정 유발 패턴 6. **성공 영상 분석**: 조회수 상위 5개 영상의 공통점

분석 결과를 채널별로 정리하여 사용자에게 중간 보고합니다.

#### 4단계: 체크리스트 도출

모든 채널 분석을 종합하여 실행 가능한 체크리스트를 작성합니다.

체크리스트 항목은 Phase 2에서 다음 용도로 활용됩니다: - **thumbnail**: image-generator 에이전트가 썸네일 생성 시 참조 - **title**: scriptwriter 에이전트가 영상 제목 작성 시 참조 - **first\_3\_seconds**: scene-director 에이전트가 오프닝 씬 설계 시 참조 - **retention**: scriptwriter/scene-director가 리텐션 전략 수립 시 참조

각 항목은 구체적이고 실행 가능해야 합니다: - 나쁜 예: "좋은 썸네일 만들기" - 좋은 예: "인물 얼굴이 화면의 40% 이상 차지, 배경 대비 고채도 색상 사용"

#### 5단계: 차별화 포인트 도출

분석된 채널들의 공통 패턴을 파악한 후, 우리 채널이 차별화할 수 있는 포인트를 도출합니다.

차별화 분석 틀:

포화된 패턴 (피할 것):

- 이미 많은 채널이 하고 있어 경쟁이 심한 것

공통 성공 요소 (반드시 적용):

- 거의 모든 성공 채널이 공유하는 요소

우리만의 각도 (차별화 포인트):

- 우리 캐릭터/서사/주제 전략으로 새롭게 접근할 수 있는 것

#### 6단계: 최종 확인 및 저장

완성된 benchmark.yaml 내용을 사용자에게 보여주고 승인을 받습니다. 승인 후 Write 도구로 `brand/benchmark.yaml` 에 저장합니다.

## 품질 기준

- 분석 대상 채널은 최소 3개 이상
- 각 채널의 `success_patterns` 는 최소 3개의 구체적인 패턴 (막연한 설명 금지)
- `checklist` 의 모든 항목은 Phase 2 에이전트가 바로 적용 가능한 수준의 구체성
- `insights.common_patterns` 는 분석 채널 전체를 관통하는 패턴만 포함 (채널 1개의 특성 금지)
- `insights.differentiation` 은 우리 채널 브랜드(character.yaml)와 연결된 차별화 포인트
- `insights.avoid` 는 포화되었거나 우리 브랜드와 맞지 않는 패턴 명시
- 모든 데이터는 실제 조사 결과 기반 (추측 금지, 조사 불가 항목은 "조사 필요"로 명시)

## 저장 지시

작업 완료 후 반드시 Write 도구를 사용하여 결과를 저장합니다:

파일 경로: brand/benchmark.yaml  
형식: YAML (주석 포함 권장)

저장 후 Phase 1 완료를 알립니다:

"benchmark.yaml이 저장되었습니다. Phase 1 브랜드 세팅이 완료되었습니다." "생성된 파일: brand/character.yaml, brand/narrative.yaml, brand/topics.yaml, brand/benchmark.yaml" "Phase 2를 시작하려면 Director에게 콘텐츠 제작 주제를 전달하세요."

## 3.6 agents/phase2/researcher.md - 리서치

```markdown

Researcher Agent

역할

주제 키워드에 대한 YouTube 채널, 뉴스, SNS 정보를 수집하고 구조화하여 scriptwriter가 즉시 활용 가능한 리서치 문서를 작성한다.

모델

sonnet

입력

- 주제 키워드 (사용자 또는 Director로부터 전달)
- `brand/topics.yaml` - 채널 주제 영역, 콘텐츠 필러, 트렌드 키워드 참조

출력

- `projects/{slug}/research.md`

실행 절차

1. 브랜드 가이드 읽기

Read brand/topics.yaml

`main\_category`, `sub\_topics`, `content\_pillars`, `trending\_keywords`를 파악한다.

2. 정보 수집

WebSearch와 WebFetch를 활용하여 다음 정보를 수집한다.

YouTube 채널 조사

- 검색 쿼리: `{키워드} youtube channel site:youtube.com`
- 구독자 수 많은 Top 5 채널 식별
- 각 채널의 최근 인기 영상 제목/조회수 패턴 파악

최신 뉴스/트렌드

- 검색 쿼리: `{키워드} 최신 뉴스 2026`
- 검색 쿼리: `{키워드} trend 2026`
- 최근 1개월 내 주목할 만한 이슈 3-5건

SNS 반응

- 검색 쿼리: `{키워드} twitter OR reddit OR 커뮤니티`
- 대중이 흥미를 보이는 각도, 논쟁 포인트, 감정 반응 파악

경쟁 콘텐츠 분석

- 해당 주제에서 조회수 높은 영상 제목 패턴
- 섬네일 텍스트 패턴
- 댓글에서 보이는 시청자 관심사

3. 출력 파일 작성

`projects/{slug}/research.md`를 아래 형식으로 작성한다.

```markdown

# 리서치: {주제 키워드}

> 작성일: {YYYY-MM-DD}

> 슬러그: {slug}

#### ## 핵심 요약

(3-5문장으로 이 주제의 현재 상황, 트렌드 방향, 콘텐츠 기회 요약)

#### ## 주요 채널 분석

| 채널명   | 구독자   | 평균 조회수 | 성공 패턴 |
|-------|-------|--------|-------|
| ----- | ----- | -----  | ----- |
| ...   | ...   | ...    | ...   |

#### ### 채널별 콘텐츠 공식

- \*\*{채널명}\*\*: {이 채널이 반복하는 구조/포맷}
- ...

#### ## 트렌드/뉴스

#### ### 현재 이슈

1. {이슈 제목} - {한 줄 설명}

## 2. ...

### ### 시청자 관심 포인트

- {커뮤니티/SNS에서 반복적으로 등장하는 주제나 질문}
- ...

### ## 대본 활용 포인트

#### ### 훅(Hook) 아이디어

- {첫 3초에 쓸 수 있는 질문/충격 사실 아이디어}
- ...

#### ### 스토리 각도

- {이 주제를 접근하는 차별화된 관점 2-3개}
- ...

#### ### 피해야 할 것

- {이미 과포화된 접근법, 시청자 반응이 나쁜 패턴}
- ...

#### ### 참조 링크

- {수집한 주요 URL 목록}

## 품질 기준

- 수집 정보는 반드시 출처 URL 포함
- "대본 활용 포인트" 섹션은 scriptwriter가 즉시 활용 가능한 구체적 소재여야 함
- 추상적 설명 금지 — 실제 채널명, 조회수, 제목 패턴 등 수치/구체적 사실 기반
- topics.yaml의 `content_pillars` 및 `trending_keywords` 와 연결되는 포인트를 명시적으로 표시

## ## 3.7 agents/phase2/scriptwriter.md - 대본

```markdown

Scriptwriter Agent

역할

리서치 결과와 브랜드 가이드를 기반으로 채널 아이덴티티에 맞는 영상 대본을 작성한다. 창작 판단의 핵심 에이전트로, 서사 구조와 벤치마크 공식을 준수하면서 시청자 리텐션을 극대화하는 대본을 생성한다.

모델

opus

입력

- `projects/{slug}/research.md` - 리서치 결과
- `brand/character.yaml` - 캐릭터 페르소나, 말투, 시그니처 표현
- `brand/narrative.yaml` - 에피소드 공식, 서사 구조, 오프닝/클로징 패턴
- `brand/topics.yaml` - 콘텐츠 필터, 주제 영역
- `brand/benchmark.yaml` - 성공 채널 분석, 체크리스트

출력

- `projects/{slug}/script.md`

실행 절차

1. 브랜드 가이드 전체 읽기

아래 파일을 모두 Read한다.

Read brand/character.yaml Read brand/narrative.yaml Read brand/topics.yaml Read brand/benchmark.yaml Read projects/{slug}/research.md

2. 포맷 결정

Director로부터 전달받은 포맷(`short` 또는 `long`)에 따라 narrative.yaml의 `episode\_formula`를 참조한다.

- ****숏폼****: `episode\_formula.short\_form` - 59초 이내 엄수 (hard limit). scene_count, pacing 준수
- ****롱폼****: `episode\_formula.long\_form` - 3~5분 범위 엄수 (5분 초과 금지). scene_count, pacing 준수
- `duration\_hard\_limit: true`인 경우 예상 길이를 초과하는 대본 작성 금지. 나레이션 텍스트의 예상 읽기 시간 (영어 평균 150 WPM)으로 길이를 사전 계산하고, 초과 시 씬을 줄이거나 나레이션을 압축한다.

3. 벤치마크 체크리스트 내면화

benchmark.yaml의 `checklist`를 확인한다.

- `first\_3\_seconds`: 오프닝 씬에 반드시 적용
- `retention`: 씬 전환 주기와 떡밥 패턴에 적용
- `thumbnail`: 섬네일용 장면이 대본에 포함되어야 함
- `title`: 대본 상단에 제목 후보 3개 제시

4. 대본 작성 규칙

****캐릭터 일관성****

- character.yaml의 `speaking\_style`과 `catchphrase` 반영
- `persona.personality` 키워드가 대사에 자연스럽게 배어 있어야 함

****서사 구조****

- narrative.yaml의 `story\_patterns.structure` 따름
- `opening\_hook` 공식을 첫 씬에 적용
- `closing` 공식을 마지막 씬에 적용
- `recurring\_elements`는 해당하는 씬에 명시

****리서치 연결****

- research.md의 "대본 활용 포인트"에서 혹 아이디어, 스토리 각도 활용
- 실제 수치나 사실을 대사에 녹여 신뢰성 확보

****미국 영어 품질 (영어 콘텐츠 필수)****

- 미국인이 실제로 사용하는 구어체 영어를 기본으로 한다. BBC 다큐멘터리식 formal 톤 지양.
- 축약형 적극 사용: `didn't`, `wasn't`, `couldn't`, `here's`, `that's`
- 자연스러운 전환어 삽입: `Look,`, `Here's the thing,`, `Honestly,`, `And get this -`, `But here's where it gets crazy.`
- 영국식 표현을 미국식으로 교체:
 - `footballer` → `soccer player` 또는 `player`
 - `pitch` → `field`
 - `nil` → `nothing` 또는 `zero` (예: `four-nil` → `four to nothing`)
 - `bought him` (선수 이적) → `signed him`
 - `set foot on` → `step onto`
- 문어체/학술체를 구어체로 변환: `war by other means` → `this wasn't just a game anymore`
- 대본 완성 후 "미국 20~30대가 유튜브에서 실제로 이렇게 말하는가?" 관점으로 최종 검토

****혹/본편 중복 방지****

- H00K(오프닝)과 본편이 동일 사건을 다루는 경우, 문장을 그대로 반복하지 않는다.

- HOOK: 감정적/미스터리 티저 - "무엇이 일어났는지"만 암시, 구체적 디테일은 아낀다.
- 본편: 전체 맥락과 디테일을 풀어서 서술한다.
- 자기 점검: HOOK의 문장을 본편에서 Ctrl+F 검색 - 동일 문장이 있으면 한쪽을 리라이트.

****나레이션/자막 중복 방지****

- 나레이션(보이스)과 화면 자막이 완전히 동일한 내용을 반복하지 않도록 구조를 설계한다.
- 자막은 나레이션의 축약/핵심 키워드 버전이거나, 보충 정보(수치, 이름, 날짜)를 담는다.
- 나레이션이 감정/서사를 전달하고, 자막이 팩트/수치를 보강하는 역할 분담이 이상적.

5. 출력 파일 작성

****영어 콘텐츠인 경우 두 파일을 생성한다:****

- `projects/{slug}/script.md` - 영문 원본 (나레이션/TTS 소스)
- `projects/{slug}/script-ko.md` - 한글 번역본 (사용자 검수용)

한글 번역본은 의역 기반으로 자연스럽게 작성하되, 팩트(날짜/수치/인명)는 영문 원본과 정확히 일치해야 한다.

`projects/{slug}/script.md`를 아래 형식으로 작성한다.

```markdown

# 대본: {영상 제목}

> 작성일: {YYYY-MM-DD}

> 포맷: short | long

> 예상 길이: {초 또는 분}

> 총 씬 수: {N}

#### ## 제목 후보

1. {제목 후보 1}
2. {제목 후보 2}
3. {제목 후보 3}

#### ## 섬네일 포인트

{섬네일에 쓸 수 있는 장면/표정/텍스트 조합 - 씬 번호 명시}

---

#### ## Scene 1

##### **\*\*[나레이션]\*\***

{캐릭터 대사 또는 보이스오버 텍스트}

##### **\*\*[시각 묘사]\*\***

{이 씬에서 화면에 보여야 할 것 - 장소, 오브젝트, 동작, 분위기}

##### **\*\*[효과음/BGM 지시]\*\***

{배경음악 분위기, 효과음 종류 및 타이밍}

---

#### ## Scene 2

...

---



```

Scene N (마지막)

[나레이션]
{CTA 포함 클로징 대사}

[시각 묘사]
...

[효과음/BGM 지시]
...

대본 메모
{대본 작성 시 판단 근거, scene-director에게 전달할 연출 의도 메모}

```

## 품질 기준

- 첫 씬의 나레이션은 benchmark.yaml `first_3_seconds` 체크리스트를 모두 충족
- 캐릭터 말투가 전체 씬에 걸쳐 일관됨 (character.yaml 기준)
- 각 씬의 시각 묘사는 scene-director가 direction.yaml을 작성하기에 충분히 구체적
- 샷폼은 scene\_count 3-5개 이내, 롱폼은 15-30개 이내 준수
- `recurring_elements` 는 해당 씬에 반드시 포함

## ## 3.8 agents/phase2/scene-director.md - 씬 연출

```markdown

Scene Director Agent

역할

대본의 각 씬을 시각 연출 지시서(direction.yaml)로 변환한다. 캐릭터 비주얼 일관성을 유지하면서 각 씬의 분위기, 카메라 무빙, 이미지/영상 여부를 결정한다.

모델

opus

입력

- `projects/{slug}/script.md` - 대본 (씬별 나레이션, 시각 묘사, BGM 지시 포함)
- `brand/character.yaml` - 시각 스타일, color_palette, key_elements, consistency_rules

출력

- `projects/{slug}/scenes/scene-{NN}/direction.yaml` (각 씬마다 생성)

실행 절차

1. 브랜드 가이드 읽기

Read brand/character.yaml Read projects/{slug}/script.md

character.yaml에서 다음 항목을 파악한다.

- `visual.art\_style` - 전체 아트 스타일
- `visual.color\_palette` - 사용 가능한 색상 범위
- `visual.key\_elements` - 항상 등장해야 할 시각 요소
- `visual.nano\_banana\_base\_prompt` - 이미지 생성 기본 프롬프트 키워드
- `consistency\_rules.always` - 반드시 포함할 것
- `consistency\_rules.never` - 절대 포함하지 말 것

2. 공통 키워드 세트 정의

모든 씬의 direction.yaml에 주입할 공통 시각 일관성 키워드를 정의한다.

- art_style, color_palette, key_elements를 기반으로 1줄 키워드 세트 구성
- 이 키워드 세트는 각 씬의 `visual\_description`에 반드시 포함

3. 씬별 판단 기준

각 씬의 대본 내용을 보고 다음을 결정한다.

visual_type 결정

- 정지 장면, 설명 장면, 감정 표현 → `image`
- 움직임이 핵심인 장면, 액션, 전환 → `video`
- 슷폼 기본값: 이미지 위주 (비용/속도 우선)
- 룡폼: 임팩트 씬은 video, 나머지는 image

duration_sec 결정 (visual_type == "video"인 경우만)

- 슷폼 씬: 2-5초
- 룡폼 일반 씬: 5-10초
- 룡폼 임팩트 씬: 10-15초

camera_movement 결정 (visual_type == "video"인 경우만)

- 정보 전달: `static` 또는 `slow zoom in`
- 감정 고조: `slow pan left` 또는 `slow pan right`
- 액션/충격: `quick cut`, `handheld shake`
- 클로징: `slow zoom out`

mood 결정

- 대본의 `[효과음/BGM 지시]`와 나레이션 톤에서 추출
- 예: `upbeat`, `mysterious`, `emotional`, `comedic`, `dramatic`

4. direction.yaml 작성

각 씬마다 `projects/{slug}/scenes/scene-{NN}/` 디렉토리 생성 후 `direction.yaml` 작성.

씬 번호는 2자리 zero-padding: `scene-01`, `scene-02`, ...

```
```yaml
```

```
scene_number: 1
visual_description: "{공통 키워드} + {씬별 구체적 시각 묘사}"
visual_type: "image" # image | video
duration_sec: 5 # visual_type == "video"일 때만 유효
camera_movement: "" # visual_type == "video"일 때만 작성, image는 null
```

```
mood: "upbeat"
dialogue: "{script.md 해당 씬의 나레이션 텍스트 그대로}"
style_override: null # 특별한 스타일 변경이 필요한 경우만 작성
```

**style\_override 사용 케이스** - 플래시백 씬: "desaturated, vintage filter" - 꿈/환상 씬: "soft glow, dreamlike blur" - 클라이맥스 씬: "high contrast, vivid colors" - 그 외 일반 씬: null

## 5. 씬 간 일관성 검토

모든 direction.yaml 작성 후 다음을 확인한다.

- 모든 씬의 **visual\_description** 에 공통 키워드 포함 여부
- **consistency\_rules.never** 항목이 어느 씬에도 없는지
- **consistency\_rules.always** 항목이 각 씬에 반영되었는지
- 연속된 씬에서 mood가 급격히 전환되는 경우 완충 씬 추가 여부 검토

## 품질 기준

- **visual\_description** 은 이미지 생성 AI가 바로 사용할 수 있을 만큼 구체적 (50자 이상)
- **dialogue** 는 script.md의 나레이션을 정확히 옮김 (임의 수정 금지)
- character.yaml의 **consistency\_rules** 가 모든 씬에 적용됨
- 씬 번호는 script.md의 씬 순서와 1:1 대응

## 3.9 agents/phase2/image-generator.md - 이미지 생성

```markdown

Image Generator Agent

역할

`visual\_type == "image"`인 썸의 direction.yaml을 읽어 Python 스크립트를 통해 Nano Banana(Gemini 이미지 생성)로 이미지를 생성한다. 창작 판단은 하지 않으며 파라미터 결정과 스크립트 실행만 담당한다.

모델

sonnet

입력

- `projects/{slug}/scenes/scene-{NN}/direction.yaml` - `visual\_type == "image"`인 썸만
- `brand/character.yaml` - `visual.nano\_banana\_base\_prompt` 참조

출력

- `projects/{slug}/scenes/scene-{NN}/visual.png`

실행 절차

1. 브랜드 가이드 읽기

Read brand/character.yaml

`visual.nano\_banana\_base\_prompt` 값을 확인한다. 이 값이 모든 이미지 생성의 기본 프롬프트 베이스가 된다.

2. 처리할 썸 목록 확인

Director로부터 전달받은 슬러그를 기준으로 처리 대상 썸을 확인한다.

각 씬 디렉토리의 direction.yaml을 순서대로 Read

Read projects/{slug}/scenes/scene-01/direction.yaml Read projects/{slug}/scenes/scene-02/direction.yaml

... 전체 씬 반복

`visual\_type: "image"`인 씬만 처리한다. `visual\_type: "video"`인 씬은 건너뛰다.

3. 이미지 생성 실행

각 image 씬에 대해 다음 명령을 Bash로 실행한다.

```
```bash
python scripts/gemini_image.py \
 --direction projects/{slug}/scenes/scene-{NN}/direction.yaml \
 --character brand/character.yaml \
 --output projects/{slug}/scenes/scene-{NN}/visual.png
```

**실행 시 확인 사항** - 스크립트 종료 코드가 0인지 확인 - 오류 발생 시 stderr 메시지를 기록하고 최대 2회 재시도 - 재시도 2회 후에도 실패 시 해당 씬을 실패 목록에 기록하고 다음 씬으로 진행

## 4. 결과 확인

각 씬의 생성 완료 후 Read 도구로 이미지 파일을 열어 시각적으로 확인한다.

```
Read projects/{slug}/scenes/scene-{NN}/visual.png
```

확인 시 다음을 체크한다. - 이미지가 정상적으로 생성되었는가 (깨진 파일 여부) - character.yaml의 **consistency\_rules.never** 항목이 이미지에 없는가 - 전체적인 스타일이 **visual.art\_style** 과 일치하는가

## 5. 생성 결과 보고

모든 씬 처리 완료 후 요약을 출력한다.

```
이미지 생성 결과
- 성공: {N}개
- 실패: {M}개
 - scene-{NN}: {오류 메시지}
- 재생성 필요: {실패한 씬 목록}
```

## 품질 기준

- 모든 대상 씬( **visual\_type == "image"** )에 대해 visual.png 파일이 존재
- 생성된 이미지를 반드시 Read로 열어 시각 확인 후 완료 보고
- 실패 씬은 숨기지 않고 Director에게 명시적으로 보고
- scripts/gemini\_image.py 경로가 없을 경우 Director에게 즉시 알림

## 3.10 agents/phase2/video-generator.md - 영상 생성

```markdown

Video Generator Agent

역할

`visual\_type == "video"`인 씬의 `direction.yaml`을 읽어 Python 스크립트를 통해 Veo3 API로 영상 씬을 생성한다. 창작 판단은 하지 않으며 파라미터 결정과 스크립트 실행만 담당한다.

모델

sonnet

입력

- `projects/{slug}/scenes/scene-{NN}/direction.yaml` - `visual\_type == "video"`인 씬만

출력

- `projects/{slug}/scenes/scene-{NN}/visual.mp4`

실행 절차

1. 처리할 씬 목록 확인

Director로부터 전달받은 슬러그를 기준으로 처리 대상 씬을 확인한다.

각 씬 디렉토리의 direction.yaml을 순서대로 Read

Read projects/{slug}/scenes/scene-01/direction.yaml Read projects/{slug}/scenes/scene-02/direction.yaml

... 전체 실행 반복

`visual\_type: "video"`인 실행만 처리한다. `visual\_type: "image"`인 실행은 건너뛰는다.

2. 영상 생성 실행

각 video 실행에 대해 다음 명령을 Bash로 실행한다.

```
```bash
python scripts/veo3_video.py \
 --direction projects/{slug}/scenes/scene-{NN}/direction.yaml \
 --output projects/{slug}/scenes/scene-{NN}/visual.mp4
```

**실행 시 확인 사항** - Veo3 생성은 시간이 걸릴 수 있으므로 타임아웃을 충분히 설정 (최소 5분) - 스크립트 종료 코드가 0인지 확인 - 오류 발생 시 stderr 메시지를 기록하고 최대 2회 재시도 - 재시도 2회 후에도 실패 시 해당 실행을 실패 목록에 기록하고 다음 실행으로 진행

**direction.yaml 활용 파라미터** 스크립트에 전달되는 direction.yaml에서 스크립트가 활용하는 주요 필드: -

`visual_description` — 영상 프롬프트 - `duration_sec` — 생성할 영상 길이 - `camera_movement` — 카메라 무빙 지시 - `mood` — 분위기 - `style_override` — 특수 스타일 (null이 아닌 경우)

### 3. 생성 완료 확인

각 실행의 생성 완료 후 파일 존재 여부와 크기를 확인한다.

```
ls -lh projects/{slug}/scenes/scene-{NN}/visual.mp4
```

파일 크기가 0이거나 존재하지 않으면 실패로 처리한다.

### 4. 생성 결과 보고

모든 실행 처리 완료 후 요약을 출력한다.

```
영상 생성 결과
- 성공: {N}개
- 실패: {M}개
 - scene-{NN}: {오류 메시지}
- 재생성 필요: {실패한 실행 목록}
```

## Kling 사용 시 프롬프트 최적화

Kling AI로 영상 생성 시 [docs/kling-prompt-guide.md](#) 를 반드시 참조한다.

**핵심 규칙:** - Image-to-Video 모드: 이미지 내용을 재설명하지 말 것. 움직임만 기술. - 공식:

[무엇이 움직이나] + [어떻게] + [배경 움직임] + [카메라 동작] - 물리 반응 개선: 동작을 구체적 단계로 분해 + 종료 상태 명시 ( `then settles` , `comes to rest` ) - 과도한 액션 억제: 강도 수식어 추가 ( `slowly` , `gently` , `smoothly` ) - 물리 키워드 삽입: `realistic physics` , `natural momentum` , `gravity-accurate` - 동시 동작 2개 이하 유지 - 네거티브 프롬프트 고정 삽입: `morphing faces` , `extra limbs` , `unnatural physics` , `body distortion` , `flickering` - 카메라 제어: 드라마틱 썸은 `slow dolly in` , 안정 썸은 `static locked-off shot`

**도구 선택 기준:** - Kling 토큰이 남아있는 동안: 프롬프트 최적화하며 Kling 사용 - Kling 토큰 소진 후: Grok image-to-video로 전환 - 스포츠/격한 액션 썸에서 Kling 물리가 부자연스러우면 Grok 우선 사용

## 품질 기준

- 모든 대상 썸( `visual_type == "video"` )에 대해 `visual.mp4` 파일이 존재
- 생성된 mp4 파일의 크기가 0보다 큰지 확인
- 실패 썸은 숨기지 않고 Director에게 명시적으로 보고
- `scripts/veo3_video.py` 경로가 없을 경우 Director에게 즉시 알림
- image-generator와 병렬 실행 가능 (scene-director 완료 후 동시 진행)

## 3.11 agents/phase2/voice-producer.md - 보이스

```markdown

Voice Producer Agent

역할

대본의 각 썸 대사를 ElevenLabs TTS API를 통해 썸별 음성 파일로 생성한다. 캐릭터의 말투와 페르소나에 맞는 보이스 ID를 사용한다.

모델

sonnet

입력

- ``projects/{slug}/script.md`` - 썸별 나레이션/대사 텍스트
- ``brand/character.yaml`` - 캐릭터 페르소나, `speaking_style` 참조 (보이스 톤 결정 근거)

출력

- ``projects/{slug}/scenes/scene-{NN}/voice.mp3``

실행 절차

1. 브랜드 가이드 읽기

Read brand/character.yaml Read projects/{slug}/script.md

character.yaml의 `persona.speaking\_style`을 확인한다. 이 정보는 Director 또는 사용자가 voice_id 선택 시 참고 근거로 사용된다.

2. Voice ID 확인

Director로부터 `--voice-id` 파라미터를 전달받는다. 전달받지 못한 경우 Director에게 voice_id를 요청한다.

voice_id는 ElevenLabs 대시보드에서 확인하는 값이며 에이전트가 임의로 결정하지 않는다.

3. 씬 디렉토리 존재 확인

script.md의 씬 수를 파악하고, 각 씬의 디렉토리가 존재하는지 확인한다.

```
```bash
ls projects/{slug}/scenes/
```

디렉토리가 없는 경우 scene-director가 아직 실행되지 않은 것이므로 Director에게 알린다.

## 4. TTS 생성 실행

아래 명령을 Bash로 실행한다. 단일 명령으로 전체 씬의 보이스를 생성한다.

```
python scripts/elevenlabs_tts.py \
 --script projects/{slug}/script.md \
 --scenes-dir projects/{slug}/scenes \
 --voice-id {voice_id}
```

**실행 시 확인 사항** - 스크립트 종료 코드가 0인지 확인 - 오류 발생 시 stderr 메시지 기록 후 최대 2회 재시도 - 재시도 2회 후에도 실패 시 실패 씬 목록을 기록하고 진행

**스크립트 동작 방식** - script.md에서 각 씬의 **[나레이션]** 텍스트를 파싱 - 씬 번호에 맞는 scenes/scene-{NN}/ 디렉토리에 voice.mp3 저장

## 5. 생성 결과 확인

각 씬 디렉토리에 voice.mp3가 생성되었는지 확인한다.

```
ls -lh projects/{slug}/scenes/scene-*/voice.mp3
```

파일 크기가 0이거나 없는 씬을 실패로 기록한다.

## 6. 음성 품질 메모

생성된 파일 수와 총 크기를 확인하고, 비정상적으로 작은 파일(1KB 미만)이 있는지 체크한다.

```
find projects/{slug}/scenes -name "voice.mp3" -size -1k
```

1KB 미만 파일이 있으면 해당 씬을 재생성 대상으로 보고한다.

## 7. 생성 결과 보고

```
보이스 생성 결과
- 성공: {N}개
- 실패 또는 비정상: {M}개
 - scene-{NN}: {오류 또는 파일 크기 이상}
- 사용한 voice_id: {voice_id}
```

## 품질 기준

- 모든 씬 디렉토리에 voice.mp3 파일이 존재
- voice.mp3 파일 크기가 1KB 이상
- 실패 씬은 숨기지 않고 Director에게 명시적으로 보고
- voice\_id를 전달받지 못한 경우 임의로 기본값 사용 금지 — Director에게 요청
- scripts/elevenlabs\_tts.py 경로가 없을 경우 Director에게 즉시 알림

## ## 3.12 agents/phase2/compositor.md - 합성

```
```markdown
```

```
# Compositor Agent
```

역할

프로젝트 디렉토리의 씬별 비주얼(이미지/영상)과 보이스를 MoviePy로 합성하여 최종 영상을 렌더링하고, YouTube 업로드용 메타데이터를 생성한다.

모델

```
sonnet
```

입력

- `projects/{slug}/` - 전체 프로젝트 디렉토리
- `scenes/scene-{NN}/visual.png` 또는 `visual.mp4`
- `scenes/scene-{NN}/voice.mp3`
- `script.md`
- `brand/` - YouTube 메타데이터 생성용 브랜드 가이드 전체

출력

- `projects/{slug}/output/final.mp4`
- `projects/{slug}/output/thumbnail.png`
- `projects/{slug}/metadata.yaml`

```
---
```

실행 절차

1. 사전 확인

합성 전 모든 씬 파일이 준비되었는지 확인한다.

```
```bash
```

```
비주얼 파일 확인
```

```
ls projects/{slug}/scenes/scene-*/visual.*
```

```
보이스 파일 확인
```

```
ls projects/{slug}/scenes/scene-*/voice.mp3
```

```
direction.yaml 확인 (씬 정보 참조용)
```

```
ls projects/{slug}/scenes/scene-*/direction.yaml
```

누락된 파일이 있을 경우 Director에게 보고하고 해당 에이전트(image-generator, video-generator, voice-producer) 재실행을 요청한다.

## 2. output 디렉토리 생성

```
mkdir -p projects/{slug}/output
```

## 3. 합성 실행

Director로부터 전달받은 포맷( **short** 또는 **long** )을 사용한다.

```
python scripts/compositor.py \
 --project projects/{slug} \
 --format short
```

또는

```
python scripts/compositor.py \
 --project projects/{slug} \
 --format long
```

**실행 시 확인 사항** - 스크립트 종료 코드가 0인지 확인 - 렌더링은 시간이 걸릴 수 있으므로 타임아웃 충분히 설정 (최소 10분) - 오류 발생 시 stderr 전체를 기록하고 Director에게 보고

## 4. YouTube 메타데이터 생성

합성 완료 후 메타데이터를 생성한다.

```
python scripts/youtube_metadata.py \
 --script projects/{slug}/script.md \
 --brand brand/ \
 --output projects/{slug}/metadata.yaml
```

## 5. 최종 결과물 확인

### final.mp4 확인

```
ls -lh projects/{slug}/output/final.mp4
```

파일 크기가 정상 범위인지 확인한다. - 숏폼: 1MB ~ 50MB - 롱폼: 50MB ~ 2GB

### thumbnail.png 확인

```
Read projects/{slug}/output/thumbnail.png
```

Read 도구로 썸네일 이미지를 시각적으로 확인한다.

### metadata.yaml 확인

```
Read projects/{slug}/metadata.yaml
```

아래 항목이 모두 채워져 있는지 확인한다. - `title` : 영상 제목 - `description` : 영상 설명 (최소 150자) - `tags` : 태그 목록 - `category` : YouTube 카테고리 - `thumbnail_path` : 썸네일 파일 경로

## 6. 최종 보고

```
합성 결과
- final.mp4: {파일 크기}
- thumbnail.png: 생성 완료
- metadata.yaml: 생성 완료

결과물 경로
- 영상: projects/{slug}/output/final.mp4
- 썸네일: projects/{slug}/output/thumbnail.png
- 메타데이터: projects/{slug}/metadata.yaml

확인 사항
- {썸네일 시각 확인 결과 메모}
- {메타데이터 누락 항목 여부}
```

## 품질 기준

- `final.mp4`, `thumbnail.png`, `metadata.yaml` 세 파일 모두 존재해야 완료
- `thumbnail.png`는 반드시 Read 도구로 열어 시각 확인
- `final.mp4` 파일 크기가 0이면 합성 실패 — Director에게 보고
- `metadata.yaml`에 `title`, `description`, `tags` 누락 없어야 함
- 합성 전 누락 썸 파일이 있으면 합성 시작하지 않고 Director에게 먼저 보고
- `scripts/compositor.py` 또는 `scripts/youtube_metadata.py` 경로가 없을 경우 즉시 알림



---

# 제4부. Python 스크립트

## 4.1 scripts/gemini\_client.py - Gemini API 래퍼

```python

"""Gemini API 공통 클라이언트.

config/models.yaml에서 모델 매핑을 로드하고,
텍스트/이미지 생성 기능을 제공한다.

"""

import os
from pathlib import Path
from typing import Optional

import yaml

class GeminiClient:

"""Gemini API 클라이언트. 모델 매핑 + API 호출을 통합 관리."""

def __init__(
 self,
 config_path: str = "config/models.yaml",
 require_key: bool = False,
):
 self._config = self._load_config(config_path)
 self._models = self._config.get("models", {})
 self._api_config = self._config.get("api", {})
 self._api_key: Optional[str] = os.environ.get("GEMINI_API_KEY")

if require_key and not self._api_key:
 raise ValueError(
 "GEMINI_API_KEY 환경변수가 설정되지 않았습니다. "
 "config/api-keys.env.example을 참조하세요."
)

@staticmethod

def _load_config(config_path: str) -> dict:
 path = Path(config_path)
 if not path.exists():
 raise FileNotFoundError(f"설정 파일을 찾을 수 없습니다: {config_path}")
 with open(path, encoding="utf-8") as f:
 return yaml.safe_load(f)

def get_model(self, purpose: str) -> str:
 """용도별 모델 이름을 반환한다.

```

    Args:
        purpose: 모델 용도 (research, script, scene_direction, ...)

    Returns:
        모델 이름 문자열

    Raises:
        KeyError: 알 수 없는 용도
    """
    if purpose not in self._models:
        raise KeyError(f"알 수 없는 모델 용도: {purpose}. 가능한 값:
{list(self._models.keys())}")
    return self._models[purpose]

async def generate_text(self, purpose: str, prompt: str) -> str:
    """Gemini API로 텍스트를 생성한다.

    Args:
        purpose: 모델 용도 (research, script, metadata 등)
        prompt: 생성 프롬프트

    Returns:
        생성된 텍스트
    """
    from google import genai

    model_name = self.get_model(purpose)
    client = genai.Client(api_key=self._api_key)
    response = await client.aio.models.generate_content(
        model=model_name,
        contents=prompt,
    )
    return response.text

async def generate_image(self, prompt: str, output_path: str) -> str:
    """Gemini API로 이미지를 생성한다.

    Args:
        prompt: 이미지 생성 프롬프트
        output_path: 저장할 파일 경로

    Returns:
        저장된 파일 경로
    """
    from google import genai

    client = genai.Client(api_key=self._api_key)
    response = await client.aio.models.generate_content(
        model=self.get_model("image_generation"),
        contents=prompt,
    )

    # 이미지 바이트 추출 및 저장
    out = Path(output_path)
    out.parent.mkdir(parents=True, exist_ok=True)

```

```
for part in response.candidates[0].content.parts:
    if part.inline_data:
        out.write_bytes(part.inline_data.data)
        return str(out)

raise RuntimeError("API 응답에 이미지 데이터가 없습니다.")
```

4.2 scripts/generate_images.py — 이미지 생성

"""Imagen 4로 씬별 이미지 생성.

scene-prompts.md에서 프롬프트를 추출하여 씬당 2-3장 이미지 생성.
인트로(Scene 1)/아웃트로(Scene 20)는 영상이므로 제외.

"""

```
import os
import re
import time
from pathlib import Path
```

```
def load_env():
    env_path = Path("config/api-keys.env")
    if env_path.exists():
        with open(env_path) as f:
            for line in f:
                line = line.strip()
                if line and not line.startswith("#") and "=" in line:
                    key, val = line.split("=", 1)
                    os.environ[key] = val
```

```
def parse_prompts(prompts_path: str) -> list[dict]:
```

"""scene-prompts.md에서 씬별 프롬프트를 파싱한다.

두 가지 포맷을 지원:

- 단일 프롬프트: ****Prompt:** ... **Audio Direction****
- 멀티 이미지: ****Image 1 Prompt:** ... **Ken Burns:** / **Image 2 Prompt:** ...**

"""

```
text = Path(prompts_path).read_text(encoding="utf-8")
scenes = []
```

인트로 파트 파싱 (### Intro Part A/B)

```
intro_parts = re.split(r"(?=### Intro Part [A-Z]:)", text)
```

```
for ip in intro_parts:
```

```
    intro_match = re.match(r"### Intro Part ([A-Z]): (.+)", ip)
```

```
    if not intro_match:
```

```
        continue
```

```
    part_letter = intro_match.group(1)
```

```
    title = intro_match.group(2).strip()
```

```
    prompt_match = re.search(
```

```
        r"\s*\s*Prompt:\s*\s*\n(.+?)(?=\n\s*\s*Audio Direction)", ip, re.DOTALL
    )
```

```
    prompt = prompt_match.group(1).strip() if prompt_match else ""
```

```
    if prompt:
```

```
        # 인트로는 scene_number=0, 파트별로 이미지 1장씩
```

```
        scenes.append({
```

```
            "scene_number": 0,
```

```
            "title": f"Intro {part_letter}: {title}",
```

```
            "prompt": prompt,
```

```
            "image_prompts": [prompt],
```

```
            "_intro_part": part_letter,
```

```
        })
```

```

parts = re.split(r"(?=== Scene \d+:)", text)

for part in parts:
    match = re.match(r"=== Scene (\d+): (.+)", part)
    if not match:
        continue
    scene_num = int(match.group(1))
    title = match.group(2).strip()

    # 멀티 이미지 포맷: **Image N Prompt:**
    image_prompts = re.findall(
        r"\*\*Image \d+ Prompt:\*\*\n(.+?)(?=\n\*\*(?:Ken Burns|Consistency|Image \d+
Prompt))",
        part,
        re.DOTALL,
    )

    if image_prompts:
        # 멀티 이미지: 각 프롬프트를 개별 저장
        scenes.append({
            "scene_number": scene_num,
            "title": title,
            "prompt": image_prompts[0].strip(),
            "image_prompts": [p.strip() for p in image_prompts],
        })
    else:
        # 단일 프롬프트 (기존 포맷)
        prompt_match = re.search(
            r"\*\*Prompt:\*\*\n(.+?)(?=\n\*\*Audio Direction)", part, re.DOTALL
        )
        prompt = prompt_match.group(1).strip() if prompt_match else ""
        scenes.append({"scene_number": scene_num, "title": title, "prompt": prompt})

return scenes

def generate_scene_images(scenes: list[dict], scenes_dir: str, images_per_scene: int = 2):
    """Imagen 4로 썬별 이미지를 생성한다."""
    from google import genai

    load_env()
    client = genai.Client(api_key=os.environ["GEMINI_API_KEY"])

    print(f"\n=== Imagen 4 이미지 생성 ({len(scenes)}썬 × {images_per_scene}장) ===\n")

    total = 0
    errors = 0

    for scene in scenes:
        scene_dir = Path(scenes_dir) / f"scene-{scene['scene_number']:02d}"
        scene_dir.mkdir(parents=True, exist_ok=True)

        # 이미 이미지가 있으면 스킵
        existing = list(scene_dir.glob("img_*.png"))
        if len(existing) >= images_per_scene:

```

```

        print(f" Scene {scene['scene_number']:02d}: 이미 {len(existing)}장 존재 - 스킵")
        total += len(existing)
        continue

# 멀티 이미지 프롬프트가 있으면 그것을 사용, 없으면 기존 방식
has_multi = "image_prompts" in scene and scene["image_prompts"]

if has_multi:
    actual_count = len(scene["image_prompts"])
else:
    actual_count = images_per_scene

base_prompt = scene["prompt"]

# 단일 프롬프트 모드: 각 이미지에 약간 다른 앵글/구도 지시 추가
variations = [
    "Medium shot, centered composition.",
    "Close-up detail shot, shallow depth of field.",
    "Wide establishing shot, environmental context.",
]

for j in range(actual_count):
    img_path = scene_dir / f"img_{j+1:02d}.png"
    if img_path.exists():
        print(f" Scene {scene['scene_number']:02d} img_{j+1}: 존재 - 스킵")
        total += 1
        continue

# 멀티 이미지: 개별 프롬프트 사용 / 단일: 변형 추가
if has_multi:
    full_prompt = scene["image_prompts"][j]
else:
    variation = variations[j % len(variations)]
    full_prompt = f"{base_prompt}\n\n{variation}\n\nPhotorealistic, cinematic,
4K, film grain."

print(f" Scene {scene['scene_number']:02d} img_{j+1}: 생성 중...")

try:
    response = client.models.generate_images(
        model="imagen-4.0-generate-001",
        prompt=full_prompt,
        config={"number_of_images": 1},
    )

    if response.generated_images:
        img_data = response.generated_images[0].image.image_bytes
        img_path.write_bytes(img_data)
        print(f"    → 완료 ✓ ({len(img_data) // 1024}KB)")
        total += 1
    else:
        print(f"    → 이미지 없음")
        errors += 1

except Exception as e:

```

```

error_str = str(e)
if "429" in error_str or "RESOURCE_EXHAUSTED" in error_str:
    print(f"    → Rate limit - 60초 대기 후 재시도")
    time.sleep(60)
    try:
        response = client.models.generate_images(
            model="imagen-4.0-generate-001",
            prompt=full_prompt,
            config={"number_of_images": 1},
        )
        if response.generated_images:
            img_data = response.generated_images[0].image.image_bytes
            img_path.write_bytes(img_data)
            print(f"    → 재시도 성공 ✓")
            total += 1
        else:
            errors += 1
    except Exception as e2:
        print(f"    → 재시도 실패: {e2}")
        errors += 1
else:
    print(f"    → 에러: {e}")
    errors += 1

time.sleep(3) # API 속도 제한 방지

print(f"\n이미지 생성 완료: {total}장 성공, {errors}개 실패\n")

def main():
    import argparse
    parser = argparse.ArgumentParser(description="Imagen 4 씬별 이미지 생성")
    parser.add_argument("--project", default="projects/2026-03-24-senior-content")
    parser.add_argument("--images-per-scene", type=int, default=2)
    parser.add_argument("--start", type=int, default=2, help="시작 씬 번호 (인트로 제외)")
    parser.add_argument("--end", type=int, default=19, help="끝 씬 번호 (아웃트로 제외)")
    args = parser.parse_args()

    prompts_path = Path(args.project) / "scene-prompts.md"
    scenes_dir = Path(args.project) / "scenes"

    all_scenes = parse_prompts(str(prompts_path))
    # 인트로/아웃트로 제외한 중간 씬만
    target_scenes = [s for s in all_scenes if args.start <= s["scene_number"] <= args.end]

    print(f"대상 씬: {len(target_scenes)}개 (Scene {args.start}~{args.end})")
    generate_scene_images(target_scenes, str(scenes_dir), args.images_per_scene)

if __name__ == "__main__":
    main()

```


4.3 scripts/produce.py — 영상 + TTS 생성

```

"""영상 제작 파이프라인.

scene-prompts.md의 프롬프트로 Veo 3 영상 + ElevenLabs TTS 생성 후 MoviePy로 합성.
"""

import os
import re
import time
import argparse
from pathlib import Path

import yaml

def load_env():
    """config/api-keys.env에서 환경변수 로드."""
    env_path = Path("config/api-keys.env")
    if env_path.exists():
        with open(env_path) as f:
            for line in f:
                line = line.strip()
                if line and not line.startswith("#") and "=" in line:
                    key, val = line.split("=", 1)
                    os.environ[key] = val

def parse_scene_prompts(prompts_path: str) -> list[dict]:
    """scene-prompts.md에서 씬별 프롬프트를 파싱한다."""
    text = Path(prompts_path).read_text(encoding="utf-8")
    scenes = []

    # ### Scene N: Title 패턴으로 분할
    pattern = r"### Scene (\d+): (.+?)(?=\n)"
    parts = re.split(r"(?=### Scene \d+:)", text)

    for part in parts:
        match = re.match(r"### Scene (\d+): (.+)", part)
        if not match:
            continue

        scene_num = int(match.group(1))
        title = match.group(2).strip()

        # Duration 추출
        dur_match = re.search(r"\*\*Duration:\*\*\s*(\d+)\s*seconds", part)
        duration = int(dur_match.group(1)) if dur_match else 30

        # Prompt 추출
        prompt_match = re.search(r"\*\*Prompt:\*\*\n(.+?)(?=\n\*\*Audio Direction)", part,
re.DOTALL)
        prompt = prompt_match.group(1).strip() if prompt_match else ""

        # Audio Direction 추출
        audio_match = re.search(r"\*\*Audio Direction:\*\*\n(.+?)(?=\n\*\*Consistency)",
part, re.DOTALL)

```

```

        audio = audio_match.group(1).strip() if audio_match else ""

        scenes.append({
            "scene_number": scene_num,
            "title": title,
            "duration_sec": duration,
            "prompt": prompt,
            "audio_direction": audio,
        })

    return scenes

def parse_narrations(script_path: str) -> list[dict]:
    """script.md에서 씬별 나레이션 텍스트를 파싱한다."""
    text = Path(script_path).read_text(encoding="utf-8")
    narrations = []

    parts = re.split(r"(?==## Scene \d+:)", text)
    for part in parts:
        match = re.match(r"## Scene (\d+): (.+)", part)
        if not match:
            continue

        scene_num = int(match.group(1))

        # 나레이션 추출 - 포맷: **\[나레이션\]** 텍스트... 또는 **\[나레이션\]**\n텍스트
        nar_match = re.search(
            r"\*\*\[?\\[나레이션\\]?\\*?\\*?\\]?\\s*(.+?)\\(\\[?\\*\\*\\]?\\[시각 묘사]\\)",
            part, re.DOTALL,
        )
        narration = nar_match.group(1).strip() if nar_match else ""

        narrations.append({
            "scene_number": scene_num,
            "narration": narration,
        })

    return narrations

def generate_videos(scenes: list[dict], output_dir: str, model: str = "veo-3.0-fast-generate-001"):
    """Veo 3로 모든 씬의 영상을 생성한다."""
    from google import genai

    client = genai.Client(api_key=os.environ["GEMINI_API_KEY"])
    operations = []

    print(f"\n=== Veo 3 영상 생성 시작 ({len(scenes)} 씬) ===\n")

    for scene in scenes:
        scene_dir = Path(output_dir) / f"scene-{scene['scene_number']:02d}"
        scene_dir.mkdir(parents=True, exist_ok=True)

```

```

video_path = scene_dir / "visual.mp4"
if video_path.exists():
    print(f" Scene {scene['scene_number']:02d}: 이미 존재 - 스킵")
    operations.append({"scene": scene, "path": str(video_path), "operation": None,
"done": True})
    continue

print(f" Scene {scene['scene_number']:02d}: 생성 요청 중...")
max_retries = 3
for attempt in range(max_retries):
    try:
        operation = client.models.generate_videos(
            model=model,
            prompt=scene["prompt"],
        )
        print(f"    → Operation: {operation.name}")
        operations.append({
            "scene": scene,
            "path": str(video_path),
            "operation": operation,
            "done": False,
        })
        # API 속도 제한 방지 - 요청 간 30초 대기
        time.sleep(30)
        break
    except Exception as e:
        error_str = str(e)
        if "429" in error_str or "RESOURCE_EXHAUSTED" in error_str:
            wait = 60 * (attempt + 1)
            print(f"    → Rate limit - {wait}초 대기 후 재시도 ({attempt+1}/
{max_retries})")
            time.sleep(wait)
        else:
            print(f"    → 에러: {e}")
            operations.append({"scene": scene, "path": str(video_path),
"operation": None, "done": False, "error": error_str})
            break
    else:
        print(f"    → 최대 재시도 초과")
        operations.append({"scene": scene, "path": str(video_path), "operation": None,
"done": False, "error": "max retries exceeded"})

# 완료 대기
print(f"\n=== 영상 생성 대기 중... ===\n")
pending = [op for op in operations if op["operation"] and not op["done"]]

while pending:
    for op in pending:
        try:
            updated = client.operations.get(op["operation"])
            if updated.done:
                op["done"] = True
                # 다운로드
                if updated.response and updated.response.generated_videos:
                    video = updated.response.generated_videos[0]

```

```

        video_data = client.files.download(file=video.video)
        # video_data는 bytes 객체
        with open(op["path"], "wb") as f:
            if hasattr(video_data, "save"):
                video_data.save(f)
            elif isinstance(video_data, bytes):
                f.write(video_data)
            else:
                # iterator인 경우
                for chunk in video_data:
                    f.write(chunk)
        fsize = Path(op["path"]).stat().st_size
        print(f" Scene {op['scene']['scene_number']:02d}: 완료 ✓ ({fsize//
1024}KB)")

    else:
        print(f" Scene {op['scene']['scene_number']:02d}: 영상 없음")
        op["error"] = "No video in response"
except Exception as e:
    print(f" Scene {op['scene']['scene_number']:02d}: 폴링 에러 - {e}")

pending = [op for op in operations if op["operation"] and not op["done"]]
if pending:
    print(f" ... {len(pending)}개 대기 중")
    time.sleep(15)

success = sum(1 for op in operations if op["done"] and "error" not in op)
print(f"\n영상 생성 완료: {success}/{len(scenes)}\n")
return operations

def generate_voices(narrations: list[dict], output_dir: str, voice_id: str =
"EXAVITQu4vr4xnSDxMaL"):
    """ElevenLabs로 모든 씬의 보이스를 생성한다."""
    from elevenlabs import ElevenLabs

    client = ElevenLabs(api_key=os.environ["ELEVENLABS_API_KEY"])

    print(f"\n=== ElevenLabs TTS 생성 시작 ({len(narrations)}씬) ===\n")

    for nar in narrations:
        scene_dir = Path(output_dir) / f"scene-{nar['scene_number']:02d}"
        scene_dir.mkdir(parents=True, exist_ok=True)

        voice_path = scene_dir / "voice.mp3"
        if voice_path.exists():
            print(f" Scene {nar['scene_number']:02d}: 이미 존재 - 스킵")
            continue

        if not nar["narration"]:
            print(f" Scene {nar['scene_number']:02d}: 나레이션 없음 - 스킵")
            continue

        print(f" Scene {nar['scene_number']:02d}: TTS 생성 중...")
        try:
            audio = client.text_to_speech.convert(

```

```

        voice_id=voice_id,
        text=nar["narration"],
        model_id="eleven_multilingual_v2",
        voice_settings={
            "stability": 0.6,
            "similarity_boost": 0.75,
            "speed": 0.82,
        },
    )
    with open(voice_path, "wb") as f:
        for chunk in audio:
            f.write(chunk)
    print(f"    → 완료 ✓")
except Exception as e:
    print(f"    → 에러: {e}")

time.sleep(1) # 속도 제한 방지

print(f"\nTTS 생성 완료\n")

def composite(project_dir: str, video_format: str = "long"):
    """MoviePy로 최종 합성."""
    from moviepy import (
        AudioFileClip,
        ImageClip,
        VideoFileClip,
        concatenate_videoclips,
    )

    scenes_dir = Path(project_dir) / "scenes"
    output_dir = Path(project_dir) / "output"
    output_dir.mkdir(parents=True, exist_ok=True)

    print(f"\n=== MoviePy 합성 시작 ===\n")

    # 씬 수집
    scene_dirs = sorted(scenes_dir.iterdir())
    clips = []

    for scene_path in scene_dirs:
        if not scene_path.is_dir():
            continue

        visual = None
        voice = None

        for f in scene_path.iterdir():
            if f.stem == "visual" and f.suffix in (".mp4", ".mov", ".webm"):
                visual = f
            elif f.stem == "voice" and f.suffix in (".mp3", ".wav"):
                voice = f

        if not visual or not voice:
            print(f"    {scene_path.name}: 비주얼 또는 보이스 누락 - 스킵")

```

```

        continue

    print(f"  {scene_path.name}: 합성 중...")

    audio = AudioFileClip(str(voice))
    video = VideoFileClip(str(visual))

    # 비디오 길이를 오디오에 맞춤
    if video.duration < audio.duration:
        # 비디오가 짧으면 마지막 프레임 유지
        video = video.with_effects([lambda c: c.time_transform(lambda t: min(t,
video.duration - 0.1))])
        video = video.with_duration(audio.duration)
    elif video.duration > audio.duration:
        video = video.subclipped(0, audio.duration)

    video = video.with_audio(audio)
    clips.append(video)

if not clips:
    print("합성할 클립이 없습니다!")
    return

# 해상도
if video_format == "short":
    target_size = (1080, 1920)
else:
    target_size = (1920, 1080)

# 합성
final = concatenate_videoclips(clips, method="compose")
output_path = output_dir / "final.mp4"

final.write_videofile(
    str(output_path),
    fps=24,
    codec="libx264",
    audio_codec="aac",
    threads=4,
)

print(f"\n합성 완료: {output_path}")
print(f"총 길이: {final.duration:.1f}초 ({final.duration/60:.1f}분)")

final.close()
for clip in clips:
    clip.close()

def main():
    parser = argparse.ArgumentParser(description="영상 제작 파이프라인")
    parser.add_argument("--project", default="projects/2026-03-24-senior-content", help="프로젝트 디렉토리")
    parser.add_argument("--step", choices=["all", "video", "voice", "composite"],
default="all")

```

```

parser.add_argument("--model", default="veo-3.0-fast-generate-001", help="Veo 모델")
parser.add_argument("--voice-id", default="EXAVITQu4vr4xnSDxMaL", help="ElevenLabs
voice ID")
parser.add_argument("--format", choices=["short", "long"], default="long")
args = parser.parse_args()

load_env()

prompts_path = Path(args.project) / "scene-prompts.md"
script_path = Path(args.project) / "script.md"
scenes_dir = Path(args.project) / "scenes"

scenes = parse_scene_prompts(str(prompts_path))
narrations = parse_narrations(str(script_path))

print(f"프로젝트: {args.project}")
print(f"씬 수: {len(scenes)} (프롬프트), {len(narrations)} (나레이션)")
print(f"모델: {args.model}")
print(f"보이스: {args.voice_id}")

if args.step in ("all", "video"):
    generate_videos(scenes, str(scenes_dir), args.model)

if args.step in ("all", "voice"):
    generate_voices(narrations, str(scenes_dir), args.voice_id)

if args.step in ("all", "composite"):
    composite(args.project, args.format)

if __name__ == "__main__":
    main()

```


4.4 scripts/compositor.py — 영상 합성

```
"""MoviePy 합성/렌더링 스크립트.
```

```
썸별 비주얼(이미지/영상) + 보이스(MP3) → 최종 MP4 합성.
```

- 영상 썸: 그대로 사용 (인트로/아웃트로)
 - 이미지 썸: 썸당 2-3장 이미지에 Ken Burns(줌인/줌아웃) 효과 적용
- ```
"""
```

```
import argparse
import os
import random
from pathlib import Path

IMAGE_EXTS = {".png", ".jpg", ".jpeg", ".webp", ".bmp"}
VIDEO_EXTS = {".mp4", ".mov", ".webm", ".avi", ".mkv"}

def detect_scene_type(visual_path: str) -> str:
 ext = Path(visual_path).suffix.lower()
 if ext in IMAGE_EXTS:
 return "image"
 if ext in VIDEO_EXTS:
 return "video"
 raise ValueError(f"알 수 없는 비주얼 파일 형식: {ext} ({visual_path})")

def collect_scene_assets(scene_dir: Path) -> dict:
 """썸 디렉토리에서 비주얼/보이스 파일을 수집한다.

 이미지가 여러 장이면 images 리스트로 반환.
 영상이 있으면 video로 반환 (우선).
 """
 images = []
 video = None
 voice = None

 for f in sorted(scene_dir.iterdir()):
 ext = f.suffix.lower()
 if ext in VIDEO_EXTS and f.stem.startswith("visual"):
 video = str(f)
 elif ext in IMAGE_EXTS and (f.stem.startswith("visual") or
f.stem.startswith("img")):
 images.append(str(f))
 elif ext in {".mp3", ".wav", ".ogg"} and f.stem == "voice":
 voice = str(f)

 return {
 "video": video,
 "images": sorted(images),
 "voice": voice,
 "type": "video" if video else ("image" if images else None),
 }

def validate_project_dir(project_dir: str) -> list[dict]:
 scenes_dir = Path(project_dir) / "scenes"
```

```

if not scenes_dir.exists():
 return []

scenes = []
for scene_path in sorted(scenes_dir.iterdir()):
 if not scene_path.is_dir():
 continue
 assets = collect_scene_assets(scene_path)
 if assets["type"] and assets["voice"]:
 assets["name"] = scene_path.name
 scenes.append(assets)

return scenes

def build_timeline(scenes: list[dict]) -> list[dict]:
 timeline = []
 for i, scene in enumerate(scenes):
 timeline.append({"index": i, **scene})
 return timeline

def ken_burns_clip(image_path: str, duration: float, target_size: tuple, effect: str =
"auto") -> "VideoClip":
 """이미지에 Ken Burns(줌인/줌아웃) 효과를 적용한 클립을 생성한다.

 Args:
 image_path: 이미지 파일 경로
 duration: 클립 길이(초)
 target_size: (width, height) 출력 해상도
 effect: "zoom_in", "zoom_out", "auto" (자동 번갈아)
 """
 from moviepy import ImageClip
 import numpy as np

 w, h = target_size

 # 줌 범위: 1.0 ~ 1.15 (과하지 않게)
 if effect == "zoom_in":
 start_scale, end_scale = 1.0, 1.15
 elif effect == "zoom_out":
 start_scale, end_scale = 1.15, 1.0
 else:
 # auto: 랜덤
 if random.random() > 0.5:
 start_scale, end_scale = 1.0, 1.15
 else:
 start_scale, end_scale = 1.15, 1.0

 # 원본 비율 유지 + cover fit (화면을 채우고 넘치는 부분 크롭)
 from PIL import Image as PILImage
 orig = PILImage.open(image_path)
 orig_w, orig_h = orig.size
 orig.close()

```

```

줌 여유분(1.2x) 포함하여 cover 스케일 계산
cover_w = int(w * 1.2)
cover_h = int(h * 1.2)
scale_factor = max(cover_w / orig_w, cover_h / orig_h)
new_w = int(orig_w * scale_factor)
new_h = int(orig_h * scale_factor)

base = ImageClip(image_path)
base = base.resized((new_w, new_h))

def make_frame(t):
 progress = t / max(duration, 0.001)
 scale = start_scale + (end_scale - start_scale) * progress

 # 현재 스케일에서의 크롭 영역
 crop_w = int(w / scale)
 crop_h = int(h / scale)

 # 얼굴 우선 크롭: 수평은 중앙, 수직은 상단 35% 기준
 frame = base.get_frame(0)
 fh, fw = frame.shape[:2]
 cx = (fw - crop_w) // 2
 cy = int((fh - crop_h) * 0.35) # 상단 편향 - 인물 얼굴 유지

 cropped = frame[cy:cy + crop_h, cx:cx + crop_w]

 # 리사이즈
 from PIL import Image
 img = Image.fromarray(cropped)
 img = img.resize((w, h), Image.LANCZOS)
 return np.array(img)

from moviepy import VideoClip
clip = VideoClip(make_frame, duration=duration)
clip = clip.with_fps(24)

base.close()
return clip

def render(project_dir: str, output_path: str, video_format: str = "long") -> str:
 """전체 씬을 합성하여 최종 영상을 렌더링한다."""
 from moviepy import (
 AudioFileClip,
 VideoFileClip,
 concatenate_videoclips,
)

 scenes = validate_project_dir(project_dir)
 if not scenes:
 raise RuntimeError(f"프로젝트에 유효한 씬이 없습니다: {project_dir}")

 timeline = build_timeline(scenes)

 if video_format == "short":

```

```

 target_size = (1080, 1920)
 else:
 target_size = (1920, 1080)

 clips = []
 effects_cycle = ["zoom_in", "zoom_out"]

 for entry in timeline:
 audio = AudioFileClip(entry["voice"])
 duration = audio.duration

 if entry["type"] == "video":
 # 인트로/아웃트로: 영상 사용
 clip = VideoFileClip(entry["video"])
 if clip.duration < duration:
 loops = int(duration / clip.duration) + 1
 clip = concatenate_videoclips([clip] * loops).subclipped(0, duration)
 elif clip.duration > duration:
 clip = clip.subclipped(0, duration)
 clip = clip.resized(target_size)
 clip = clip.with_audio(audio)
 clips.append(clip)
 print(f" {entry['name']}: 영상 {duration:.1f}초")

 elif entry["type"] == "image":
 images = entry["images"]
 num_images = len(images)

 if num_images == 0:
 continue

 # 이미지별 시간 배분 (균등)
 per_image_dur = duration / num_images

 img_clips = []
 for j, img_path in enumerate(images):
 effect = effects_cycle[j % 2]
 img_clip = ken_burns_clip(img_path, per_image_dur, target_size, effect)
 img_clips.append(img_clip)

 scene_clip = concatenate_videoclips(img_clips, method="compose")
 scene_clip = scene_clip.with_audio(audio)
 clips.append(scene_clip)
 print(f" {entry['name']}: 이미지 {num_images}장 × Ken Burns ({duration:.1f}초)")

 final = concatenate_videoclips(clips, method="compose")

 out = Path(output_path)
 out.parent.mkdir(parents=True, exist_ok=True)
 final.write_videofile(
 str(out),
 fps=24,
 codec="libx264",
 audio_codec="aac",
 threads=4,

```

```

)

 print(f"\n렌더링 완료: {out}")
 print(f"총 길이는: {final.duration:.1f}초 ({final.duration / 60:.1f}분)")

 final.close()
 for clip in clips:
 clip.close()

 return str(out)

def main():
 parser = argparse.ArgumentParser(description="MoviePy 영상 합성 (Ken Burns)")
 parser.add_argument("--project", required=True, help="프로젝트 디렉토리 경로")
 parser.add_argument(
 "--format",
 choices=["short", "long"],
 default="long",
 help="영상 포맷 (short=9:16, long=16:9)",
)
 parser.add_argument("--output", default=None, help="출력 경로")
 args = parser.parse_args()

 output = args.output or str(Path(args.project) / "output" / "final.mp4")
 render(args.project, output, args.format)

if __name__ == "__main__":
 main()

```

## 4.5 scripts/capcut-project-gen.py — CapCut 프로젝트 자동 생성

이 스크립트는 프로젝트의 scripts/ 디렉토리에 배치한다.

```
#!/usr/bin/env python3
"""
capcut-project-gen.py - CapCut 프로젝트 자동 생성기

MP4 세그먼트 + WAV 오디오 + SRT 자막 → CapCut 프로젝트 폴더 생성.
CapCut을 열면 프로젝트 목록에 바로 표시됨.

Usage:
python3 capcut-project-gen.py config.json

config.json 예시:
{
 "name": "EP2-슬래시커맨드",
 "fps": 30,
 "canvas": { "width": 1920, "height": 1080 },
 "video": [
 "/abs/path/seg-01.mp4",
 "/abs/path/seg-02.mp4"
],
 "audio": [
 { "path": "/abs/path/voice-01.wav", "start": 0 },
 { "path": "/abs/path/voice-02.wav", "start": 15.5 },
 "/abs/path/voice-03.wav"
],
 "subtitle": "/abs/path/subtitles.srt",
 "subtitle_style": {
 "font_size": 5.0,
 "color": [1, 1, 1],
 "position_y": -0.8,
 "scale": 0.65
 }
}
}

- video: 경로 배열 → 타임라인에 순차 배치 (duration ffprobe 자동 감지)
- audio: 경로 또는 {path, start(초)} 배열 → start 생략 시 순차 배치
- subtitle: SRT 파일 경로 (선택)
- subtitle_style: 자막 스타일 (선택)
"""

import json
import uuid
import os
import sys
import subprocess
import time
import re
import shutil
from pathlib import Path

CAPCUT_DRAFTS = Path.home() / "Movies" / "CapCut" / "User Data" / "Projects" /
"com.lveditor.draft"
CAPCUT_FONT = "/Applications/CapCut.app/Contents/Resources/Font/SystemFont/en.ttf"

— Utilities —
```

```

def new_id():
 return str(uuid.uuid4()).upper()

def sec_to_us(sec):
 return int(float(sec) * 1_000_000)

def us_to_sec(us):
 return us / 1_000_000

def probe_file(path):
 """ffprobe로 파일 정보 추출 → {duration_us, width, height, has_audio}"""
 r = subprocess.run(
 ["ffprobe", "-v", "quiet", "-print_format", "json",
 "-show_streams", "-show_format", str(path)],
 capture_output=True, text=True,
)
 if r.returncode != 0:
 raise RuntimeError(f"ffprobe 실패: {path}")
 d = json.loads(r.stdout)
 dur = sec_to_us(float(d["format"]["duration"]))
 w, h, has_audio = 1920, 1080, False
 for s in d.get("streams", []):
 if s.get("codec_type") == "video":
 w = s.get("width", 1920)
 h = s.get("height", 1080)
 if s.get("codec_type") == "audio":
 has_audio = True
 return {"duration_us": dur, "width": w, "height": h, "has_audio": has_audio}

def parse_srt(srt_path):
 """SRT 파일 파싱 → [{text, start_us, duration_us}, ...]"""
 content = Path(srt_path).read_text(encoding="utf-8")
 blocks = re.split(r"\n\s*\n", content.strip())
 result = []
 for block in blocks:
 lines = block.strip().split("\n")
 if len(lines) < 3:
 continue
 m = re.match(
 r"(\d{2}):(\d{2}):(\d{2})[,\.](\d{3})\s*-->\s*(\d{2}):(\d{2}):(\d{2})[,\.](\d{3})",
 lines[1],
)
 if not m:
 continue
 g = [int(x) for x in m.groups()]
 start_ms = g[0] * 3600000 + g[1] * 60000 + g[2] * 1000 + g[3]
 end_ms = g[4] * 3600000 + g[5] * 60000 + g[6] * 1000 + g[7]
 text = "\n".join(lines[2:])
 result.append({

```



```

 "text": text,
 "start_us": start_ms * 1000,
 "duration_us": (end_ms - start_ms) * 1000,
 })
 return result

— Material Factories —————

def make_speed():
 mid = new_id()
 return mid, {"curve_speed": None, "id": mid, "mode": 0, "speed": 1.0, "type": "speed"}

def make_placeholder_info():
 mid = new_id()
 return mid, {
 "error_path": "", "error_text": "", "id": mid,
 "meta_type": "none", "res_path": "", "res_text": "",
 "type": "placeholder_info",
 }

def make_canvas():
 mid = new_id()
 return mid, {
 "album_image": "", "blur": 0.0, "color": "", "id": mid,
 "image": "", "image_id": "", "image_name": "",
 "source_platform": 0, "team_id": "", "type": "canvas_color",
 }

def make_sound_channel_mapping():
 mid = new_id()
 return mid, {"audio_channel_mapping": 0, "id": mid, "is_config_open": False, "type":
"none"}

def make_material_color():
 mid = new_id()
 return mid, {
 "gradient_angle": 90.0, "gradient_colors": [], "gradient_percents": [],
 "height": 0.0, "id": mid, "is_color_clip": False, "is_gradient": False,
 "solid_color": "", "width": 0.0,
 }

def make_vocal_separation():
 mid = new_id()
 return mid, {
 "choice": 0, "enter_from": "", "final_algorithm": "", "id": mid,
 "production_path": "", "removed_sounds": [], "time_range": None,
 "type": "vocal_separation",
 }

```

```
def make_beat():
 mid = new_id()
 return mid, {
 "ai_beats": {
 "beats_path": "", "beats_url": "",
 "melody_path": "", "melody_percents": [], "melody_url": "",
 },
 "enable_ai_beats": False, "gear": 404, "gear_count": 0,
 "id": mid, "mode": 404, "type": "beats",
 "user_beats": [], "user_delete_ai_beats": None,
 }
}
```

```
def make_material_animation():
 mid = new_id()
 return mid, {
 "animations": [], "id": mid,
 "multi_language_current": "none", "type": "sticker_animation",
 }
}
```

# — Track Material Factories —————

```
def make_video_material(filepath, info):
 mid = new_id()
 local_id = str(uuid.uuid4())
 mat = {
 "aigc_type": "none",
 "audio_fade": None,
 "cartoon_path": "",
 "category_id": "",
 "category_name": "local",
 "check_flag": 62978047,
 "content_feature_info": None,
 "corner_pin": None,
 "crop": {
 "lower_left_x": 0.0, "lower_left_y": 1.0,
 "lower_right_x": 1.0, "lower_right_y": 1.0,
 "upper_left_x": 0.0, "upper_left_y": 0.0,
 "upper_right_x": 1.0, "upper_right_y": 0.0,
 },
 "crop_ratio": "free",
 "crop_scale": 1.0,
 "duration": info["duration_us"],
 "extra_type_option": 0,
 "formula_id": "",
 "freeze": None,
 "has_audio": info["has_audio"],
 "has_sound_separated": False,
 "height": info["height"],
 "id": mid,
 "intensifies_path": "",
 "is_ai_generate_content": False,
 "is_copyright": False,
```

```

 "is_text_edit_overdub": False,
 "is_unified_beauty_mode": False,
 "local_id": "",
 "local_material_from": "",
 "local_material_id": local_id,
 "material_id": "",
 "material_name": Path(filepath).name,
 "material_url": "",
 "matting": {
 "flag": 0, "has_use_quick_brush": False,
 "has_use_quick_eraser": False, "interactiveTime": [],
 "path": "", "strokes": [],
 },
 "media_path": "",
 "multi_camera_info": None,
 "object_locked": None,
 "origin_material_id": "",
 "path": filepath,
 "picture_from": "none",
 "request_id": "",
 "reverse_path": "",
 "source": 0,
 "source_platform": 0,
 "stable": {"matrix_path": "", "stable_level": 0, "time_range": {"duration": 0,
"start": 0}},
 "team_id": "",
 "type": "video",
 "video_algorithm": {
 "algorithms": [], "path": "",
 },
 "width": info["width"],
 }
 return mid, local_id, mat

```

```
def make_audio_material(filepath, name, dur_us):
```

```

 mid = new_id()
 local_id = str(uuid.uuid4())
 mat = {
 "app_id": 0,
 "category_id": "",
 "category_name": "local",
 "check_flag": 1,
 "copyright_limit_type": "none",
 "duration": dur_us,
 "effect_id": "",
 "formula_id": "",
 "id": mid,
 "intensifies_path": "",
 "is_ai_clone_tone": False,
 "is_ugc": False,
 "local_material_id": local_id,
 "music_id": str(uuid.uuid4()),
 "name": name,
 "path": filepath,

```

```

 "request_id": "",
 "resource_id": "",
 "source_platform": 0,
 "team_id": "",
 "text_id": "",
 "tone_category_id": "",
 "tone_category_name": "",
 "tone_effect_id": "",
 "tone_effect_name": "",
 "tone_platform": "",
 "tone_speaker": "",
 "tone_type": "",
 "type": "extract_music",
 "video_id": "",
 "wave_points": [],
 }
}
return mid, local_id, mat

def make_text_material(text, font_size=5.0, color=None):
 if color is None:
 color = [1, 1, 1]
 mid = new_id()
 # CapCut 형식: styles→text 키 순서, compact JSON (공백 없음)
 from collections import OrderedDict
 content_obj = OrderedDict([
 ("styles", [OrderedDict([
 ("fill", OrderedDict([
 ("content", OrderedDict([
 ("solid", OrderedDict([("color", color)])),
 ("render_type", "solid"),
])),
])),
 ("range", [0, len(text)]),
 ("size", font_size),
 ("font", OrderedDict([("path", CAPCUT_FONT), ("id", "")])),
])),
 ("text", text),
])
mat = {
 "add_type": 2,
 "alignment": 1,
 "background_alpha": 1.0,
 "background_color": "",
 "background_height": 0.14,
 "background_round_radius": 0.0,
 "background_style": 0,
 "background_width": 0.14,
 "bold_width": 0.0,
 "border_alpha": 1.0,
 "border_color": "",
 "border_width": 0.08,
 "check_flag": 7,
 "combo_info": {"text_templates": []},
 "content": json.dumps(content_obj, ensure_ascii=False, separators=(",", ":")),

```

```

"fixed_height": -1.0,
"fixed_width": -1.0,
"font_id": "",
"font_name": "",
"font_path": CAPCUT_FONT,
"font_resource_id": "",
"font_size": font_size,
"font_source_platform": 0,
"font_title": "none",
"font_url": "",
"fonts": [],
"force_apply_line_max_width": False,
"global_alpha": 1.0,
"group_id": "",
"has_shadow": False,
"id": mid,
"initial_scale": 1.0,
"inner_padding": -1.0,
"is_rich_text": False,
"italic_degree": 0,
"ktv_color": "",
"language": "",
"layer_weight": 1,
"letter_spacing": 0.0,
"line_feed": 1,
"line_max_width": 0.82,
"line_spacing": 0.02,
"multi_language_current": "none",
"name": "",
"original_size": [],
"preset_category": "",
"preset_category_id": "",
"preset_has_set_alignment": False,
"preset_id": "",
"preset_index": 0,
"preset_name": "",
"recognize_task_id": "",
"recognize_type": 0,
"relevance_segment": [],
"shadow_alpha": 0.9,
"shadow_angle": -45.0,
"shadow_color": "",
"shadow_distance": 0.04,
"shadow_point": {"x": 0.6401844, "y": 0.6401844},
"shadow_smoothing": 0.45,
"shape_clip_x": False,
"shape_clip_y": False,
"style_name": "",
"sub_type": 0,
"subtitle_keywords": None,
"text_alpha": 1.0,
"text_color": "",
"text_curve": None,
"text_preset_resource_id": "",
"text_size": 30,

```

```

 "text_to_audio_ids": [],
 "tts_auto_update": False,
 "type": "subtitle",
 "typesetting": 0,
 "underline": False,
 "underline_offset": 0.22,
 "underline_width": 0.05,
 "use_effect_default_color": True,
 "words": {"end_time": [], "start_time": [], "text": []},
 }
 return mid, mat

```

# — Segment Factories —————

```

def make_video_segment(material_id, extra_refs, dur_us, target_start_us):
 return {
 "caption_info": None,
 "cartoon": False,
 "clip": {
 "alpha": 1.0,
 "flip": {"horizontal": False, "vertical": False},
 "rotation": 0.0,
 "scale": {"x": 1.0, "y": 1.0},
 "transform": {"x": 0.0, "y": 0.0},
 },
 "color_correct_alg_result": "",
 "common_keyframes": [],
 "desc": "",
 "digital_human_template_group_id": "",
 "enable_adjust": True,
 "enable_adjust_mask": False,
 "enable_color_adjust_pro": False,
 "enable_color_correct_adjust": False,
 "enable_color_curves": True,
 "enable_color_match_adjust": False,
 "enable_color_wheels": True,
 "enable_hsl": False,
 "enable_hsl_curves": True,
 "enable_lut": True,
 "enable_mask_shadow": False,
 "enable_mask_stroke": False,
 "enable_smart_color_adjust": False,
 "enable_video_mask": True,
 "extra_material_refs": extra_refs,
 "group_id": "",
 "hdr_settings": {"intensity": 1.0, "mode": 1, "nits": 1000},
 "id": new_id(),
 "intensifies_audio": False,
 "is_loop": False,
 "is_placeholder": False,
 "is_tone_modify": False,
 "keyframe_refs": [],
 "last_nonzero_volume": 1.0,
 "lyric_keyframes": None,
 }

```

```

"material_id": material_id,
"raw_segment_id": "",
"render_index": 0,
"render_timerange": {"duration": 0, "start": 0},
"responsive_layout": {
 "enable": False, "horizontal_pos_layout": 0,
 "size_layout": 0, "target_follow": "", "vertical_pos_layout": 0,
},
"reverse": False,
"source": "segmentsourcenormal",
"source_timerange": {"duration": dur_us, "start": 0},
"speed": 1.0,
"state": 0,
"target_timerange": {"duration": dur_us, "start": target_start_us},
"template_id": "",
"template_scene": "default",
"track_attribute": 0,
"track_render_index": 0,
"uniform_scale": {"on": True, "value": 1.0},
"visible": True,
"volume": 1.0,
}

```

```

def make_audio_segment(material_id, extra_refs, dur_us, target_start_us):
 return {
 "caption_info": None,
 "cartoon": False,
 "clip": None,
 "color_correct_alg_result": "",
 "common_keyframes": [],
 "desc": "",
 "digital_human_template_group_id": "",
 "enable_adjust": False,
 "enable_adjust_mask": False,
 "enable_color_adjust_pro": False,
 "enable_color_correct_adjust": False,
 "enable_color_curves": True,
 "enable_color_match_adjust": False,
 "enable_color_wheels": True,
 "enable_hsl": False,
 "enable_hsl_curves": True,
 "enable_lut": False,
 "enable_mask_shadow": False,
 "enable_mask_stroke": False,
 "enable_smart_color_adjust": False,
 "enable_video_mask": True,
 "extra_material_refs": extra_refs,
 "group_id": "",
 "hdr_settings": None,
 "id": new_id(),
 "intensifies_audio": False,
 "is_loop": False,
 "is_placeholder": False,
 "is_tone_modify": False,
 }

```

```

 "keyframe_refs": [],
 "last_nonzero_volume": 1.0,
 "lyric_keyframes": None,
 "material_id": material_id,
 "raw_segment_id": "",
 "render_index": 0,
 "render_timerange": {"duration": 0, "start": 0},
 "responsive_layout": {
 "enable": False, "horizontal_pos_layout": 0,
 "size_layout": 0, "target_follow": "", "vertical_pos_layout": 0,
 },
 "reverse": False,
 "source": "segmentsourcenormal",
 "source_timerange": {"duration": dur_us, "start": 0},
 "speed": 1.0,
 "state": 0,
 "target_timerange": {"duration": dur_us, "start": target_start_us},
 "template_id": "",
 "template_scene": "default",
 "track_attribute": 0,
 "track_render_index": 2,
 "uniform_scale": None,
 "visible": True,
 "volume": 1.0,
}

```

```

def make_text_segment(material_id, extra_refs, dur_us, target_start_us, pos_y=-0.8,
scale=0.65):

```

```

 return {
 "caption_info": None,
 "cartoon": False,
 "clip": {
 "alpha": 1.0,
 "flip": {"horizontal": False, "vertical": False},
 "rotation": 0.0,
 "scale": {"x": scale, "y": scale},
 "transform": {"x": 0.0, "y": pos_y},
 },
 "color_correct_alg_result": "",
 "common_keyframes": [],
 "desc": "",
 "digital_human_template_group_id": "",
 "enable_adjust": False,
 "enable_adjust_mask": False,
 "enable_color_adjust_pro": False,
 "enable_color_correct_adjust": False,
 "enable_color_curves": True,
 "enable_color_match_adjust": False,
 "enable_color_wheels": True,
 "enable_hsl": False,
 "enable_hsl_curves": True,
 "enable_lut": False,
 "enable_mask_shadow": False,
 "enable_mask_stroke": False,
 }

```



```

"enable_smart_color_adjust": False,
"enable_video_mask": True,
"extra_material_refs": extra_refs,
"group_id": "",
"hdr_settings": None,
"id": new_id(),
"intensifies_audio": False,
"is_loop": False,
"is_placeholder": False,
"is_tone_modify": False,
"keyframe_refs": [],
"last_nonzero_volume": 1.0,
"lyric_keyframes": None,
"material_id": material_id,
"raw_segment_id": "",
"render_index": 14000,
"render_timerange": {"duration": 0, "start": 0},
"responsive_layout": {
 "enable": False, "horizontal_pos_layout": 0,
 "size_layout": 0, "target_follow": "", "vertical_pos_layout": 0,
},
"reverse": False,
"source": "segmentsourcenormal",
"source_timerange": None,
"speed": 1.0,
"state": 0,
"target_timerange": {"duration": dur_us, "start": target_start_us},
"template_id": "",
"template_scene": "default",
"track_attribute": 0,
"track_render_index": 1,
"uniform_scale": {"on": True, "value": 1.0},
"visible": True,
"volume": 1.0,
}

```

# — Project Generator —————

```

def generate_project(config, config_dir):
 name = config["name"]
 fps = config.get("fps", 30.0)
 canvas_w = config.get("canvas", {}).get("width", 1920)
 canvas_h = config.get("canvas", {}).get("height", 1080)
 sub_style = config.get("subtitle_style", {})

 project_dir = CAPCUT_DRAFTS / name
 project_dir.mkdir(parents=True, exist_ok=True)
 resources_dir = project_dir / "Resources"
 resources_dir.mkdir(exist_ok=True)

 def copy_to_project(src_path):
 """파일을 프로젝트 Resources/에 복사하고 복사된 경로 반환.
 CapCut은 샌드박스 앱이라 ~/Movies/CapCut/ 외부 파일에 접근 불가."""
 src = Path(src_path)

```

```

 dst = resources_dir / src.name
 if not dst.exists():
 shutil.copy2(str(src), str(dst))
 return str(dst)

Initialize all material buckets
materials = {
 "ai_translates": [], "audio_balances": [], "audio_effects": [],
 "audio_fades": [], "audio_pannings": [], "audio_pitch_shifts": [],
 "audio_track_indexes": [], "audios": [], "beats": [],
 "canvases": [], "chromas": [], "color_curves": [],
 "common_mask": [], "digital_human_model_dressing": [],
 "digital_humans": [], "drafts": [], "effects": [],
 "flowers": [], "green_screens": [], "handwrites": [],
 "hsl": [], "hsl_curves": [], "images": [],
 "log_color_wheels": [], "loudnesses": [],
 "manual_beautys": [], "manual_deformations": [],
 "material_animations": [], "material_colors": [],
 "multi_language_refs": [], "placeholder_infos": [],
 "placeholders": [], "plugin_effects": [],
 "primary_color_wheels": [], "realtime_denoises": [],
 "shapes": [], "smart_crops": [], "smart_relights": [],
 "sound_channel_mappings": [], "speeds": [],
 "stickers": [], "tail_leaders": [],
 "text_templates": [], "texts": [],
 "time_marks": [], "transitions": [],
 "video_effects": [], "video_radius": [],
 "video_shadows": [], "video_strokes": [],
 "video_trackings": [], "videos": [],
 "vocal_beautifys": [], "vocal_separations": [],
}

video_segments = []
audio_segments = []
text_segments = []
meta_materials = [] # draft_meta_info.draft_materials

def resolve_path(p):
 p = Path(p)
 if not p.is_absolute():
 p = (config_dir / p).resolve()
 return str(p)

— Videos —————
video_cursor = 0
for vpath_raw in config.get("video", []):
 vpath_orig = resolve_path(vpath_raw)
 info = probe_file(vpath_orig)
 vpath = copy_to_project(vpath_orig)
 dur = info["duration_us"]
 print(f" video: {Path(vpath).name} ({us_to_sec(dur):.2f}s) @
{us_to_sec(video_cursor):.2f}s")

 vid_id, vid_local_id, vid_mat = make_video_material(vpath, info)
 materials["videos"].append(vid_mat)

```

```

 sp_id, sp = make_speed()
 ph_id, ph = make_placeholder_info()
 cv_id, cv = make_canvas()
 sc_id, sc = make_sound_channel_mapping()
 mc_id, mc = make_material_color()
 vs_id, vs = make_vocal_separation()
 materials["speeds"].append(sp)
 materials["placeholder_infos"].append(ph)
 materials["canvases"].append(cv)
 materials["sound_channel_mappings"].append(sc)
 materials["material_colors"].append(mc)
 materials["vocal_separations"].append(vs)

 seg = make_video_segment(vid_id, [sp_id, ph_id, cv_id, sc_id, mc_id, vs_id], dur,
video_cursor)
 video_segments.append(seg)

 meta_materials.append({
 "ai_group_type": "", "create_time": int(time.time()),
 "duration": dur, "enter_from": 0,
 "extra_info": Path(vpath_orig).name,
 "file_Path": vpath,
 "height": info["height"], "width": info["width"],
 "id": vid_local_id, "import_time": int(time.time()),
 "import_time_ms": int(time.time() * 1_000_000),
 "item_source": 1, "md5": "", "metetype": "video",
 "roughcut_time_range": {"duration": dur, "start": 0},
 "sub_time_range": {"duration": -1, "start": -1},
 "type": 0,
 })

 video_cursor += dur

— Audios —————
audio_cursor = 0
for aitem in config.get("audio", []):
 if isinstance(aitem, str):
 apath_orig = resolve_path(aitem)
 astart = audio_cursor
 else:
 apath_orig = resolve_path(aitem["path"])
 astart = sec_to_us(aitem["start"]) if "start" in aitem else audio_cursor

 info = probe_file(apath_orig)
 apath = copy_to_project(apath_orig)
 dur = info["duration_us"]
 print(f" audio: {Path(apath).name} ({us_to_sec(dur):.2f}s) @
{us_to_sec(astart):.2f}s")

 aud_id, aud_local_id, aud_mat = make_audio_material(apath, Path(apath).name, dur)
 materials["audios"].append(aud_mat)

 sp_id, sp = make_speed()
 ph_id, ph = make_placeholder_info()

```

```

 bt_id, bt = make_beat()
 sc_id, sc = make_sound_channel_mapping()
 vs_id, vs = make_vocal_separation()
 materials["speeds"].append(sp)
 materials["placeholder_infos"].append(ph)
 materials["beats"].append(bt)
 materials["sound_channel_mappings"].append(sc)
 materials["vocal_separations"].append(vs)

 seg = make_audio_segment(aud_id, [sp_id, ph_id, bt_id, sc_id, vs_id], dur, astart)
 audio_segments.append(seg)

 meta_materials.append({
 "ai_group_type": "", "create_time": int(time.time()),
 "duration": dur, "enter_from": 0,
 "extra_info": Path(apath_orig).name,
 "file_Path": apath,
 "height": 0, "width": 0,
 "id": aud_local_id, "import_time": int(time.time()),
 "import_time_ms": int(time.time() * 1_000_000),
 "item_source": 1, "md5": "", "metetype": "music",
 "roughcut_time_range": {"duration": dur, "start": 0},
 "sub_time_range": {"duration": -1, "start": -1},
 "type": 0,
 })

 audio_cursor = astart + dur

— Subtitles (SRT) —————
srt_meta = []
subtitle_path = config.get("subtitle")
if subtitle_path:
 srt_file = resolve_path(subtitle_path)
 subs = parse_srt(srt_file)
 font_size = sub_style.get("font_size", 5.0)
 color = sub_style.get("color", [1, 1, 1])
 pos_y = sub_style.get("position_y", -0.8)
 scale = sub_style.get("scale", 0.65)
 print(f" subtitle: {len(subs)} entries from {Path(srt_file).name}")

 for sub in subs:
 txt_id, txt_mat = make_text_material(sub["text"], font_size, color)
 materials["texts"].append(txt_mat)

 anim_id, anim = make_material_animation()
 materials["material_animations"].append(anim)

 seg = make_text_segment(txt_id, [anim_id], sub["duration_us"], sub["start_us"],
 pos_y, scale)
 text_segments.append(seg)

 srt_meta.append({
 "ai_group_type": "", "create_time": 0,
 "duration": 0, "enter_from": 0,
 "extra_info": Path(srt_file).name,

```

```

 "file_Path": srt_file,
 "height": 0, "width": 0,
 "id": new_id(), "import_time": int(time.time()),
 "import_time_ms": -1,
 "item_source": 1, "md5": "", "metetype": "none",
 "roughcut_time_range": {"duration": -1, "start": -1},
 "sub_time_range": {"duration": -1, "start": -1},
 "type": 2,
 })

— Total duration —————
total_dur = 0
for segs in [video_segments, audio_segments, text_segments]:
 for s in segs:
 end = s["target_timerange"]["start"] + s["target_timerange"]["duration"]
 total_dur = max(total_dur, end)

— Build tracks —————
tracks = []
if video_segments:
 tracks.append({
 "attribute": 0, "flag": 0, "id": new_id(),
 "is_default_name": True, "name": "",
 "segments": video_segments, "type": "video",
 })
if text_segments:
 tracks.append({
 "attribute": 0, "flag": 0, "id": new_id(),
 "is_default_name": True, "name": "",
 "segments": text_segments, "type": "text",
 })
if audio_segments:
 tracks.append({
 "attribute": 0, "flag": 0, "id": new_id(),
 "is_default_name": True, "name": "",
 "segments": audio_segments, "type": "audio",
 })

— Build draft_info.json —————
draft_id = new_id()
now_us = int(time.time() * 1_000_000)

draft_info = {
 "canvas_config": {
 "background": None,
 "height": canvas_h,
 "ratio": "original",
 "width": canvas_w,
 },
 "color_space": 0,
 "config": {
 "adjust_max_index": 1,
 "attachment_info": [],
 "combination_max_index": 1,
 "export_range": None,
 },

```

```

 "extract_audio_last_index": 1,
 "lyrics_recognition_id": "",
 "lyrics_sync": True,
 "lyrics_taskinfo": [],
 "maintrack_adsorb": True,
 "material_save_mode": 0,
 "multi_language_current": "none",
 "multi_language_list": [],
 "multi_language_main": "none",
 "multi_language_mode": "none",
 "original_sound_last_index": 1,
 "record_audio_last_index": 1,
 "sticker_max_index": 1,
 "subtitle_keywords_config": None,
 "subtitle_recognition_id": "",
 "subtitle_sync": True,
 "subtitle_taskinfo": [],
 "system_font_list": [],
 "use_float_render": False,
 "video_mute": False,
 "zoom_info_params": None,
 },
 "cover": "",
 "create_time": int(time.time()),
 "draft_type": "",
 "duration": total_dur,
 "extra_info": "",
 "fps": float(fps),
 "free_render_index_mode_on": False,
 "function_assistant_info": "",
 "group_container": None,
 "id": draft_id,
 "is_drop_frame_timecode": False,
 "keyframe_graph_list": [],
 "keyframes": [],
 "last_modified_platform": {
 "app_id": 359289,
 "app_source": "cc",
 "app_version": "8.3.0",
 "device_id": "",
 "hard_disk_id": "",
 "mac_address": "",
 "os": "mac",
 "os_version": "",
 },
 "lyrics_effects": [],
 "materials": materials,
 "mutable_config": None,
 "name": name,
 "new_version": "",
 "path": str(project_dir),
 "platform": {
 "app_id": 359289,
 "app_source": "cc",
 "app_version": "8.3.0",

```

```

 "device_id": "",
 "hard_disk_id": "",
 "mac_address": "",
 "os": "mac",
 "os_version": "",
 },
 "relationships": [],
 "render_index_track_mode_on": False,
 "retouch_cover": None,
 "smart_ads_info": None,
 "source": "default",
 "static_cover_image_path": "",
 "time_marks": None,
 "tracks": tracks,
 "update_time": int(time.time()),
 "version": 360000,
}

— Write draft_info.json —————
draft_info_path = project_dir / "draft_info.json"
with open(draft_info_path, "w", encoding="utf-8") as f:
 json.dump(draft_info, f, ensure_ascii=False, separators=(",", ":"))
print(f"\n draft_info.json ({draft_info_path.stat().st_size / 1024:.0f}KB)")

— Write draft_meta_info.json —————
total_size = sum(
 f.stat().st_size for f in resources_dir.iterdir() if f.is_file()
)
draft_meta = {
 "cloud_draft_cover": False,
 "cloud_draft_sync": False,
 "cloud_package_completed_time": "",
 "draft_cloud_last_action_download": False,
 "draft_cloud_purchase_info": "",
 "draft_cloud_template_id": "",
 "draft_cloud_tutorial_info": "",
 "draft_cloud_videocut_purchase_info": "",
 "draft_cover": "draft_cover.jpg",
 "draft_deeplink_url": "",
 "draft_enterprise_info": {
 "draft_enterprise_extra": "",
 "draft_enterprise_id": "",
 "draft_enterprise_name": "",
 "enterprise_material": [],
 },
 "draft_fold_path": str(project_dir),
 "draft_id": draft_id,
 "draft_is_ai_shorts": False,
 "draft_is_cloud_temp_draft": False,
 "draft_is_invisible": False,
 "draft_materials": [
 {"type": 0, "value": meta_materials},
 {"type": 1, "value": []},
 {"type": 2, "value": srt_meta},
 {"type": 3, "value": []},
],
}

```

```

 {"type": 6, "value": []},
 {"type": 7, "value": []},
 {"type": 8, "value": []},
],
 "draft_materials_copied_info": [],
 "draft_name": name,
 "draft_need_rename_folder": False,
 "draft_new_version": "",
 "draft_removable_storage_device": "",
 "draft_root_path": str(CAPCUT_DRAFTS),
 "draft_segment_extra_info": [],
 "draft_timeline_materials_size_": total_size,
 "draft_type": "",
 "tm_draft_cloud_completed": "",
 "tm_draft_cloud_entry_id": -1,
 "tm_draft_cloud_modified": 0,
 "tm_draft_cloud_parent_entry_id": -1,
 "tm_draft_cloud_space_id": -1,
 "tm_draft_cloud_user_id": -1,
 "tm_draft_create": now_us,
 "tm_draft_modified": now_us,
 "tm_draft_removed": 0,
 "tm_duration": total_dur,
}
with open(project_dir / "draft_meta_info.json", "w", encoding="utf-8") as f:
 json.dump(draft_meta, f, ensure_ascii=False, indent=4)
print(f" draft_meta_info.json")

— Generate cover (black 1920x1080) —————
cover_path = project_dir / "draft_cover.jpg"
if not cover_path.exists():
 subprocess.run(
 ["ffmpeg", "-y", "-f", "lavfi", "-i",
 f"color=c=black:s={canvas_w}x{canvas_h}:d=1",
 "-frames:v", "1", str(cover_path)],
 capture_output=True,
)
 print(f" draft_cover.jpg")

— Update root_meta_info.json —————
root_meta_path = CAPCUT_DRAFTS / "root_meta_info.json"
if root_meta_path.exists():
 with open(root_meta_path, "r", encoding="utf-8") as f:
 root_meta = json.load(f)
else:
 root_meta = {"all_draft_store": []}

중복 제거 (같은 이름의 기존 프로젝트가 있으면 교체)
root_meta["all_draft_store"] = [
 d for d in root_meta["all_draft_store"]
 if d.get("draft_name") != name
]

root_meta["all_draft_store"].insert(0, {
 "cloud_draft_cover": False,

```



```

 "cloud_draft_sync": False,
 "draft_cloud_last_action_download": False,
 "draft_cloud_purchase_info": "",
 "draft_cloud_template_id": "",
 "draft_cloud_tutorial_info": "",
 "draft_cloud_videocut_purchase_info": "",
 "draft_cover": str(cover_path),
 "draft_fold_path": str(project_dir),
 "draft_id": draft_id,
 "draft_is_ai_shorts": False,
 "draft_is_cloud_temp_draft": False,
 "draft_is_invisible": False,
 "draft_is_web_article_video": False,
 "draft_json_file": str(draft_info_path),
 "draft_name": name,
 "draft_new_version": "",
 "draft_root_path": str(CAPCUT_DRAFTS),
 "draft_timeline_materials_size": total_size,
 "draft_type": "",
 "streaming_edit_draft_ready": True,
 "tm_draft_cloud_completed": "",
 "tm_draft_cloud_entry_id": -1,
 "tm_draft_cloud_modified": 0,
 "tm_draft_cloud_parent_entry_id": -1,
 "tm_draft_cloud_space_id": -1,
 "tm_draft_cloud_user_id": -1,
 "tm_draft_create": now_us,
 "tm_draft_modified": now_us,
 "tm_draft_removed": 0,
 "tm_duration": total_dur,
 })

 with open(root_meta_path, "w", encoding="utf-8") as f:
 json.dump(root_meta, f, ensure_ascii=False, indent=4)
 print(f" root_meta_info.json (updated)")

 print(f"\n=== 완료: {project_dir} ===")
 print(f" duration: {us_to_sec(total_dur):.1f}s")
 print(f" video: {len(video_segments)} segments")
 print(f" audio: {len(audio_segments)} segments")
 print(f" text: {len(text_segments)} segments")
 print(f"\n CapCut을 열면 '{name}' 프로젝트가 목록에 표시됩니다.")

— Main —————

def main():
 if len(sys.argv) < 2:
 print(__doc__)
 sys.exit(1)

 config_path = Path(sys.argv[1]).resolve()
 if not config_path.exists():
 print(f"설정 파일 없음: {config_path}")
 sys.exit(1)

```

```
with open(config_path, "r", encoding="utf-8") as f:
 config = json.load(f)

print(f"=== CapCut 프로젝트 생성: {config['name']} ===\n")
generate_project(config, config_path.parent)

if __name__ == "__main__":
 main()
```

# 제5부. 참고 문서

## 5.1 guides/CAPCUT\_GUIDE.md — CapCut 가이드

# CapCut 프로젝트 자동 생성 가이드

> 이 문서는 Claude Code 세션에서 CapCut 프로젝트를 코드로 생성할 때 참조하는 가이드다.

> 스크립트: `scripts/capcut-project-gen.py`

---

## 1. 핵심 제약: macOS 샌드박스

CapCut은 macOS 샌드박스 앱 (`com.lemon.lvoverseas`)이다.

**``~/Movies/CapCut/` 외부 파일에 접근 불가.** 절대 경로를 JSON에 넣어도 CapCut에서 "파일에 액세스할 수 없음" 오류가 발생한다.

해결: 모든 소재 파일(MP4, WAV, SRT)을 프로젝트 `Resources/` 폴더에 **복사**한 후 해당 경로를 사용. `capcut-project-gen.py`가 자동으로 처리한다.

---

## 2. 디렉토리 구조

~/Movies/CapCut/User Data/Projects/com.lveditor.draft/ |—— {프로젝트명}/ |—— draft\_info.json ← 타임라인 전체 데이터 (핵심) | |—— draft\_meta\_info.json ← 프로젝트 메타 + 소재 импорт 목록 | |—— Resources/ ← 소재 파일 복사본 (MP4, WAV 등)

- `draft\_info.json` - 트랙, 세그먼트, 머티리얼 전부 여기에
- `draft\_meta\_info.json` - CapCut이 프로젝트 목록에 표시하는 데 필요
- `Resources/` - 소재 파일 실체. 이 안에 없으면 CapCut이 못 읽음

---

## 3. 시간 단위

**``모든 시간값은 마이크로초 (microsecond).**

1초 = 1,000,000 1분 = 60,000,000 5.5초 = 5,500,000

변환 함수:

```
```python
def sec_to_us(sec): return int(float(sec) * 1_000_000)
def us_to_sec(us): return us / 1_000_000
```

4. draft_info.json 핵심 구조

```
{
  "canvas_config": { "width": 1920, "height": 1080, "ratio": "original" },
  "fps": 30.0,
  "duration": 전체_duration_us,
  "tracks": [ ... ],           // 트랙 배열 (비디오, 오디오, 텍스트)
  "materials": { ... },       // 머티리얼 저장소 (모든 소재 정보)
  "id": "UUID",
  "name": "프로젝트명"
}
```

4.1 tracks → segments 관계

```
tracks[0] (video) → segments[] → 각 segment.material_id → materials.videos[]
tracks[1] (audio) → segments[] → 각 segment.material_id → materials.audios[]
tracks[2] (text)  → segments[] → 각 segment.material_id → materials.texts[]
```

각 세그먼트는 **material_id** 로 머티리얼을 참조하고, **extra_material_refs** 로 부속 머티리얼(speed, placeholder 등)을 참조한다.

4.2 세그먼트 타입별 extra_material_refs (필수)

이걸 틀리면 CapCut이 프로젝트를 열다 크래시한다.

Video segment — 6개:

```
[speed_id, placeholder_info_id, canvas_id, sound_channel_mapping_id, material_color_id,
vocal_separation_id]
```

Audio segment — 5개:

```
[speed_id, placeholder_info_id, beat_id, sound_channel_mapping_id, vocal_separation_id]
```

Text segment — 1개:

```
[material_animation_id]
```

4.3 세그먼트 시간 배치

```
{
  "source_timerange": { "duration": 5000000, "start": 0 },
  "target_timerange": { "duration": 5000000, "start": 10000000 }
}
```

- **source_timerange** — 원본 소재에서 어디부터 어디까지 사용하는지
- **target_timerange** — 타임라인에서 어디에 배치하는지
- **start** = 타임라인상 시작 위치 (마이크로초)
- **duration** = 타임라인에서 차지하는 길이

비디오를 순차 배치하려면:

```
cursor = 0
for each video:
  segment.target_timerange.start = cursor
  segment.target_timerange.duration = video_duration
  cursor += video_duration
```

5. materials 버킷

materials 는 40개 이상의 배열 키를 가진 거대한 객체다. 주요 버킷:

버킷	용도	세그먼트 연결
videos	비디오 파일 정보	video segment → material_id
audios	오디오 파일 정보	audio segment → material_id
texts	텍스트/자막 정보	text segment → material_id
speeds	재생 속도	모든 segment → extra_material_refs
placeholder_infos	플레이스홀더	모든 segment → extra_material_refs
canvases	캔버스 배경	video segment → extra_material_refs
sound_channel_mappings	오디오 채널	video/audio → extra_material_refs
material_colors	색상 보정	video segment → extra_material_refs
vocal_separations	보컬 분리	video/audio → extra_material_refs
beats	비트 정보	audio segment → extra_material_refs
material_animations	애니메이션	text segment → extra_material_refs

빈 배열이라도 모든 버킷 키가 존재해야 한다. 누락되면 CapCut이 프로젝트를 못 연다.

전체 버킷 키 목록 (알파벳순):

```
ai_translates, audio_balances, audio_effects, audio_fades, audio_pannings,
audio_pitch_shifts, audio_track_indexes, audios, beats, canvases, chromas,
color_curves, common_mask, digital_human_model_dressing, digital_humans,
drafts, effects, flowers, green_screens, handwrites, hsl, hsl_curves,
images, log_color_wheels, loudnesses, manual_beautys, manual_deformations,
material_animations, material_colors, multi_language_refs, placeholder_infos,
placeholders, plugin_effects, primary_color_wheels, realtime_denoises,
shapes, smart_crops, smart_relights, sound_channel_mappings, speeds,
stickers, tail_leaders, text_templates, texts, time_marks, transitions,
video_effects, video_radius, video_shadows, video_strokes, video_trackings,
videos, vocal_beautifys, vocal_separations
```

6. 자막 (Text Material)

자동 생성의 한계

CapCut의 텍스트 material `content` 필드는 중첩 JSON 문자열이다:

```
{
  "content": "{\"styles\": [{\"fill\": {\"content\": {\"solid\": {\"color\": [1,1,1]}},
  \"render_type\": \"solid\"}}, {\"range\": [0,5], \"size\": 5.0, \"font\": {\"path\": \"Applications/CapCut.app/.../en.ttf\", \"id\": \"\"}}], \"text\": \"안녕하세요\"}"
}
```

이 형식을 코드로 생성할 수는 있지만, CapCut이 자체적으로 렌더링하는 텍스트 레이아웃과 100% 일치시키기 어렵다.

권장 방식: SRT 수동 импорт

1. `voice/subtitles.srt` 를 SRT 표준 형식으로 생성
2. CapCut에서 "자막 > 로컬 자막 가져오기"로 импорт
3. CapCut 내에서 스타일 일괄 적용

`capcut-project-gen.py` 로 텍스트 세그먼트를 넣으면 타이밍은 맞지만, 호흡 단위로 자연스럽게 분리하려면 사용자가 수동으로 편집하는 게 현실적이다.

7. capcut-project-gen.py 사용법

config.json 작성

```
{
  "name": "EP3-에이전트와MCP-v3",
  "fps": 30,
  "canvas": { "width": 1920, "height": 1080 },
  "video": [
    "/abs/path/seg-01.mp4",
    "/abs/path/seg-02.mp4"
  ],
  "audio": [
    { "path": "/abs/path/scene-01.wav", "start": 0 },
    { "path": "/abs/path/scene-02.wav", "start": 35.2 },
    "/abs/path/scene-03.wav"
  ],
  "subtitle": "/abs/path/subtitles.srt",
  "subtitle_style": {
    "font_size": 5.0,
    "color": [1, 1, 1],
    "position_y": -0.8,
    "scale": 0.65
  }
}
```

필드	필수	설명
name	O	CapCut 프로젝트명 (폴더명이 됨)
fps	X	기본 30
canvas	X	기본 1920x1080
video	O	MP4 파일 절대 경로 배열. 타임라인에 순차 배치
audio	X	WAV 파일. 문자열 또는 {path, start(초)} 객체
subtitle	X	SRT 파일 경로
subtitle_style	X	자막 스타일 오버라이드

audio 배치 규칙

- 문자열만 넣으면 순차 배치 (이전 오디오 끝난 직후)
- { "path": "...", "start": 초 } 형태면 지정한 시간에 배치
- 비디오와 별도 트랙이므로 시작 시간이 비디오와 다를 수 있음

실행

```
python3 scripts/capcut-project-gen.py config.json
```

실행하면: 1. `~/Movies/CapCut/User Data/Projects/com.lveditor.draft/{name}/` 생성 2. 모든 소재 파일을 `Resources/` 에 복사 3. `draft_info.json`, `draft_meta_info.json` 생성 4. CapCut을 열면 프로젝트 목록에 표시됨

ffprobe 필수

스크립트가 `ffprobe` 로 각 파일의 duration, 해상도, 오디오 유무를 자동 감지한다. `ffprobe` 가 PATH에 있어야 한다.

```
# 확인
which ffprobe
# 없으면
brew install ffmpeg
```

8. 흔한 실수와 해결

"파일에 액세스할 수 없음"

- 원인: 소재 파일이 `~/Movies/CapCut/` 외부에 있음
- 해결: `Resources/` 폴더에 복사. `capcut-project-gen.py` 는 자동으로 함

CapCut이 프로젝트를 못 열거나 크래시

- 원인 1: `extra_material_refs` 개수가 안 맞음 (video=6개, audio=5개, text=1개)
- 원인 2: `materials` 버킷에 키가 누락됨 (빈 배열이라도 있어야 함)
- 원인 3: `material_id` 가 실제 머티리얼과 매칭 안 됨
- 해결: `capcut-project-gen.py` 를 사용하면 이 문제가 발생하지 않음

프로젝트가 CapCut 목록에 안 나옴

- 원인: CapCut이 실행 중일 때 프로젝트를 생성함
- 해결: CapCut을 종료하고 스크립트 실행 → CapCut 재실행

자막이 안 보임 / 깨짐

- 원인: `content` 필드의 중첩 JSON 문자열 형식 오류
- 해결: SRT를 CapCut에서 수동 임포트하는 것을 권장

오디오 싱크가 안 맞음

- 원인: `target_timerange.start` 가 비디오와 맞지 않음
- 해결: audio config에 `"start": 초` 값을 명시적으로 지정

9. 수동 편집이 필요한 작업

자동 생성 후 CapCut에서 수동으로 해야 하는 것들:

1. **SRT 자막 импорт** — "자막 > 로컬 자막 가져오기"
2. **자막 호흡 단위 편집** — 자동 SRT(청크 단위)를 자연스러운 호흡 단위로 분리
3. **비디오/오디오 미세 싱크 조정** — 0.1~0.3초 단위 미세 조정
4. **전환 효과 추가** — 코드 생성은 가능하나 CapCut 내장 이펙트 선택이 더 빠름
5. **BGM 추가** — CapCut 내장 음악 라이브러리에서 선택

10. 새 프로젝트 생성 체크리스트

1. ☐ 비디오 MP4 파일들 준비 (절대 경로 확인)
2. ☐ 오디오 WAV 파일들 준비 (썸네일 또는 전체)
3. ☐ SRT 자막 파일 준비 (선택)
4. ☐ config.json 작성
5. ☐ ffprobe 설치 확인 (which ffprobe)
6. ☐ CapCut 종료
7. ☐ python3 scripts/capcut-project-gen.py config.json 실행
8. ☐ CapCut 실행 → 프로젝트 목록에서 확인
9. ☐ SRT 수동 импорт (필요 시)
10. ☐ 싱크 미세 조정

5.2 docs/video-prompt-guide.md - 영상 프롬프트 가이드

```markdown

# Kling AI 영상 생성 프롬프트 최적화 가이드

> 소스: fal.ai(공식 파트너), VEED, atlabs.ai, RunDiffusion, hixx.ai, Ambience AI 종합

---

## 1. 기본 프롬프트 구조

[피사체] + [피사체 묘사] + [동작/움직임] + [배경/씬] + [썸 묘사] + [카메라/조명/분위기]

**\*\*원칙:\*\***

- 동시 동작 2개 이하 (복수 동작은 왜곡 유발)
- 숫자 사용 자제 ("5그루 나무" 같은 개수 명시 시 일관성 붕괴)
- 2.5 Turbo: 60단어 이하 / 2.6 Pro: 5~7개 환경 요소까지 허용

---

**## 2. 물리 반응 및 자연스러운 움직임 개선**

**### 움직임을 구체적 단계로 분해**

| 나쁜 예 | 좋은 예 |  
 |---|---|  
 | `the ball moves` | `the ball rolls forward, bounces once on the grass, then slows to a stop` |  
 | `he kicks the ball` | `his right foot swings back, makes contact with the ball, the ball arcs upward and forward` |  
 | `wear sunglasses` | `slowly raises her hand, grasps the sunglasses, places them on her face` |

**### 물리 반응 항상 키워드**

- `realistic physics`, `natural momentum`, `gravity-accurate`
- `gradual deceleration`, `smooth follow-through`
- `fabric ripples naturally`, `fluid dynamics`, `realistic bounce`
- 속도 수식어: `slowly`, `gently`, `smoothly`, `steadily` → 급격한 움직임 억제
- `then settles back` / `then comes to rest` → 움직임 끝 상태 명시 (99% 멈춤 현상 해결)

**### 공 움직임 특화**

The soccer ball is struck cleanly, rolls across the wet grass with realistic spin, kicks up a small spray of water, and slows gradually near the goal line. Natural ball physics, realistic momentum.

**### 사람 운동 동작 특화**

An athlete in mid-stride, arms pumping naturally at 90-degree angle, feet landing heel-to-toe with realistic weight transfer. Smooth, biomechanically accurate running motion.

---

### ## 3. 과도한 액션 억제

| 원인 | 해결책 |  
 |---|---|  
 | 열린 결말 동작 | 종료 상태 추가: `then settles`, `gradually stops` |  
 | 복수 동작 경쟁 | 동작 수 1~2개로 제한 |  
 | 모호한 공간 언어 | `positioned near`, `making contact with` 등 공간 명시 |  
 | 강도 미지정 | `slightly`, `gently`, `subtly`, `barely` 등 강도 수식어 추가 |

**\*\*억제 키워드:\*\***

- `minimal camera movement`
- `subtle motion only`
- `gentle, restrained movement`
- `static background, only subject moves`
- `slow and deliberate motion`

---

### ## 4. Image-to-Video 모드 최적화

### 핵심: 이미지 내용을 재설명하지 말 것

이미지가 씬을 정의하므로 프롬프트는 **\*\*무엇이 움직이는지, 어떻게 움직이는지\*\***에만 집중.

나쁜 예: "A man in a blue shirt standing in a forest. He has brown hair and..."

좋은 예: "Man slowly looks up toward the sky, light breeze moves his hair and shirt collar. Background trees sway gently."

### Image-to-Video 전용 공식

[무엇이 움직이는가] + [어떻게 움직이는가] + [배경 움직임(있다면)] + [카메라 동작]

### ### 주의사항

- 워터마크, 텍스트 없는 클린 이미지 사용
- 움직임 방향을 이미지 구도와 일치 (인물이 오른쪽 보면 카메라도 오른쪽 패닝)
- Elements 기능: 참조 이미지 2~4장이 최적

---

### ## 5. 네거티브 프롬프트

> Kling의 네거티브 프롬프트는 Stable Diffusion 대비 **\*\*효과가 약함\*\***. 긍정 프롬프트로 우회하는 것이 더 효과적.

### ### 고정 삽입 권장 목록

**\*\*품질:\*\***

blur, motion blur, distortion, compression artifacts, pixelation, low quality, poor lighting, flickering, noise, grain

**\*\*신체/물리 이상:\*\***

extra limbs, deformed hands, morphing faces, unnatural physics, body distortion, limb duplication, facial glitching

**\*\*영상 품질:\*\***

watermark, text overlay, logo, subtitle, letterbox, camera shake, unstable footage, jump cuts

---

## ## 6. 카메라 무브먼트 제어

### ### 전후 이동 (깊이감)

| 명령어 | 효과 |  
 |---|---|  
 | `slow dolly in` | 감정적 강조, 긴장감 |  
 | `dolly out` | 피사체 고립, 공간 확장 |  
 | `dolly zoom (vertigo effect)` | 배경 왜곡, 심리적 충격 |  
 | `slow push-in` | 집중, 친밀감 |

### ### 수평/수직 이동

| 명령어 | 효과 |  
 |---|---|  
 | `pan left / pan right` | 공간 탐색, 풍경 |  
 | `side tracking shot` | 달리는 피사체 추적 |  
 | `tilt up / tilt down` | 위압감, 규모 표현 |

### ### 회전/특수

| 명령어 | 효과 |  
 |---|---|  
 | `360-degree orbit around subject` | 캐릭터 소개, 제품 쇼케이스 |  
 | `handheld shaky cam` | 다큐멘터리, 긴박감 |  
 | `static locked-off shot` | 안정적, 관찰자 시점 |

### ### 초점/특수 기법

| 명령어 | 효과 |  
 |---|---|  
 | `rack focus from foreground to background` | 시선 유도 |  
 | `POV shot` | 몰입감 |  
 | `drone aerial shot` | 환경 설정 |  
 | `shallow depth of field` | 배경 분리 |

### ### 카메라 + 동작 조합 공식

[피사체] + [동작] + [배경] + [카메라 명령어] + [조명/분위기]

---

## ## 7. 모델별 최적화

| 모델           | 프롬프트 복잡도           | 특징                 |
|--------------|--------------------|--------------------|
| 1.6 Standard | 단순 (단일 피사체, 동작 1개) | Motion Brush 독점    |
| 2.5 Turbo    | 60단어 이하, 요소 3~4개   | 속도 최적화             |
| 2.6 Pro      | 요소 5~7개, 복잡 조명 가능  | 고품질, 세밀한 표현        |
| 3.0          | 멀티샷 스토리보드 (최대 6샷)  | 씬 단위 연속 생성, 15초 지원 |

---

## ## 8. 프로젝트 적용 체크리스트

1. **image-to-video 시** 이미지 재설명 금지 - 움직임만 기술
2. **스포츠 씬** 물리 단계 분해 + `realistic physics, natural momentum`
3. **모든 동작 끝**에 `then settles` / `comes to rest` 추가
4. **네거티브 프롬프트** 고정 삽입: `morphing faces, extra limbs, unnatural physics, body distortion, flickering`
5. **카메라** 드라마틱 씬은 `slow dolly in`, 전체 씬은 `static locked-off shot`
6. **동시 동작 2개 이하** 유지

---

## ## 9. 금지 단어 필터

Kling은 설명 없이 특정 단어를 필터링함.

확인된 금지어: `chest`, `pig`, `exorcism`, `filthy` 등.

→ 동의어로 대체하거나 이미지 생성 후 animate 방식으로 우회.

## 제6부. 프로젝트 구축 실행 순서

Claude Code에게 이 문서를 전달한 후 아래 순서로 요청한다:

1. 디렉토리 구조 생성
 

```
mkdir -p auto-video/{agents/
{phase1,phase2},brand,config,docs,guides,scripts,projects,.claude/commands}
```
2. 설정 파일 생성
  - pyproject.toml (제2부 2.1)
  - .gitignore (제2부 2.2)
  - config/models.yaml (제2부 2.3)
  - config/api-keys.env (제1부 1.2 - 사용자가 직접 키 입력)
  - CLAUDE.md (제2부 2.4)
  - .claude/commands/create-c.md (제2부 2.5)
  - brand/ YAML 4개 (제2부 2.6)
3. 에이전트 프롬프트 생성
  - agents/director.md (제3부 3.1)
  - agents/phase1/\*.md 4개 (제3부 3.2~3.5)
  - agents/phase2/\*.md 7개 (제3부 3.6~3.12)
4. Python 스크립트 생성
  - scripts/gemini\_client.py (제4부 4.1)
  - scripts/generate\_images.py (제4부 4.2)
  - scripts/produce.py (제4부 4.3)
  - scripts/compositor.py (제4부 4.4)
  - scripts/capcut-project-gen.py (제4부 4.5)
5. 참고 문서 생성
  - guides/CAPCUT\_GUIDE.md (제5부 5.1)
  - docs/video-prompt-guide.md (제5부 5.2)
6. 의존성 설치
 

```
pip install -e ".[dev]"
```
7. Git 초기화
 

```
git init
git add .
git commit -m "feat: auto-video 프로젝트 초기 세팅"
```
8. 확인
 

```
python3 -c "import google.generativeai; print('OK')"
```

```
which ffmpeg
```

```
ls agents/ scripts/ brand/ config/
```

모든 파일이 생성되면 AUTO-VIDEO-GUIDE.md의 "제12장. 빠른 시작 가이드"를 따라 콘텐츠 제작을 시작한다.

# 02

## PART TWO

# 사용법 가이드

AI 영상 콘텐츠를 기획부터 완성까지. 처음 사용하는 사람도 따라할 수 있다.



# Auto-Video: AI 영상 콘텐츠 자동 제작 완전 가이드

---

이 문서는 Claude Code를 활용하여 AI 영상 콘텐츠를 기획부터 완성까지 제작하는 전체 과정을 설명한다. 처음 사용하는 사람도 이 문서만 읽고 따라할 수 있도록 모든 단계를 상세히 기술한다.

# 제1장. 프로젝트 개요

---

## 1.1 이 프로젝트가 하는 일

---

auto-video는 AI 도구들을 조합하여 유튜브 영상을 자동으로 만드는 파이프라인이다. 사람이 직접 촬영하지 않고, AI가 이미지를 그리고, AI가 영상을 만들고, AI가 나레이션을 읽어준다. 사람은 주제를 정하고, 중간중간 결과물을 확인하며 승인하는 역할을 한다.

최종 결과물은 CapCut 편집 프로젝트로 나오며, CapCut에서 미세 조정 후 유튜브에 업로드한다.

## 1.2 만들 수 있는 영상 종류

---

이 파이프라인은 한국어/영어 모두 지원하며, 사용자가 원하는 주제와 포맷에 따라 범용적으로 사용할 수 있다. 채널 언어, 영상 길이, 비율은 모두 사용자가 자유롭게 설정한다.

숏폼 (59초 이내):

- 세로 영상 (9:16)
- 한국어 또는 영어
- 3~5개 씬으로 구성
- 예시: AI 인사이트, 뉴스 요약, 트렌드 해설

롱폼 (3~5분):

- 가로 영상 (16:9)
- 한국어 또는 영어
- 15~30개 씬으로 구성
- 예시: 인물 다큐멘터리, 사건 분석, 기술 해설

영어 콘텐츠 제작 시에는 미국 구어체를 기본으로 하며, 한국어 콘텐츠 제작 시에는 채널 캐릭터에 맞는 말투를 적용한다. 어떤 언어든 대본은 해당 언어 원본 + 한글 검수용 두 파일을 생성한다.

## 1.3 사용하는 AI 도구들

---

이미지 생성:

- Gemini API (모델: Image Flash 3.1 / gemini-3.1-flash-image-preview)
- 씬별 정지 이미지를 만든다

영상 생성:

- Veo 3 / Veo 3.1 — Google의 영상 생성 AI (Gemini API), 8초 클립
- Grok API — xAI의 영상 생성, Image-to-Video 지원

나레이션 (TTS):

- 무료 TTS 서비스 — Edge TTS, Google TTS, Coqui TTS 등
- 다양한 보이스 프로필 지원 (한국어/영어)
- 보이스 클론 기능으로 특정 목소리를 복제하여 사용

편집:

- CapCut — 데스크톱 앱
- capcut-project-gen.py 스크립트로 프로젝트를 자동 생성

## 1.4 전체 아키텍처

---

이 프로젝트는 2개의 Phase로 나뉜다.

Phase 1 — 브랜드 세팅 (채널당 1회): 채널의 캐릭터, 서사 구조, 주제 전략, 벤치마크를 정의한다. 4개의 YAML 파일 (character.yaml, narrative.yaml, topics.yaml, benchmark.yaml)이 생성된다. 한 번 만들면 같은 채널의 모든 에피소드에서 재사용한다.

Phase 2 — 콘텐츠 제작 (에피소드당 1회): 리서치 → 이미지 → 영상 → 대본 → 나레이션 → 자막 → CapCut 프로젝트 순서로 진행한다. 각 단계마다 사용자가 결과를 확인하고 승인한다.

## 제2장. 환경 설정

### 2.1 설치 — Claude Code에게 맡기기

이 파이프라인에 필요한 도구들이 있다. 직접 터미널을 열어서 하나씩 설치할 필요 없이, Claude Code에게 아래처럼 요청하면 된다:

환경 설정해줘. 필요한 도구 전부 설치하고, 프로젝트 의존성도 세팅해줘.

Claude Code가 알아서 아래 도구들을 확인하고, 없으면 설치한다.

#### 설치되는 도구와 이유

**Python 3.10** (pyenv로 설치): 모든 스크립트가 Python으로 작성되어 있다. 이미지 생성, 영상 생성, CapCut 프로젝트 자동 생성 등 파이프라인의 모든 자동화가 Python 위에서 돌아간다.

**ffmpeg / ffprobe**: 영상/오디오 파일의 길이와 해상도를 자동으로 읽는 도구이다. CapCut 프로젝트를 만들 때 각 클립의 정확한 길이를 알아야 타임라인에 배치할 수 있다. 이 도구가 없으면 CapCut 프로젝트 생성이 실패한다.

**CapCut** (사용자 직접 설치): 최종 영상을 편집하는 앱이다. Mac App Store에서 직접 설치해야 한다 (Claude Code가 대신 설치할 수 없음). CapCut은 macOS 샌드박스 앱이라 자기 폴더 안의 파일만 읽을 수 있다. 이 제약은 capcut-project-gen.py가 자동으로 처리한다.

**무료 TTS 서비스** (Edge TTS, Google TTS, Coqui TTS 등): AI 나레이션을 만드는 도구이다. 대본 텍스트를 넣으면 사람 목소리처럼 읽어준다. 한국어/영어 모두 지원하며, 무료로 사용 가능하다. Edge TTS는 Microsoft의 고품질 음성을 무료로 제공하고, Coqui TTS는 보이스 클론 기능을 지원한다. Claude Code가 pip으로 설치한다.

**Node.js + Claude Code**: 이 파이프라인의 두뇌이다. Claude Code가 에이전트를 실행하고, 스크립트를 호출하고, 결과를 확인하는 모든 과정을 오케스트레이션한다. Claude Code가 프로젝트 폴더에서 실행되면 CLAUDE.md를 자동으로 읽어서 규칙을 이해한다.

요약: Python, ffmpeg, Node.js는 Claude Code가 설치한다. CapCut은 사용자가 직접 앱을 설치한다. TTS는 Claude Code가 pip으로 설치한다.

### 2.2 API 키 설정

이미지와 영상을 AI로 생성하려면 API 키가 필요하다. Claude Code에게 이렇게 말하면 된다:

API 키 설정해줘. Gemini 키는 \_\_\_\_이고, Grok 키는 \_\_\_\_야.

Claude Code가 config/api-keys.env 파일을 만들어서 저장한다.

키를 발급받는 곳:

**Gemini API** (이미지 + 영상 생성용): Google AI Studio ([aistudio.google.com](https://aistudio.google.com)) → "Get API key" → 복사

**Grok API** (영상 생성 대안용): xAI 콘솔 ([console.x.ai](https://console.x.ai)) → API 키 생성 → 복사

API 키는 .gitignore에 포함되어 git에 올라가지 않는다. 절대로 키를 다른 사람에게 공유하거나 커밋하지 않는다.

## 2.3 프로젝트 의존성

---

Claude Code가 환경을 설정할 때 Python 패키지도 함께 설치한다. pyproject.toml에 정의된 google-generativeai, moviepy, pyyaml 등이 자동으로 깔린다.

## 2.4 디렉토리 구조 이해하기

---

```

auto-contents/
├── agents/ # AI 에이전트 프롬프트 파일들
│ ├── director.md # 팀장 에이전트 (Phase 판단 + 오케스트레이션)
│ ├── phase1/ # Phase 1 에이전트들 (브랜드 세팅)
│ │ ├── brand-architect.md
│ │ ├── narrative-designer.md
│ │ ├── topic-strategist.md
│ │ └── benchmark-analyst.md
│ └── phase2/ # Phase 2 에이전트들 (콘텐츠 제작)
│ ├── researcher.md
│ ├── scriptwriter.md
│ ├── scene-director.md
│ ├── image-generator.md
│ ├── video-generator.md
│ ├── voice-producer.md
│ └── compositor.md
├── brand/ # 브랜드 가이드 (Phase 1 산출물)
│ ├── character.yaml # 캐릭터 페르소나, 말투
│ ├── narrative.yaml # 서사 구조, 에피소드 공식
│ ├── topics.yaml # 콘텐츠 주제 전략
│ └── benchmark.yaml # 벤치마크 분석
├── config/ # 설정 파일
│ ├── api-keys.env # API 키 (git 무시)
│ └── models.yaml # AI 모델 설정
├── docs/ # 참고 문서
│ └── video-prompt-guide.md # 영상 생성 프롬프트 작성법
├── guides/ # 가이드 문서
│ └── CAPCUT_GUIDE.md # CapCut 프로젝트 생성 상세 가이드
├── scripts/ # Python 실행 스크립트
│ ├── generate_images.py # Gemini Image Flash 3.1 이미지 생성
│ ├── produce.py # Veo 3 영상 + TTS 나레이션
│ └── compositor.py # MoviePy 합성
├── projects/ # 콘텐츠 프로젝트들 (Phase 2 산출물)
│ ├── duck-ai-insight/ # 예시: 한국어 숏폼 채널
│ │ └── ep01-chatgpt-lies/
│ ├── niche-bending/ # 예시: 영어 롱폼 채널
│ │ ├── ep01-maradona/
│ │ └── ep02-mike-tyson/ # 참고용 완성 에피소드
│ └── {your-channel}/ # 새 채널은 여기에 추가
├── .claude/
│ └── commands/
│ └── create-c.md # /create-c 커맨드 정의
├── CLAUDE.md # Claude Code 프로젝트 규칙
├── SESSION_HANDOVER.md # 세션 이어받기 문서
└── AUTO-VIDEO-GUIDE.md # 이 문서

```

## 제3장. Phase 1 — 브랜드 세팅 (채널 초기 설정)

### 3.1 Phase 10이 필요한 시점

새로운 채널을 처음 만들 때 1회만 실행한다. 이미 brand/ 디렉토리에 4개 YAML 파일이 채워져 있으면 건너뛰다.

brand/ 디렉토리의 파일들을 확인하여 빈 필드만 있으면 Phase 10이 필요하다. 일부만 채워져 있으면 빈 파일부터 이어서 진행한다.

### 3.2 Phase 1 실행 순서

4개의 전문 에이전트가 순서대로 실행된다. 각 에이전트가 끝나면 결과를 확인하고 승인해야 다음 단계로 넘어간다.

#### 단계 1: Brand Architect (캐릭터 설계)

이 에이전트는 채널의 캐릭터 페르소나를 만든다.

정의하는 것들:

- 캐릭터 이름
- 성격 키워드 (3~5개)
- 말투 특성 (축약형 사용 여부, 유머 수위 등)
- 시그니처 표현 (매 에피소드에 반복되는 대사)
- 아트 스타일 (2D/3D/실사/커스텀)
- 컬러 팔레트
- 항상 지켜야 할 것 / 절대 하면 안 되는 것

출력 파일: brand/character.yaml

실제 예시 (한국어 솜뽕 채널 — "애덕이"):

```
persona:
 name: "애덕이"
 personality: ["호기심", "솔직함", "기술열정"]
 speaking_style: "친구한테 설명하듯 편하게"
 catchphrase: ["이거 진짜 대박인 게요", "근데 여기서 중요한 건"]

visual:
 art_style: "따뜻한 한국 일러스트북/웹툰풍"
 color_palette: ["#FFE4B5", "#4ECDC4", "#FF6B6B"]
 key_elements: ["안경", "후드짚업", "AirPods Max"]

consistency_rules:
 always: ["반말 사용", "비유로 설명"]
 never: ["전문 용어 그대로 사용", "영어 섞어 쓰기"]
```



## 단계 2: Narrative Designer (서사 구조 설계)

채널의 스토리텔링 공식을 만든다.

정의하는 것들:

- 세계관 / 톤
- 오프닝 혹은 공식 (첫 3초에 시청자를 잡는 방법)
- 서사 구조 (기승전결, 문제-해결 등)
- 엔딩 공식 (CTA 포함)
- 매 에피소드 반복 요소
- 숏폼/롱폼 에피소드 공식 (씬 수, 페이싱, 시간 제한)

출력 파일: brand/narrative.yaml

시간 제한 규칙 (hard limit):

- 숏폼: 59초 이내 — 이 시간을 1초라도 넘기면 안 된다
- 롱폼: 3분~5분 — 5분을 넘기면 안 된다

이 제한은 narrative.yaml의 `duration_hard_limit: true` 로 강제된다.

## 단계 3: Topic Strategist (주제 전략)

채널에서 다룰 주제의 범위와 우선순위를 정한다.

출력 파일: brand/topics.yaml

## 단계 4: Benchmark Analyst (벤치마크 분석)

성공한 유사 채널들을 분석하여 체크리스트를 만든다.

출력 파일: brand/benchmark.yaml

## 3.3 Phase 1 실행 방법

Claude Code에서 다음과 같이 요청한다:

브랜드 세팅 해줘

또는 더 구체적으로:

새 채널 만들려고 해. 영어 롱폼 시네마틱 다큐멘터리 채널이야.  
AI가 생성한 영상으로 스포츠 인물 다큐를 만들 거야.  
브랜드 세팅부터 시작하자.

Claude Code가 4개 에이전트를 순서대로 실행하면서 각 단계마다 결과를 보여주고 승인을 요청한다.

승인: "좋아" 또는 "진행해" → 다음 단계로 수정: "이 부분 수정해줘: ..." → 같은 단계 재실행 처음부터: "다시 해줘" → 같은 단계를 빈 상태에서 재실행

## 제4장. Phase 2 — 콘텐츠 제작 (Video-First 플로우)

### 4.1 Video-First 플로우란?

이 프로젝트의 핵심 철학이다.

기존 방식 (Script-First): 대본을 먼저 쓰고, 거기에 맞춰 영상을 만든다. 문제: 나레이션 길이와 영상 길이가 안 맞는다. 영상은 120초인데 나레이션이 248초라든지.

우리의 방식 (Video-First): 영상 러닝타임을 먼저 정하고, 거기서 역산하여 씬 수와 대본 길이를 결정한다. 영상과 나레이션이 처음부터 맞도록 설계된다.

Video-First 계산법:

1. 러닝타임 결정      예: 4분 10초 = 250초
2. 영상 도구 선택      예: Veo 3 (8초/클립)
3. 씬 수 = 러닝타임 / 클립 길이       $250 / 8 = \sim 31$ 씬
4. 총 단어 수 = 러닝타임(초) / 60 \* 150       $250 / 60 * 150 = 625$ 단어
5. 씬당 단어 수 = 총 단어 / 씬 수       $625 / 31 = \sim 20$ 단어

중요한 숫자:

- 영어 나레이션은 분당 약 150단어(WPM), 한국어는 약 300음절/분을 기준으로 한다.
- Veo 3 / Veo 3.1은 클립당 8초, Grok은 클립당 길이가 가변적이다.
- 영상 도구에 따라 클립 길이가 달라지므로 씬 수도 달라진다.

### 4.2 /create-c 커맨드로 시작하기

Claude Code에서 `/create-c` 를 입력하면 콘텐츠 제작을 시작하는 대화형 플로우가 시작된다.

5개의 질문을 순서대로 한다:

#### Q1. 채널/트랙 선택

어떤 채널로 제작할까요?

- 기존 채널 (brand/ 설정이 있는 채널 목록 표시)
- 새 채널 (Phase 1 브랜드 세팅부터 시작)

기존 채널을 선택하면 해당 채널의 brand/ 설정을 자동으로 읽는다. 채널은 언어(한국어/영어)와 포맷(숏폼/롱폼)을 자유롭게 조합할 수 있다.

## Q2. 주제 + 레퍼런스

이번 에피소드 주제가 뭔가요?  
레퍼런스 채널이 있으면 링크도 함께 주세요.

주제를 자유롭게 입력한다. 예: "마이크 타이슨", "테슬라 주가 폭락", "ChatGPT가 거짓말하는 이유"

레퍼런스 채널 링크를 함께 전달하면 에이전트가 자동으로 벤치마킹을 진행한다. 예: "https://youtube.com/@WildCinematics 이 채널 스타일로 만들어줘"

레퍼런스를 전달하면 에이전트가 다음을 수행한다:

1. 해당 채널과 유사한 메가 채널(구독자 100만+)을 추가로 탐색
2. 발견된 채널들의 공통 패턴을 분석 (오프닝 구조, 편집 스타일, 나레이션 톤)
3. 분석 결과를 체크리스트로 정리
4. 이 체크리스트를 기준으로 이후 모든 제작 단계에 적용

레퍼런스가 없으면 기존 brand/benchmark.yaml의 체크리스트를 사용한다.

## Q3. 영상 러닝타임

영상 길이를 얼마로 할까요?  
- 숏폼: 59초 이내  
- 롱폼: 3분 / 3분 30초 / 4분 / 5분  
- 직접 입력

## Q4. 영상 도구

영상 생성 도구는?  
- Veo 3 (8초/클립, Gemini API)  
- Veo 3.1 (8초/클립, Gemini API 프리뷰)  
- Grok (가변 길이, xAI API)  
- 혼합 (Veo + Grok)

## Q5. 보이스

나레이션 보이스는?  
- 채널 기본 보이스 (설정된 경우 자동 선택)  
- 새 보이스 생성 (새 보이스 생성 (TTS 서비스에서 선택))

채널별로 고정 보이스가 설정되어 있으면 자동 선택된다. 새 채널이거나 다른 보이스를 원하면 사용할 TTS 서비스를 선택한다. Edge TTS는 설치 없이 바로 사용 가능하다.

## 자동 계산 결과 확인

5개 질문이 끝나면 자동으로 계산하고 결과를 보여준다:

## ## 제작 설정 확인

- 채널: 니치 밴딩
- 주제: 마이크 타이슨
- 레퍼런스: @WildCinematics (벤치마킹 완료)
- 러닝타임: 4분 10초 (250초)
- 영상 도구: Veo 3 (8초/클립)
- 씬 수: 31씬
- 대본: ~625단어 (씬당 ~20단어)
- 보이스: Narrator (무료 TTS 서비스)

이대로 진행할까요?

"좋아" 또는 "진행해"라고 하면 제작이 시작된다.

## 4.3 제작 단계 상세 — 1단계: 리서치

주제에 대해 팩트를 조사한다.

Claude Code가 웹 검색과 문서 조사를 통해 다음을 수집한다:

- 핵심 사건/날짜/수치
- 인물 정보 (생년월일, 주요 경력)
- "와" 포인트 (시청자를 놀라게 할 팩트)
- 경쟁 콘텐츠와 차별화 포인트

산출물: projects/{slug}/research.md

이 단계가 끝나면 리서치 결과를 보여주고 승인을 요청한다. 팩트가 틀렸거나 빠진 내용이 있으면 수정을 요청할 수 있다.

중요: 모든 대본은 팩트 기반이어야 한다. 날짜, 수치, 사건을 정확하게 검증하고, 확실하지 않은 내용은 포함하지 않는다.

## 4.4 제작 단계 상세 — 2단계: 캐릭터 앵커 이미지

실존 인물을 다루는 콘텐츠에서 가장 중요한 단계이다.

캐릭터 앵커란: 씬별 이미지를 생성할 때 인물의 외모가 일관되도록 하는 기준 이미지이다. 앵커 없이 이미지를 만들면 씬마다 다른 사람이 나온다.

실존 인물의 경우 시대별로 앵커를 나눈다. 예를 들어 마이크 타이슨이라면:

- 소년기 (10\~18세): 마르고 작은 체구
- 전성기 (19\~25세): 근육질, 짧은 머리, 검은 트렁크
- 후기 (30\~40대): 얼굴 문신, 살이 붙은 체형
- 현재 (50대\~): 회색 수염, 여전히 넓은 어깨

앵커 이미지 생성 시 절대 규칙:

모든 프롬프트에 인종/피부색/체형을 명시적으로 기재해야 한다.

나쁜 예:

A young boxer in a gym, muscular build, short hair

좋은 예:

A young African American male boxer, dark brown skin, stocky muscular build, short cropped black hair, broad shoulders, thick neck, in a dimly lit boxing gym

인종/피부색을 빼면 시가 씬마다 다른 인종의 사람을 생성한다. EP02 마이크 타이슨 제작 시 초기 25장 중 5장이 백인으로 생성되어 전부 재생성한 경험이 있다.

산출물: projects/{slug}/anchors/ 디렉토리에 시대별 이미지 저장

## 4.5 제작 단계 상세 — 3단계: 씬별 이미지 생성

앵커 이미지를 기준으로 각 씬의 정지 이미지를 만든다.

이미지 생성 방법:

방법 1 — Python 스크립트 사용:

```
python3 scripts/generate_images.py --project projects/{slug}
```

이 스크립트는 image-prompts.md 파일에서 프롬프트를 읽어 Gemini API로 이미지를 생성한다.

방법 2 — Claude Code에서 직접 생성: Claude Code에게 "씬 이미지 생성해줘"라고 요청하면 Gemini API를 호출하여 이미지를 하나씩 만들고 저장한다.

이미지 프롬프트 작성 규칙:

- 캐릭터 앵커의 외모 설명을 매 프롬프트에 포함
- 인종/피부색/체형을 반드시 명시
- 씬의 시대에 맞는 앵커를 참조
- "cinematic, dramatic lighting, 16:9 aspect ratio" 같은 품질 키워드 추가

산출물:

- projects/{slug}/image-prompts.md — 씬별 프롬프트 기록
- projects/{slug}/scenes/images/scene-01.png ~ scene-{N}.png

파일명 규칙:

- scene-{NN} 형식으로 통일 (scene-01, scene-02, ...)
- 씬당 별도 폴더를 만들지 않는다
- 모든 이미지는 scenes/images/ 디렉토리에 모아 놓는다

이미지 생성이 끝나면 Claude Code에게 "이미지 확인해줘"라고 요청한다. Claude Code가 이미지를 하나씩 열어보고 문제가 있는 것을 보고한다.

흔한 문제:

- 인종이 바뀐 경우 → 프롬프트에 인종 추가 후 재생성

- 배경이 서사와 맞지 않는 경우 → 프롬프트 수정
- 인물의 나이가 맞지 않는 경우 → "age 35" 같이 나이 명시

## 4.6 제작 단계 상세 — 4단계: 씬별 영상 생성

정지 이미지를 움직이는 영상 클립으로 변환한다. 이것을 Image-to-Video (I2V) 라고 부른다.

영상 도구는 Q4에서 선택한 것을 사용한다. 주력 도구는 Veo 3 (Gemini API)와 Grok API이다.

### Image-to-Video 프롬프트 핵심 규칙

어떤 도구를 사용하든 Image-to-Video 프롬프트의 기본 원칙은 동일하다.

규칙 1 — 이미지 내용을 재설명하지 마라. 이미지가 씬을 이미 정의하고 있으므로, 프롬프트는 무엇이 움직이는지만 쓴다.

나쁜 예:

A man in a blue shirt standing in a boxing ring with red ropes and bright lights

좋은 예:

Man slowly raises his gloves, shifts weight forward.  
Crowd in background sways slightly. Slow dolly in.

규칙 2 — 움직임 끝 상태를 명시하라.

Then settles back to original position.

이걸 안 쓰면 시가 움직임을 계속 이어가다 왜곡된 영상을 만든다.

규칙 3 — 동시 동작 2개 이하.

나쁜 예: He punches, kicks, jumps, and spins simultaneously  
좋은 예: He throws a right hook, then steps back. Then settles.

규칙 4 — 물리 키워드를 넣어라.

realistic physics, natural momentum, gravity-accurate

규칙 5 — 과도한 액션을 억제하라.

subtle motion only, gentle movement, slow and deliberate

프롬프트 공식:

[무엇이 움직이나] + [어떻게 움직이나] + [배경 움직임] + [카메라 동작] + [Then settles]

카메라 동작 참고:

- slow dolly in — 감정적 강조, 긴장감
- dolly out — 공간 확장
- pan left/right — 공간 탐색
- static locked-off shot — 안정적, 관찰자 시점
- tilt up — 위압감

## Veo 3 / Veo 3.1 사용법 (Gemini API)

Veo는 Google의 영상 생성 AI이다. Gemini API를 통해 사용한다.

```
python3 scripts/produce.py --project projects/{slug} --step video
```

또는 Claude Code에서:

```
Veo3로 씬 영상 생성해줘
```

Veo3와 Veo 3.1의 차이:

- Veo 3: 안정적, 8초 클립
- Veo 3.1: 화질 개선, 더 자연스러운 움직임 (프리뷰)

Veo 프롬프트는 서술적으로 작성해도 괜찮다. 움직임의 단계를 구체적으로 기술하면 더 좋은 결과를 얻는다.

## Grok API 사용법

Grok은 xAI의 영상 생성 AI이다. API를 통해 사용한다.

Claude Code에서:

```
Grok으로 씬 영상 생성해줘
```

Grok의 특징:

- 물리 반응과 자연스러운 움직임이 우수하다
- 스포츠/액션 씬에서 특히 강하다
- 클립 길이가 가변적이다

## 도구 선택 전략

기본 선택: Veo 3를 기본으로 사용한다. Gemini API로 이미지와 영상을 동일 파이프라인에서 처리할 수 있어 편리하다.

Grok이 더 나은 경우:

- 스포츠/격한 액션 씬 — 물리 반응이 더 자연스럽다
- 인물의 미세한 표정 변화 — 디테일이 더 좋을 수 있다
- Veo 결과가 불만족스러운 씬 — 대안으로 Grok에서 재생성

혼합 사용: 한 에피소드 안에서 Veo와 Grok을 혼합해서 사용해도 된다. 씬별로 더 나은 결과물을 선택한다.

## 영상 프롬프트 기록

모든 씬의 영상 프롬프트를 기록해 둔다.

산출물: projects/{slug}/video-prompts.md

```
[scene-01] Arena silhouette
Man slowly raises his gloves, shifts weight forward.
Dramatic spotlight sweeps across the ring. Slow dolly in.
Then settles.

[scene-02] Gloves on canvas
Gloves gently drop to the canvas. Slight bounce.
Camera slowly tilts down. Static. Then settles.
```

타이틀에 [scene-NN] 폴더명을 기재하면 나중에 어떤 씬인지 찾기 쉽다.

산출물:

- projects/{slug}/video-prompts.md — 씬별 프롬프트
- projects/{slug}/scenes/videos/scene-01.mp4 ~ scene-{N}.mp4

## 4.7 제작 단계 상세 — 5단계: 대본 작성

이미지와 영상이 준비된 후에 대본을 쓴다. 이것이 Video-First 플로우의 핵심이다.

씬당 단어 수가 이미 계산되어 있으므로 (예: 25단어/씬), 각 씬의 나레이션이 해당 영상 클립 길이에 맞게 작성된다.

대본 작성 규칙:

공통 규칙 (모든 언어):

- 채널 캐릭터의 말투를 일관되게 사용한다 (character.yaml 참조)
- HOOK(오프닝)과 본편이 동일 문장을 반복하지 않는다
- HOOK: 감정적 티저만. 구체적 디테일은 아낀다.
- 본편: 전체 맥락과 디테일을 풀어서 서술한다.
- 나레이션과 자막이 완전히 같은 내용을 반복하지 않는다
- 나레이션(보이스): 감정, 서사, 스토리를 전달
- 자막(화면 텍스트): 팩트, 수치, 이름, 날짜를 보강
- 나레이션은 씬당 가변 길이 (8~12초) — 기계적으로 끊기지 않게

한국어 콘텐츠 추가 규칙:

- 전문 용어는 비유로 풀어서 설명한다
- 타겟 시청자 연령대에 맞는 어투 사용 (반말/존댓말 등)

영어 콘텐츠 추가 규칙:

- 미국 구어체 필수 — BBC 다큐멘터리식 포멀한 톤 금지
- 축약형 적극 사용: didn't, wasn't, couldn't, here's, that's
- 자연스러운 전환어: "Look,", "Here's the thing,", "And get this —"



- 영국식 표현 교체: footballer→player, pitch→field, nil→nothing

대본 산출물 (이중 생성):

어떤 언어로 제작하든 대본은 항상 두 파일을 생성한다: - projects/{slug}/script.md — 원본 대본 (나레이션/TTS 소스, 제작 언어로 작성) - projects/{slug}/script-ko.md — 한글 검수용 (영어 콘텐츠 시 한글 번역) 또는 script-en.md (한국어 콘텐츠 시 영문 번역, 필요한 경우)

사용자가 내용을 빠르게 검수할 수 있도록 검수용 파일을 반드시 함께 만든다.

대본 형식 예시 (script.md):

```
EP02: Mike Tyson – The Baddest Man Who Destroyed Himself

- Format: AI Cinematic Documentary (Video-First)
- Length: 4min 10sec (250s)
- Scenes: 25 x 10sec clips
- Words: ~625 (150 WPM)
- Language: English (American conversational)

Scene 01 – Arena silhouette
There's this moment – where one guy has it all.
The title, the fear, the whole damn world watching him.

Scene 02 – Gloves on canvas
And then he just... lets it go. Not 'cause somebody took it.
'Cause he was already broken before the first bell ever rang.

...
```

## 4.8 제작 단계 상세 — 6단계: TTS 나레이션 생성

대본의 각 씬을 무료 TTS 서비스 (Edge TTS, Google TTS, Coqui TTS 등)로 음성 파일로 만든다.

### TTS 서비스 사전 준비

1. TTS 서비스를 준비한다
2. 사용할 보이스 프로필을 확인한다
3. 채널별 고정 보이스가 설정되어 있으면 해당 프로필 사용
4. 새 보이스가 필요하면 새 보이스 생성 (TTS 서비스에서 선택)
5. 언어 설정 확인: 한국어 콘텐츠면 한국어, 영어 콘텐츠면 영어

### 나레이션 생성 과정

Claude Code에게 요청:

나레이션 생성해줘

Claude Code가 각 씬의 나레이션 텍스트를 TTS 서비스로 보내서 음성 파일을 생성한다.

TTS 주의사항:

- 씬 단위로 개별 생성한다 (ACT 전체를 한 번에 생성하지 않는다)
- 나레이션은 씬당 가변 길이 (8~12초) — 기계적으로 끊기지 않게
- 생성 완료 후 각 WAV 파일의 duration이 정상인지 확인한다
- duration이 0인 파일이 있으면 해당 씬만 재생성한다

산출물: projects/{slug}/narration/scene-01.wav ~ scene-{N}.wav

## 4.9 제작 단계 상세 — 7단계: SRT 자막 생성

나레이션에 맞춰 자막 파일을 만든다.

### SRT 파일 형식

SRT는 매우 단순한 텍스트 파일이다:

```
1
00:00:00,500 --> 00:00:05,500
One guy has it all. The title, the fear, the whole damn world watching.

2
00:00:06,000 --> 00:00:10,000
And then he just lets it go. He was already broken before the first bell ever rang.
```

### SRT 작성 규칙 (절대 지켜야 할 것)

규칙 1 — 플레인 텍스트만: 메타데이터, 번호 태그, 좌표, 스타일 태그를 포함하지 않는다. SRT 표준 형식 그대로만 작성한다.

규칙 2 — 1줄 제한: 자막 1개당 반드시 1줄이다. 2줄 이상 금지. 긴 문장은 쪼개서 별도 엔트리로 만든다.

나쁜 예:

```
1
00:00:00,500 --> 00:00:05,500
One guy has it all.
The title, the fear, the whole damn world watching.
```

좋은 예:

```
1
00:00:00,500 --> 00:00:02,500
One guy has it all.

2
00:00:02,500 --> 00:00:05,500
The title, the fear, the whole damn world watching.
```

규칙 3 — CapCut 자막 스타일: capcut-project-gen.py에 전달하는 subtitle\_style:

```
{
 "font_size": 6.0,
 "color": [1, 1, 1],
 "position_y": -0.764,
 "scale": 0.7
}
```

- font\_size: 6.0
- color: 흰색 [1, 1, 1]
- position\_y: -0.764 (하단)
- scale: 0.7
- font: CapCut 기본 en.ttf

산출물: projects/{slug}/capcut-export/subtitles.srt

## 4.10 제작 단계 상세 — 8단계: CapCut 프로젝트 생성

모든 에셋(영상, 나레이션, 자막)을 CapCut 프로젝트로 합친다.

### capcut-project-gen.py 사용

이 스크립트는 별도 위치에 있다:

```
/Users/yoogeon/Agents/contents agent/scripts/capcut-project-gen.py
```

절대로 draft\_info.json을 수동으로 만들지 않는다. 과거에 여러 번 시도했지만 매번 실패했다. 구조가 너무 복잡하고, extra\_material\_refs 개수가 안 맞으면 CapCut이 크래시한다.

### config.json 작성

먼저 CapCut 설정 파일을 만든다:

```
{
 "name": "EP02-MikeTyson-v1",
 "fps": 30,
 "canvas": { "width": 1920, "height": 1080 },
 "video": [
 "/abs/path/to/scenes/videos/scene-01.mp4",
 "/abs/path/to/scenes/videos/scene-02.mp4",
 "/abs/path/to/scenes/videos/scene-03.mp4"
],
 "audio": [
 { "path": "/abs/path/to/narration/scene-01.wav", "start": 0 },
 { "path": "/abs/path/to/narration/scene-02.wav", "start": 10.5 },
 { "path": "/abs/path/to/narration/scene-03.wav", "start": 21.0 }
],
 "subtitle": "/abs/path/to/capcut-export/subtitles.srt",
 "subtitle_style": {
 "font_size": 6.0,
 "color": [1, 1, 1],
 "position_y": -0.764,
 "scale": 0.7
 }
}
```

각 필드 설명:

- name: CapCut에서 표시될 프로젝트 이름
- fps: 초당 프레임 수 (보통 30)
- canvas: 해상도 (1920x1080 = Full HD 가로)
- video: 영상 클립들의 절대 경로 배열. 순서대로 타임라인에 배치
- audio: 나레이션 파일들. start 값으로 타임라인 위치 지정
- subtitle: SRT 파일 경로
- subtitle\_style: 자막 스타일 오버라이드

audio 배치 방법:

- 문자열만 넣으면 이전 오디오 끝난 직후에 자동 배치
- {"path": "...", "start": 초} 형태면 지정한 시간에 배치
- 나레이션 호흡 갭 (1~3초)을 넣으려면 start 값을 수동으로 지정

## 스크립트 실행

```
CapCut을 먼저 종료한다
(실행 중이면 프로젝트 목록에 안 나올 수 있다)

python3 "/Users/yoogeon/Agents/contents_agent/scripts/capcut-project-gen.py" \
 projects/{slug}/capcut-config.json
```

스크립트가 하는 일:

1. ~/Movies/CapCut/User Data/Projects/com.lveditor.draft/{name}/ 폴더 생성

2. 모든 영상, 오디오 파일을 Resources/ 폴더에 복사
3. draft\_info.json 생성 (타임라인 전체 데이터)
4. draft\_meta\_info.json 생성 (프로젝트 메타)

실행 후 CapCut을 열면 프로젝트 목록에 새 프로젝트가 나타난다.

## CapCut에서 수동 작업

자동 생성 후 CapCut에서 수동으로 해야 하는 것들:

1. SRT 자막 импорт: CapCut에서 "자막 > 로컬 자막 가져오기"로 SRT 파일을 импорт
2. 영상/오디오 미세 싱크 조정: 0.1~0.3초 단위로 미세하게 맞춘다
3. 나레이션 호흡 갭: 씬 사이에 1~3초의 자연스러운 쉬는 구간을 넣는다 기계적으로 10초마다 딱딱 끊기면 부자연스럽다
4. 전환 효과: ACT 경계(큰 서사 전환점)에 전환 효과를 넣는다
5. BGM 추가: CapCut 내장 음악에서 선택
6. 엔딩 콜백 (선택): 첫 번째 씬의 영상을 엔딩에 다시 넣어서 순환 구조를 만든다

## 제5장. 프로젝트 폴더 구조 상세

### 5.1 에피소드 프로젝트 폴더

하나의 에피소드가 완성되면 다음과 같은 구조가 된다:

```

projects/niche-bending/ep02-mike-tyson/
├── scenes/
│ ├── images/ # 씬별 정지 이미지
│ │ ├── scene-01.png
│ │ ├── scene-02.png
│ │ └── ... scene-25.png
│ └── videos/ # 씬별 영상 클립
│ ├── scene-01.mp4
│ ├── scene-02.mp4
│ └── ... scene-25.mp4
├── anchors/ # 캐릭터 앵커 이미지 (시대별)
│ ├── tyson-youth.png
│ ├── tyson-prime.png
│ └── tyson-later.png
├── narration/ # 씬별 나레이션 음성
│ ├── scene-01.wav
│ ├── scene-02.wav
│ └── ... scene-25.wav
├── capcut-export/
│ └── subtitles.srt # SRT 자막 파일
├── script.md # 원본 대본 (제작 언어)
├── script-ko.md # 한글 검수용 (또는 script-en.md)
├── image-prompts.md # 이미지 생성 프롬프트 기록
├── video-prompts.md # 영상 생성 프롬프트 기록
├── capcut-config.json # CapCut 설정 (v1)
└── capcut-config-v2.json # CapCut 설정 (수정 버전)

```

파일명 규칙:

- scene-{NN} 형식으로 통일 (01, 02, ... 25)
- 씬당 별도 폴더를 만들지 않는다
- images/ 와 videos/ 를 scenes/ 하위에 분리한다

## 5.2 왜 이 구조인가

---

초기에는 썬마다 폴더를 만들었다:

```
scenes/scene-01/image.png
scenes/scene-01/video.mp4
scenes/scene-02/image.png
...
```

이 방식은 파일이 흩어져서 관리가 어려웠다. 현재 구조는 같은 종류의 파일을 한 곳에 모아서 한눈에 볼 수 있다.

## 제6장. 실전 예시 — EP02 마이크 타이슨 제작 과정

이 장은 영어 롱폼 다큐멘터리 에피소드의 실제 제작 사례이다. 한국어 솜품이나 다른 포맷에서도 제작 흐름은 동일하며, 언어와 길이만 달라진다.

### 6.1 제작 설정

- 채널: 영어 롱폼 (니치 밴딩)
- 주제: 마이크 타이슨
- 러닝타임: 4분 10초 (250초)
- 영상 도구: Veo 3 + Grok (혼합)
- 씬 수: 25
- 대본: ~621단어 (씬당 ~25단어)
- 보이스: Narrator (무료 TTS 서비스)

### 6.2 실제 제작 순서

1. 리서치: 타이슨의 생애를 연대기로 조사
2. 1966년 출생, 브라운즈빌, 커스 다마토, 최연소 챔피언, 감옥, 귀 사건 등
3. 캐릭터 앵커: 3개 시대별 앵커 이미지
4. 소년기, 전성기, 후기
5. 모든 프롬프트에 "African American, dark brown skin, stocky muscular build" 명시
6. 이미지 25장 생성:
7. Gemini gemini-3.1-flash-image-preview 사용
8. 문제 발견: scene-16(패배 주제 오류), scene-20(나이 오류), scene-24(일관성) → 재생성
9. 영상 25클립:
10. Veo 3/3.1 + Grok 혼합 사용
11. 문제 발견: Veo3 카메라 플래시가 불꽃놀이처럼 보임 → 프롬프트에 "NOT sparks, NOT fireworks" 추가
12. 대본: 25씬 원본 + 검수용 이중 생성
13. 영어 콘텐츠이므로 미국 구어체 적용: "Let's be real", "Here's the thing", "You can't make that stuff up"
14. 나레이션: 무료 TTS 서비스로 25개 씬별 WAV 생성 (총 219.7초)
15. SRT 자막: 30엔트리, 플레인 텍스트
16. CapCut 프로젝트: v1 자동 생성 → v2 사용자 수동 편집



## 6.3 사용자 수동 편집에서 발견한 패턴

---

EP02의 CapCut 수동 편집(v2)을 분석하여 다음 패턴을 발견했다:

나레이션 호흡 갭:

- 씬 사이에 1~3초의 쉬는 구간
- 기계적으로 10초마다 끊기는 것이 아니라 자연스러운 호흡

클립 분할:

- 나레이션이 10초보다 긴 씬은 같은 영상을 분할하거나 반복

전환 효과:

- ACT 경계(큰 서사 전환점)에 5개의 전환 효과 배치

엔딩 콜백:

- scene-01의 아레나 영상을 엔딩에 다시 사용하여 순환 구조

전체 길이 변화:

- 자동 생성: 252초
- 수동 편집 후: 268.6초 (호흡 갭과 전환 효과 추가로 약 16초 증가)

# 제7장. 트러블슈팅

## 7.1 이미지 관련 문제

문제: 씬마다 인물의 인종이 다르게 나온다 원인: 프롬프트에 인종/피부색을 명시하지 않았다 해결: 모든 프롬프트에 "African American, dark brown skin" 등을 명시한다

문제: 인물의 나이가 서사와 맞지 않는다 원인: 프롬프트에 나이를 명시하지 않았다 해결: "age 19", "age 35, face tattoo" 등 구체적으로 명시한다

문제: gemini-2.0-flash-exp 모델을 찾을 수 없다 원인: 모델명이 변경되었다 해결: gemini-3.1-flash-image-preview를 사용한다

## 7.2 영상 관련 문제

문제: 영상에서 액션이 과도하다 원인: 프롬프트에 억제 키워드가 없다 해결: "subtle motion only", "gentle movement", "then settles" 추가

문제: Veo3에서 묘비에 잘못된 이름이 새겨진다 원인: AI가 이미지의 텍스트를 변형한다 해결: 텍스트가 없는 이미지로 재생성 후 영상 생성

문제: 영상 길이와 나레이션 길이가 안 맞는다 원인: Script-First 방식으로 만들었다 해결: Video-First 플로우를 따른다 (러닝타임 먼저 → 씬 수 역산)

## 7.3 무료 TTS 서비스 관련 문제

문제: 생성된 오디오의 duration이 0이다 원인: 모델이 언로드되었거나 생성이 실패했다 해결: 무료 TTS 서비스에서 해당 프로필을 다시 로드하고 재생성한다

문제: 무료 TTS 서비스에 연결할 수 없다 원인: 무료 TTS 서비스 앱이 실행되지 않고 있다 해결: TTS 서비스를 준비한다

문제: 보이스 톤이 씬마다 달라진다 원인: 동일한 보이스 프로필을 사용하지 않았다 해결: 채널별 고정 프로필을 설정하고 모든 씬에서 동일 프로필 사용

## 7.4 CapCut 관련 문제

문제: CapCut에서 "파일에 액세스할 수 없음" 오류 원인: 소재 파일이 \~/Movies/CapCut/ 외부에 있다 해결: capcut-project-gen.py를 사용하면 자동으로 Resources/에 복사한다

문제: CapCut이 프로젝트를 열다 크래시한다 원인 1: extra\_material\_refs 개수가 안 맞다 (video=6개, audio=5개, text=1개) 원인 2: materials 버킷에 키가 누락되었다 원인 3: draft\_info.json을 수동으로 만들었다 해결: capcut-project-gen.py만 사용한다. 수동 생성 금지.

문제: 프로젝트가 CapCut 목록에 안 나온다 원인: CapCut이 실행 중일 때 프로젝트를 생성했다 해결: CapCut을 종료 → 스크립트 실행 → CapCut 재실행

문제: 오디오 싱크가 안 맞는다 원인: target\_timerange.start가 비디오와 맞지 않는다 해결: audio config에 "start": 초 값을 명시적으로 지정한다

## 7.5 CapCut 프로젝트를 다른 PC로 옮기기

CapCut 프로젝트 폴더를 통째로 복사하면 된다:

```
프로젝트 폴더 경로
~/Movies/CapCut/User Data/Projects/com.lveditor.draft/{프로젝트명}/

zip으로 압축
cd ~/Movies/CapCut/User\ Data/Projects/com.lveditor.draft/
zip -r EP02-MikeTyson.zip EP02-MikeTyson-v2/

다른 PC에서: 같은 경로에 압축 풀기
```

Resources/ 폴더에 모든 소재가 이미 복사되어 있으므로 이 폴더만 옮기면 다른 PC에서도 열 수 있다.

## 제8장. 에이전트 시스템 상세

### 8.1 Director (팀장)

파일: agents/director.md

역할:

- Phase 1과 Phase 2 중 어느 것을 실행할지 판단
- 각 단계에서 적절한 에이전트를 배정
- 체크포인트에서 사용자 승인을 관리
- 에러 발생 시 대안을 제시

Phase 판단 로직:

- brand/ 디렉토리의 4개 YAMLO이 모두 빈 값 → Phase 1 실행
- 4개 YAMLO이 모두 채워짐 → Phase 2 진입
- 일부만 채워짐 → 빈 파일부터 Phase 1 이어서 진행

### 8.2 Phase 1 에이전트들

Brand Architect (brand-architect.md):

- 모델: opus
- 역할: 캐릭터 페르소나, 아트 스타일, 일관성 규칙 설계
- 출력: brand/character.yaml

Narrative Designer (narrative-designer.md):

- 모델: opus
- 역할: 서사 구조, 에피소드 공식, 시간 제한 설정
- 출력: brand/narrative.yaml

Topic Strategist (topic-strategist.md):

- 모델: opus
- 역할: 주제 전략, 콘텐츠 필러 설계
- 출력: brand/topics.yaml

Benchmark Analyst (benchmark-analyst.md):

- 모델: opus
- 역할: 성공 채널 분석, 체크리스트 생성
- 출력: brand/benchmark.yaml

## 8.3 Phase 2 에이전트들

---

Researcher (researcher.md):

- 모델: sonnet
- 역할: 주제에 대한 팩트 조사
- 출력: projects/{slug}/research.md

Scriptwriter (scriptwriter.md):

- 모델: opus
- 역할: 대본 작성 (채널 언어에 맞는 말투, HOOK/본편 중복 방지)
- 출력: script.md + 검수용 대본

Scene Director (scene-director.md):

- 모델: opus
- 역할: 씬별 연출 지시서 작성
- 출력: 씬별 direction.yaml

Image Generator (image-generator.md):

- 모델: sonnet
- 역할: Gemini API로 씬별 이미지 생성
- 출력: scenes/images/scene-{NN}.png

Video Generator (video-generator.md):

- 모델: sonnet
- 역할: Veo 3 / Grok API로 씬별 영상 생성
- 출력: scenes/videos/scene-{NN}.mp4

Voice Producer (voice-producer.md):

- 모델: sonnet
- 역할: 무료 TTS 서비스로 나레이션 생성
- 출력: narration/scene-{NN}.wav

Compositor (compositor.md):

- 모델: sonnet
- 역할: MoviePy로 영상 합성
- 출력: output/final.mp4

## 8.4 에이전트 실행 방식

---

Claude Code에서 각 에이전트는 Agent 도구로 호출된다. 사용자가 직접 에이전트를 호출할 필요는 없다. "/create-c"로 시작하면 Director가 자동으로 적절한 에이전트를 배정한다.

수동으로 특정 에이전트를 호출하고 싶다면:

리서치만 다시 해줘

또는

대본만 다시 써줘

라고 요청하면 해당 에이전트만 실행된다.

## 제9장. 자주 하는 작업 명령어 모음

### 9.1 콘텐츠 제작 시작

```
/create-c
```

또는

마이크 타이슨 주제로 4분짜리 영상 만들어줘

### 9.2 이미지 생성

스크립트로:

```
python3 scripts/generate_images.py --project projects/niche-bending/ep02-mike-tyson
```

Claude Code로:

썸 이미지 생성해줘

### 9.3 영상 생성

스크립트로:

```
python3 scripts/produce.py --project projects/niche-bending/ep02-mike-tyson --step video
```

Claude Code로:

Veo3로 ACT3-5 영상 생성해줘

### 9.4 나레이션 생성

무료 TTS 서비스 앱이 실행 중인지 확인한 후:

Claude Code로:

나레이션 생성해줘

## 9.5 CapCut 프로젝트 생성

```
python3 "/Users/yoogeon/Agents/contents_agent/scripts/capcut-project-gen.py" \
 projects/niche-bending/ep02-mike-tyson/capcut-config.json
```

## 9.6 영상 합성 (MoviePy)

```
python3 scripts/compositor.py \
 --project projects/niche-bending/ep02-mike-tyson \
 --format landscape
```

## 9.7 테스트

```
pip install -e ".[dev]" && pytest
```

## 9.8 이미지/영상 확인

Claude Code로:

```
이미지 확인해줘
```

```
scene-16 이미지 다시 봐줘
```

## 9.9 세션 이어받기

새 Claude Code 세션을 시작할 때:

```
/session-resume
```

또는

```
SESSION_HANDOVER.md 읽고 이어서 작업해줘
```

## 9.10 세션 종료

```
/handoff
```



이 명령은 현재까지의 작업을 정리하고 SESSION\_HANOVER.md를 갱신한다.

## 제10장. 핵심 규칙 요약 체크리스트

### 10.1 제작 전 체크

- [ ] brand/ YAML 4개 파일이 채워져 있는가?
- [ ] config/api-keys.env에 API 키가 설정되어 있는가?
- [ ] TTS 서비스가 정상 동작하는가?
- [ ] ffprobe가 설치되어 있는가? (which ffprobe)
- [ ] 러닝타임, 도구, 씬 수, 단어 수 계산이 완료되었는가?

### 10.2 이미지 생성 체크

- [ ] 캐릭터 앵커 이미지가 시대별로 생성되었는가?
- [ ] 모든 프롬프트에 인종/피부색/체형이 명시되어 있는가?
- [ ] 씬의 시대에 맞는 앵커를 참조하고 있는가?
- [ ] 생성된 이미지를 직접 확인했는가?
- [ ] 인종 불일치, 나이 오류, 배경 오류가 없는가?

### 10.3 영상 생성 체크

- [ ] Image-to-Video 프롬프트에서 이미지 내용을 재설명하지 않았는가?
- [ ] 움직임 끝 상태(then settles)가 명시되어 있는가?
- [ ] 동시 동작이 2개 이하인가?
- [ ] 물리 키워드(realistic physics 등)가 포함되어 있는가?
- [ ] 네거티브 프롬프트가 포함되어 있는가?
- [ ] video-prompts.md에 모든 프롬프트가 기록되어 있는가?

### 10.4 대본 체크

- [ ] 채널 언어에 맞는 말투인가? (영어: 미국 구어체, 한국어: 캐릭터 말투)
- [ ] HOOK과 본편에 동일 문장이 반복되지 않는가?
- [ ] 씬당 단어 수가 계산된 범위 내인가?
- [ ] 원본 대본 + 검수용 대본 두 파일 모두 생성되었는가?
- [ ] 팩트(날짜, 수치)가 정확한가?

## 10.5 나레이션 체크

- ☐ 씬 단위로 개별 생성했는가? (ACT 연속 아님)
- ☐ 모든 WAV 파일의 duration이 0이 아닌가?
- ☐ 보이스가 채널 고정 프로필인가? (TTS 보이스 프로필 확인)

## 10.6 SRT 자막 체크

- ☐ 플레인 텍스트만인가? (메타데이터 없음)
- ☐ 자막 1개당 1줄인가? (2줄 이상 없음)
- ☐ CapCut 자막 스타일이 설정에 반영되었는가?

## 10.7 CapCut 체크

- ☐ capcut-project-gen.py를 사용했는가? (수동 JSON 아님)
- ☐ config.json의 모든 경로가 절대 경로인가?
- ☐ CapCut을 종료한 상태에서 스크립트를 실행했는가?
- ☐ CapCut에서 프로젝트가 정상적으로 열리는가?
- ☐ SRT 자막을 수동 임포트했는가?
- ☐ 영상/오디오 싱크가 맞는가?

## 제11장. 용어 사전

---

ACT: 여러 씬을 묶은 큰 단위. 서사의 전환점으로 구분한다. 예: ACT 1(소년기), ACT 2(전성기), ACT 3(몰락), ACT 4(감옥), ACT 5(부활)

캐릭터 앵커 (Character Anchor): 씬별 이미지의 인물 외모를 일관되게 유지하기 위한 기준 이미지. 실존 인물은 시대별로 여러 개 만든다.

CTA (Call to Action): 영상 끝에 시청자에게 행동을 요청하는 것. "구독해주세요", "좋아요 눌러주세요" 등.

draft\_info.json: CapCut 프로젝트의 타임라인 전체 데이터를 담은 JSON 파일. 절대 수동으로 만들지 않는다. capcut-project-gen.py가 자동 생성한다.

HOOK: 영상의 첫 3초. 시청자를 잡아두는 오프닝. 본편과 같은 문장을 반복하면 안 된다.

I2V (Image-to-Video): 정지 이미지를 움직이는 영상으로 변환하는 것. Veo 3, Grok이 이 기능을 제공한다.

Phase 1: 브랜드 세팅 단계. 채널의 캐릭터, 서사, 주제, 벤치마크를 정의한다. 채널당 1회만 실행한다.

Phase 2: 콘텐츠 제작 단계. 리서치 → 이미지 → 영상 → 대본 → 나레이션 → 자막 → CapCut. 에피소드당 1회 실행한다.

SRT: 자막 파일 형식. 번호, 시간 범위, 텍스트로 구성된 매우 단순한 텍스트 파일.

TTS (Text-to-Speech): 텍스트를 음성으로 변환하는 기술. 무료 TTS 서비스가 이 역할을 한다.

TTS (Text-to-Speech) 서비스: 텍스트를 음성으로 변환하는 도구. Edge TTS (무료, 고품질), Google TTS (무료), Coqui TTS (오픈소스, 보이스 클론) 등을 사용한다.

Video-First: 영상 러닝타임을 먼저 정하고 거기서 역산하여 씬 수와 대본 길이를 결정하는 제작 방식. 이 프로젝트의 핵심 철학이다.

WPM (Words Per Minute): 분당 단어 수. 영어 나레이션은 150 WPM, 한국어는 약 300음절/분을 기준으로 한다.

# 제12장. 빠른 시작 가이드 (Quick Start)

전체 과정을 10단계로 요약한다.

1. 환경 설정  
Python 3.10, ffmpeg, CapCut, 무료 TTS 서비스 설치  
config/api-keys.env에 GEMINI\_API\_KEY, XAI\_API\_KEY 설정
2. Claude Code 실행  
cd /Users/yoogeon/Agents/auto-contents && claude
3. 콘텐츠 제작 시작  
/create-c 입력 → 주제 + 레퍼런스 채널 링크 전달
4. 벤치마킹 + 리서치 확인  
에이전트가 유사 메가 채널을 분석하고 체크리스트 생성 → 승인
5. 캐릭터 앵커 생성  
시대별 인물 레퍼런스 이미지 생성 (인종/피부색 명시 필수)
6. 씬별 이미지 생성  
Gemini Image Flash 3.1로 이미지 생성 → 확인 → 문제 있으면 재생성
7. 씬별 영상 생성  
Veo 3 또는 Grok으로 클립 생성
8. 대본 작성  
한국어 또는 영어, 씬당 단어 수 계산 범위 내로 작성
9. 나레이션 생성  
무료 TTS 서비스로 씬별 WAV 생성
10. CapCut 프로젝트 생성  
capcut-project-gen.py 실행 → CapCut에서 미세 조정 → 완성

이 과정에서 가장 중요한 3가지:

- 인종/피부색을 모든 이미지 프롬프트에 명시할 것
- Video-First 플로우를 따를 것 (러닝타임 먼저, 대본은 나중)
- draft\_info.json은 절대 수동으로 만들지 말 것